

```
src/ssscreen/__init__.py -> src/ssscreen/cli/app.py
```

```
F0001 0001 | """ss-screen — screen materials pairs that form solid-solutions with tunable band gaps."""
```

```
F0002 0002 |
```

```
F0003 0003 | __version__ = "0.1.0"
```

```
F0004 0001 | """CLI entry point: ``ss-screen``."""
```

```
F0005 0002 |
```

```
F0006 0003 | from .app import cli
```

```
F0007 0004 |
```

```
F0008 0005 | __all__ = ["cli"]
```

```
F0009 0001 | """Command-line interface: ``ss-screen``.
```

```
F0010 0002 |
```

```
F0011 0003 | Subcommands mirroring the implemented pipeline stages:
```

```
F0012 0004 |
```

```
F0013 0005 | ``ss-screen valence-filter`` pre-filter metallic / mixed-valence structures
```

```
F0014 0006 | ``ss-screen dataset mp`` build normalized MP dataset DataFrames
```

```
F0015 0007 | ``ss-screen composition-screen`` pre-screen composition-template candidates
```

```
F0016 0008 | ``ss-screen condense`` build robocrys condensed JSON inputs
```

```
F0017 0009 | ``ss-screen structure-match`` match candidates by condensed structure archive
```

```
F0018 0010 | ``ss-screen gap-export`` export candidates for external gap calculations
```

```
F0019 0011 | ``ss-screen gap-compare`` compare final pairs across gap methods
```

```
F0020 0012 | ``ss-screen stability`` generate optional stability-screening inputs
```

```
F0021 0013 | ``ss-screen recommend`` rank candidates from Stage 5 and Stage 8-10 evidence
```

```
F0022 0014 | ``ss-screen group`` group by composition template + environment
```

```
F0023 0015 | ``ss-screen pair`` enumerate promising alloying pairs from gaps
```

```
F0024 0016 |
```

```
F0025 0017 | Each command surfaces the notebook magic numbers as ``--options`` with sensible
```

```
F0026 0018 | defaults (the research values). All paths are CLI arguments — nothing is
```

```
F0027 0019 | hard-coded.
```

```
F0028 0020 | """
```

```
F0029 0021 |
```

```
F0030 0022 | from __future__ import annotations
```

```
F0031 0023 |
```

```
F0032 0024 | from pathlib import Path
```

```
F0033 0025 |
```

```
F0034 0026 | import click
```

```
F0035 0027 | import pandas as pd
```

```
F0036 0028 |
```

```
F0037 0029 | from ..config import (
```

```
F0038 0030 |     CompetingPhaseSettings,
```

```
F0039 0031 |     MLPRelaxationSettings,
```

```
F0040 0032 |     PhononSettings,
```

```
F0041 0033 |     RecommendationSettings,
```

```
F0042 0034 |     Thresholds,
```

```
F0043 0035 |     coerce_excluded,
```

```
F0044 0036 | )
```

```
F0045 0037 | from ..data.io import (
```

```
F0046 0038 |     CondenseLoader,
```

```
F0047 0039 |     dump_group_df,
```

```
F0048 0040 |     load_group_df,
```

```
F0049 0041 |     read_mbj_gaps,
```

```
F0050 0042 |     write_pairs_csv,
```

```
src/ssscreen/cli/app.py
```

```
F0051 0043 | )
```

```
F0052 0044 | from ..pair.envmatch import find_unique_envs, group_similar_structures
```

```
F0053 0045 | from ..pair.filters import apply_element_exclusion, apply_size_gate, has_gap_diversity
```

```
F0054 0046 | from ..pair.grouping import (
```

```
F0055 0047 |     attach_band_gaps,
```

```
F0056 0048 |     group_by_composition_template,
```

```
F0057 0049 |     screen_composition_candidates,
```

```
F0058 0050 | )
```

```
F0059 0051 | from ..pair.pairing import enumerate_pairs, pairs_to_dataframe
```

```
F0060 0052 |
```

```
F0061 0053 |
```

```
F0062 0054 | @click.group()
```

```
F0063 0055 | @click.version_option(package_name="ss-screen")
```

```
F0064 0056 | def cli() -> None:
```

```
F0065 0057 |     """Screen materials pairs for solid-solution formation."""
```

```
F0066 0058 |
```

```
F0067 0059 |
```

```
F0068 0060 | def _parse_supercell(value: str) -> tuple[int, int, int]:
```

```
F0069 0061 |     parts = [part.strip() for part in value.replace("x", ", ").split(", ")]
```

```
F0070 0062 |     if len(parts) != 3:
```

```
F0071 0063 |         raise click.BadParameter("expected three integers, for example 2,2,2")
```

```
F0072 0064 |     try:
```

```
F0073 0065 |         supercell = tuple(int(part) for part in parts)
```

```
F0074 0066 |     except ValueError as exc:
```

```
F0075 0067 |         raise click.BadParameter("expected three integers, for example 2,2,2") from exc
```

```
F0076 0068 |     if any(dim <= 0 for dim in supercell):
```

```
F0077 0069 |         raise click.BadParameter("supercell dimensions must be positive")
```

```
F0078 0070 |     return supercell
```

```
F0079 0071 |
```

```
F0080 0072 |
```

```
F0081 0073 | def _parse_optional_supercell(
```

```
F0082 0074 |     _ctx: click.Context, _param: click.Parameter, value: str | None
```

```
F0083 0075 | ) -> tuple[int, int, int] | None:
```

```
F0084 0076 |     return None if value is None else _parse_supercell(value)
```

```
F0085 0077 |
```

```
F0086 0078 |
```

```
F0087 0079 | def _prompt_mp_api_key() -> str:
```

```
F0088 0080 |     """Read an MP API key without echoing it or accepting a leaky CLI option."""
```

```
F0089 0081 |     return click.prompt("Materials Project API key", hide_input=True, type=str)
```

```
F0090 0082 |
```

```
F0091 0083 |
```

```
F0092 0084 | # -----
```

```
F0093 0085 | # dataset
```

```
F0094 0086 | # -----
```

```
F0095 0087 | @cli.group("dataset")
```

```
F0096 0088 | def dataset_cmd() -> None:
```

```
F0097 0089 |     """Build normalized source dataset DataFrames."""
```

```
F0098 0090 |
```

```
F0099 0091 |
```

```
F0100 0092 | @dataset_cmd.command("mp")
```

```
src/ssscreen/cli/app.py
```

```
F0101 0093 | @click.option(
F0102 0094 |     "--backend",
F0103 0095 |     type=click.Choice(["offline", "api"]),
F0104 0096 |     default="offline",
F0105 0097 |     show_default=True,
F0106 0098 |     help="Materials Project source backend.",
F0107 0099 | )
F0108 0100 | @click.option(
F0109 0101 |     "--offline-db",
F0110 0102 |     "offline_db",
F0111 0103 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F0112 0104 |     help="Optional mp_offline SQLite database path.",
F0113 0105 | )
F0114 0106 | @click.option(
F0115 0107 |     "--output",
F0116 0108 |     "output",
F0117 0109 |     type=click.Path(path_type=Path),
F0118 0110 |     required=True,
F0119 0111 |     help="Where to write the normalized MP pickle DataFrame.",
F0120 0112 | )
F0121 0113 | @click.option(
F0122 0114 |     "--provenance",
F0123 0115 |     "provenance_path",
F0124 0116 |     type=click.Path(path_type=Path),
F0125 0117 |     help="MP dataset provenance JSON (default: <output>.provenance.json).",
F0126 0118 | )
F0127 0119 | @click.option(
F0128 0120 |     "--max-e-hull",
F0129 0121 |     "max_e_hull",
F0130 0122 |     type=float,
F0131 0123 |     default=0.01,
F0132 0124 |     show_default=True,
F0133 0125 |     help="Maximum MP energy above hull to request (eV/atom).",
F0134 0126 | )
F0135 0127 | def dataset_mp_cmd(
F0136 0128 |     backend: str,
F0137 0129 |     offline_db: Path | None,
F0138 0130 |     output: Path,
F0139 0131 |     provenance_path: Path | None,
F0140 0132 |     max_e_hull: float,
F0141 0133 | ) -> None:
F0142 0134 |     """Fetch Materials Project summaries into the screening schema."""
F0143 0135 |     from ..data.mp import load_mp_dataset
F0144 0136 |
F0145 0137 |     try:
F0146 0138 |         df = load_mp_dataset(
F0147 0139 |             output=output,
F0148 0140 |             max_e_hull=max_e_hull,
F0149 0141 |             backend=backend,
F0150 0142 |             offline_db=offline_db,
```

```
src/ssscreen/cli/app.py
```

```
F0151 0143 |         provenance_path=provenance_path,
F0152 0144 |     )
F0153 0145 |     except (ImportError, FileNotFoundError, ValueError) as exc:
F0154 0146 |         raise click.ClickException(str(exc)) from exc
F0155 0147 |
F0156 0148 |     from ..data.mp import default_mp_provenance_path
F0157 0149 |
F0158 0150 |     sidecar = provenance_path or default_mp_provenance_path(output)
F0159 0151 |     click.echo(f"Wrote {len(df)} MP rows to {output}; provenance={sidecar}")
F0160 0152 |
F0161 0153 |
F0162 0154 | @dataset_cmd.command("wbm")
F0163 0155 | @click.option(
F0164 0156 |     "--xyz",
F0165 0157 |     "xyz_path",
F0166 0158 |     type=click.Path(exists=True, path_type=Path),
F0167 0159 |     required=True,
F0168 0160 |     help="WBM extxyz file, e.g. wbm-dataset.xyz.",
F0169 0161 | )
F0170 0162 | @click.option(
F0171 0163 |     "--summary",
F0172 0164 |     "summary_path",
F0173 0165 |     type=click.Path(exists=True, path_type=Path),
F0174 0166 |     help="Optional WBM summary TSV/CSV file.",
F0175 0167 | )
F0176 0168 | @click.option(
F0177 0169 |     "--output",
F0178 0170 |     "output",
F0179 0171 |     type=click.Path(path_type=Path),
F0180 0172 |     required=True,
F0181 0173 |     help="Where to write the normalized WBM pickle DataFrame.",
F0182 0174 | )
F0183 0175 | def dataset_wbm_cmd(xyz_path: Path, summary_path: Path | None, output: Path) -> None:
F0184 0176 |     """Load WBM extxyz data into the screening schema."""
F0185 0177 |     from ..data.wbm import load_wbm_dataset
F0186 0178 |
F0187 0179 |     try:
F0188 0180 |         df = load_wbm_dataset(xyz_path=xyz_path, output=output, summary_path=summary_path)
F0189 0181 |     except ImportError as exc:
F0190 0182 |         raise click.ClickException(str(exc)) from exc
F0191 0183 |
F0192 0184 |     click.echo(f"Wrote {len(df)} WBM rows to {output}")
F0193 0185 |
F0194 0186 |
F0195 0187 | # -----
F0196 0188 | # valence-filter
F0197 0189 | # -----
F0198 0190 | @cli.command("valence-filter")
F0199 0191 | @click.option(
F0200 0192 |     "--df-mp",
```

```
src/ssscreen/cli/app.py
```

```
F0201 0193 |     "df_mp",
F0202 0194 |     type=click.Path(exists=True, path_type=Path),
F0203 0195 |     help="Pickled MP dataset DataFrame (e.g. mp.df).",
F0204 0196 | )
F0205 0197 | @click.option(
F0206 0198 |     "--df_wbm",
F0207 0199 |     "df_wbm",
F0208 0200 |     type=click.Path(exists=True, path_type=Path),
F0209 0201 |     help="Pickled WBM dataset DataFrame (e.g. wbm-dataset.df).",
F0210 0202 | )
F0211 0203 | @click.option(
F0212 0204 |     "--output",
F0213 0205 |     "output",
F0214 0206 |     type=click.Path(path_type=Path),
F0215 0207 |     required=True,
F0216 0208 |     help="Where to write the JSON list of valid material IDs.",
F0217 0209 | )
F0218 0210 | @click.option("--no-verbose", is_flag=True, default=False, help="Disable the progress bar.")
F0219 0211 | def valence_filter_cmd(
F0220 0212 |     df_mp: Path | None, df_wbm: Path | None, output: Path, no_verbose: bool
F0221 0213 | ) -> None:
F0222 0214 |     """Pre-filter metallic / mixed-valence structures via bond-valence analysis.
F0223 0215 |
F0224 0216 |     Loads one or both pickled DataFrames (indexed by material ID, with a
F0225 0217 |     ``structure`` column of pymatgen Structures) and writes the IDs whose
F0226 0218 |     oxidation states BVAnalyzer can assign.
F0227 0219 |     """
F0228 0220 |     from monty.serialization import dumpfn
F0229 0221 |
F0230 0222 |     from ..pair.filters import valence_filter
F0231 0223 |
F0232 0224 |     frames = []
F0233 0225 |     if df_mp:
F0234 0226 |         frames.append(pd.read_pickle(df_mp))
F0235 0227 |     if df_wbm:
F0236 0228 |         frames.append(pd.read_pickle(df_wbm))
F0237 0229 |     if not frames:
F0238 0230 |         raise click.UsageError("Provide at least one of --df-mp / --df-wbm.")
F0239 0231 |     df = pd.concat(frames, axis=0)
F0240 0232 |
F0241 0233 |     valid = valence_filter(df, verbose=not no_verbose)
F0242 0234 |     dumpfn(valid, str(output))
F0243 0235 |     click.echo(f"Wrote {len(valid)} valence-valid IDs to {output}")
F0244 0236 |
F0245 0237 |
F0246 0238 | # -----
F0247 0239 | # condense
F0248 0240 | # -----
F0249 0241 | @cli.command("condense")
F0250 0242 | @click.option(
```

```
src/ssscreen/cli/app.py
```

```
F0251 0243 |     "--input",
F0252 0244 |     "inputs",
F0253 0245 |     multiple=True,
F0254 0246 |     type=click.Path(exists=True, path_type=Path),
F0255 0247 |     help="Structure file or directory to condense (repeatable).",
F0256 0248 | )
F0257 0249 | @click.option(
F0258 0250 |     "--df",
F0259 0251 |     "df_path",
F0260 0252 |     type=click.Path(exists=True, path_type=Path),
F0261 0253 |     help="Pickled normalized dataset DataFrame to condense.",
F0262 0254 | )
F0263 0255 | @click.option(
F0264 0256 |     "--structure-column",
F0265 0257 |     "structure_col",
F0266 0258 |     default="structure",
F0267 0259 |     show_default=True,
F0268 0260 |     help="DataFrame column containing pymatgen Structure objects.",
F0269 0261 | )
F0270 0262 | @click.option(
F0271 0263 |     "--material-id-column",
F0272 0264 |     "material_id_col",
F0273 0265 |     help="Optional DataFrame column for material IDs; default uses the DataFrame index.",
F0274 0266 | )
F0275 0267 | @click.option(
F0276 0268 |     "--output-dir",
F0277 0269 |     "output_dir",
F0278 0270 |     type=click.Path(path_type=Path),
F0279 0271 |     required=True,
F0280 0272 |     help="Directory for {material_id}.json condensed outputs.",
F0281 0273 | )
F0282 0274 | @click.option("--manifest", "manifest_path", type=click.Path(path_type=Path))
F0283 0275 | @click.option("--index", "index_path", type=click.Path(path_type=Path))
F0284 0276 | @click.option("--limit", type=int, help="Limit number of DataFrame rows to condense.")
F0285 0277 | @click.option("--overwrite", is_flag=True, default=False, help="Overwrite existing outputs.")
F0286 0278 | @click.option(
F0287 0279 |     "--stop-on-error",
F0288 0280 |     is_flag=True,
F0289 0281 |     default=False,
F0290 0282 |     help="Stop on the first condensation error instead of recording failures.",
F0291 0283 | )
F0292 0284 | def condense_cmd(
F0293 0285 |     inputs: tuple[Path, ...],
F0294 0286 |     df_path: Path | None,
F0295 0287 |     structure_col: str,
F0296 0288 |     material_id_col: str | None,
F0297 0289 |     output_dir: Path,
F0298 0290 |     manifest_path: Path | None,
F0299 0291 |     index_path: Path | None,
F0300 0292 |     limit: int | None,
```

```
src/ssscreen/cli/app.py
```

```
F0301 0293 |     overwrite: bool,
F0302 0294 |     stop_on_error: bool,
F0303 0295 | ) -> None:
F0304 0296 |     """Build robocrys condensed-structure JSON files."""
F0305 0297 |     from ..data.condense import condense_dataframe, condense_paths
F0306 0298 |
F0307 0299 |     if bool(inputs) == bool(df_path):
F0308 0300 |         raise click.UsageError("Provide exactly one of --input or --df.")
F0309 0301 |     try:
F0310 0302 |         if df_path:
F0311 0303 |             summary = condense_dataframe(
F0312 0304 |                 df_path=df_path,
F0313 0305 |                 output_dir=output_dir,
F0314 0306 |                 structure_col=structure_col,
F0315 0307 |                 material_id_col=material_id_col,
F0316 0308 |                 limit=limit,
F0317 0309 |                 overwrite=overwrite,
F0318 0310 |                 manifest_path=manifest_path,
F0319 0311 |                 index_path=index_path,
F0320 0312 |                 continue_on_error=not stop_on_error,
F0321 0313 |             )
F0322 0314 |         else:
F0323 0315 |             summary = condense_paths(
F0324 0316 |                 list(inputs),
F0325 0317 |                 output_dir,
F0326 0318 |                 overwrite=overwrite,
F0327 0319 |                 manifest_path=manifest_path,
F0328 0320 |                 index_path=index_path,
F0329 0321 |                 continue_on_error=not stop_on_error,
F0330 0322 |             )
F0331 0323 |     except ImportError as exc:
F0332 0324 |         raise click.ClickException(str(exc)) from exc
F0333 0325 |
F0334 0326 |     click.echo(
F0335 0327 |         "Condense summary: "
F0336 0328 |         f"written={summary.written} skipped={summary.skipped} failed={summary.failed}"
F0337 0329 |     )
F0338 0330 |
F0339 0331 |
F0340 0332 | @cli.command("condense-index")
F0341 0333 | @click.option(
F0342 0334 |     "--condensed-dir",
F0343 0335 |     type=click.Path(exists=True, file_okay=False, path_type=Path),
F0344 0336 |     required=True,
F0345 0337 | )
F0346 0338 | @click.option("--output", type=click.Path(path_type=Path), required=True)
F0347 0339 | def condense_index_cmd(condensed_dir: Path, output: Path) -> None:
F0348 0340 |     """Build a CSV index for a condensed-structure JSON directory."""
F0349 0341 |     from ..data.condense import build_archive_index
F0350 0342 |
```

```
src/sssreen/cli/app.py
F0351 0343 | index = build_archive_index(condensed_dir, output)
F0352 0344 | click.echo(f"Indexed {len(index)} condensed files to {output}")
F0353 0345 |
F0354 0346 |
F0355 0347 | @cli.command("condense-validate")
F0356 0348 | @click.option(
F0357 0349 |     "--condensed-dir",
F0358 0350 |     type=click.Path(exists=True, file_okay=False, path_type=Path),
F0359 0351 |     required=True,
F0360 0352 | )
F0361 0353 | @click.option("--output", type=click.Path(path_type=Path), required=True)
F0362 0354 | def condense_validate_cmd(condensed_dir: Path, output: Path) -> None:
F0363 0355 |     """Validate required robocrys keys in a condensed-structure archive."""
F0364 0356 |     from ..data.condense import validate_archive
F0365 0357 |
F0366 0358 |     validation = validate_archive(condensed_dir, index_path=output)
F0367 0359 |     counts = validation["status"].value_counts().to_dict() if len(validation) else {}
F0368 0360 |     click.echo(
F0369 0361 |         f"Validation summary: valid={counts.get('valid', 0)} "
F0370 0362 |         f"invalid={counts.get('invalid', 0)} output={output}"
F0371 0363 |     )
F0372 0364 |
F0373 0365 |
F0374 0366 | # -----
F0375 0367 | # structure-match
F0376 0368 | # -----
F0377 0369 | @cli.command("structure-match")
F0378 0370 | @click.option(
F0379 0371 |     "--candidates",
F0380 0372 |     type=click.Path(exists=True, path_type=Path),
F0381 0373 |     required=True,
F0382 0374 |     help="Composition-candidate CSV from `ss-screen composition-screen`. ",
F0383 0375 | )
F0384 0376 | @click.option(
F0385 0377 |     "--condensed-dir",
F0386 0378 |     "condensed_dirs",
F0387 0379 |     multiple=True,
F0388 0380 |     required=True,
F0389 0381 |     type=click.Path(exists=True, file_okay=False, path_type=Path),
F0390 0382 |     help="Condensed JSON archive directory (repeatable). ",
F0391 0383 | )
F0392 0384 | @click.option("--min-x-elements", type=int, default=2, show_default=True)
F0393 0385 | @click.option(
F0394 0386 |     "--output",
F0395 0387 |     type=click.Path(path_type=Path),
F0396 0388 |     required=True,
F0397 0389 |     help="Output structure-group JSON. ",
F0398 0390 | )
F0399 0391 | @click.option(
F0400 0392 |     "--summary",
```

```
src/ssscreen/cli/app.py
```

```
F0401 0393 |     type=click.Path(path_type=Path),
F0402 0394 |     help="Optional JSON structure-match summary.",
F0403 0395 | )
F0404 0396 | def structure_match_cmd(
F0405 0397 |     candidates: Path,
F0406 0398 |     condensed_dirs: tuple[Path, ...],
F0407 0399 |     min_x_elements: int,
F0408 0400 |     output: Path,
F0409 0401 |     summary: Path | None,
F0410 0402 | ) -> None:
F0411 0403 |     """Match composition candidates by robocrys environment descriptors."""
F0412 0404 |     from monty.serialization import dumpfn
F0413 0405 |
F0414 0406 |     from ..pair.structure_match import match_structure_candidates
F0415 0407 |
F0416 0408 |     groups, report = match_structure_candidates(
F0417 0409 |         candidates,
F0418 0410 |         list(condensed_dirs),
F0419 0411 |         min_x_elements=min_x_elements,
F0420 0412 |     )
F0421 0413 |     output.parent.mkdir(parents=True, exist_ok=True)
F0422 0414 |     dump_group_df(groups, output)
F0423 0415 |     if summary:
F0424 0416 |         summary.parent.mkdir(parents=True, exist_ok=True)
F0425 0417 |         dumpfn(report, str(summary))
F0426 0418 |     click.echo(
F0427 0419 |         f"Wrote {len(groups)} structure groups to {output} "
F0428 0420 |         f"(missing_descriptions={report['missing_descriptions']})"
F0429 0421 |     )
F0430 0422 |
F0431 0423 |
F0432 0424 | # -----
F0433 0425 | # gap-export / gap-validate
F0434 0426 | # -----
F0435 0427 | @cli.command("gap-export")
F0436 0428 | @click.option(
F0437 0429 |     "--groups", "groups_path", type=click.Path(exists=True, path_type=Path), required=True
F0438 0430 | )
F0439 0431 | @click.option(
F0440 0432 |     "--dataset",
F0441 0433 |     "dataset_path",
F0442 0434 |     type=click.Path(exists=True, path_type=Path),
F0443 0435 |     required=True,
F0444 0436 |     help="Normalized dataset pickle containing source structures.",
F0445 0437 | )
F0446 0438 | @click.option("--method", default="external", show_default=True)
F0447 0439 | @click.option("--structure-dir", type=click.Path(path_type=Path))
F0448 0440 | @click.option(
F0449 0441 |     "--structure-format",
F0450 0442 |     "structure_formats",
```

```
src/ssscreen/cli/app.py
```

```
F0451 0443 |     multiple=True,
F0452 0444 |     type=click.Choice(["json", "cif", "poscar"]),
F0453 0445 |     default=("json", "cif", "poscar"),
F0454 0446 |     show_default=True,
F0455 0447 |     help="Portable structure format(s) written under --structure-dir.",
F0456 0448 | )
F0457 0449 | @click.option("--results-template", type=click.Path(path_type=Path))
F0458 0450 | @click.option("--method-metadata-template", type=click.Path(path_type=Path))
F0459 0451 | @click.option("--structure-column", "structure_col", default="structure", show_default=True)
F0460 0452 | @click.option("--max-natoms", type=int, default=30, show_default=True)
F0461 0453 | @click.option("--output", type=click.Path(path_type=Path), required=True)
F0462 0454 | def gap_export_cmd(
F0463 0455 |     groups_path: Path,
F0464 0456 |     dataset_path: Path,
F0465 0457 |     method: str,
F0466 0458 |     structure_dir: Path | None,
F0467 0459 |     structure_formats: tuple[str, ...],
F0468 0460 |     results_template: Path | None,
F0469 0461 |     method_metadata_template: Path | None,
F0470 0462 |     structure_col: str,
F0471 0463 |     max_natoms: int | None,
F0472 0464 |     output: Path,
F0473 0465 | ) -> None:
F0474 0466 |     """Export matched group members for external high-level gap calculations."""
F0475 0467 |     from ..pair.gap_export import export_gap_candidates
F0476 0468 |
F0477 0469 |     exported = export_gap_candidates(
F0478 0470 |         groups_path=groups_path,
F0479 0471 |         dataset_path=dataset_path,
F0480 0472 |         output=output,
F0481 0473 |         method=method,
F0482 0474 |         structure_dir=structure_dir,
F0483 0475 |         structure_formats=structure_formats,
F0484 0476 |         results_template=results_template,
F0485 0477 |         method_metadata_template=method_metadata_template,
F0486 0478 |         structure_col=structure_col,
F0487 0479 |         max_natoms=max_natoms,
F0488 0480 |     )
F0489 0481 |     click.echo(f"Wrote {len(exported)} gap candidates to {output}")
F0490 0482 |
F0491 0483 |
F0492 0484 | @cli.command("gap-validate")
F0493 0485 | @click.option("--gaps", type=click.Path(exists=True, path_type=Path), required=True)
F0494 0486 | @click.option("--tasks", "tasks_path", type=click.Path(exists=True, path_type=Path))
F0495 0487 | @click.option(
F0496 0488 |     "--method-metadata",
F0497 0489 |     "method_metadata_path",
F0498 0490 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F0499 0491 | )
F0500 0492 | @click.option("--output", "output_path", type=click.Path(path_type=Path))
```

```
src/ssscreen/cli/app.py
```

```
F0501 0493 | @click.option("--rejected", "rejected_path", type=click.Path(path_type=Path))
F0502 0494 | @click.option("--report", "report_path", type=click.Path(path_type=Path))
F0503 0495 | def gap_validate_cmd(
F0504 0496 |     gaps: Path,
F0505 0497 |     tasks_path: Path | None,
F0506 0498 |     method_metadata_path: Path | None,
F0507 0499 |     output_path: Path | None,
F0508 0500 |     rejected_path: Path | None,
F0509 0501 |     report_path: Path | None,
F0510 0502 | ) -> None:
F0511 0503 |     """Validate and normalize externally calculated band-gap results."""
F0512 0504 |     from ..pair.gap_export import validate_gap_results
F0513 0505 |
F0514 0506 |     results = validate_gap_results(
F0515 0507 |         gaps,
F0516 0508 |         tasks_path=tasks_path,
F0517 0509 |         method_metadata_path=method_metadata_path,
F0518 0510 |         output_path=output_path,
F0519 0511 |         rejected_path=rejected_path,
F0520 0512 |         report_path=report_path,
F0521 0513 |     )
F0522 0514 |     report = results.attrs.get(
F0523 0515 |         "validation_report",
F0524 0516 |         {"accepted_count": len(results), "rejected_count": 0},
F0525 0517 |     )
F0526 0518 |     click.echo(
F0527 0519 |         "Gap-result validation: "
F0528 0520 |         f"accepted={report['accepted_count']} rejected={report['rejected_count']} source={gaps}"
F0529 0521 |     )
F0530 0522 |     if report["rejected_count"]:
F0531 0523 |         raise click.ClickException(
F0532 0524 |             f"gap validation rejected {report['rejected_count']} malformed row(s)"
F0533 0525 |         )
F0534 0526 |
F0535 0527 |
F0536 0528 | @cli.command("gap-compare")
F0537 0529 | @click.option(
F0538 0530 |     "--groups", "groups_path", type=click.Path(exists=True, path_type=Path), required=True
F0539 0531 | )
F0540 0532 | @click.option(
F0541 0533 |     "--gap-results",
F0542 0534 |     "gap_results",
F0543 0535 |     multiple=True,
F0544 0536 |     required=True,
F0545 0537 |     type=click.Path(exists=True, path_type=Path),
F0546 0538 |     help="External gap-result CSV using the stable schema from `gap-validate`. ",
F0547 0539 | )
F0548 0540 | @click.option("--low-gap", "low_gap", type=float, default=0.15, show_default=True)
F0549 0541 | @click.option("--direct-min", "direct_min", type=float, default=0.15, show_default=True)
F0550 0542 | @click.option("--direct-max", "direct_max", type=float, default=1.5, show_default=True)
```

```
src/ssscreen/cli/app.py
```

```
F0551 0543 | @click.option("--pair-any-below", "pair_any_below", type=float, default=0.3, show_default=True)
F0552 0544 | @click.option("--pair-any-above", "pair_any_above", type=float, default=0.2, show_default=True)
F0553 0545 | @click.option("--pair-both-below", "pair_both_below", type=float, default=0.8, show_default=True)
F0554 0546 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F0555 0547 | @click.option("--summary", type=click.Path(path_type=Path), required=True)
F0556 0548 | def gap_compare_cmd(
F0557 0549 |     groups_path: Path,
F0558 0550 |     gap_results: tuple[Path, ...],
F0559 0551 |     low_gap: float,
F0560 0552 |     direct_min: float,
F0561 0553 |     direct_max: float,
F0562 0554 |     pair_any_below: float,
F0563 0555 |     pair_any_above: float,
F0564 0556 |     pair_both_below: float,
F0565 0557 |     output_dir: Path,
F0566 0558 |     summary: Path,
F0567 0559 | ) -> None:
F0568 0560 |     """Compare final pair outputs across external gap methods."""
F0569 0561 |     from ..config import PairThresholds
F0570 0562 |     from ..pair.gap_feedback import compare_gap_methods
F0571 0563 |
F0572 0564 |     report = compare_gap_methods(
F0573 0565 |         groups_path=groups_path,
F0574 0566 |         gap_paths=gap_results,
F0575 0567 |         output_dir=output_dir,
F0576 0568 |         summary=summary,
F0577 0569 |         thresholds=PairThresholds(
F0578 0570 |             low_gap=low_gap,
F0579 0571 |             direct_min=direct_min,
F0580 0572 |             direct_max=direct_max,
F0581 0573 |             pair_any_below=pair_any_below,
F0582 0574 |             pair_any_above=pair_any_above,
F0583 0575 |             pair_both_below=pair_both_below,
F0584 0576 |         ),
F0585 0577 |     )
F0586 0578 |     click.echo(f"Compared {len(report['methods'])} methods to {summary} (outputs={output_dir})")
F0587 0579 |
F0588 0580 |
F0589 0581 | # -----
F0590 0582 | # stability
F0591 0583 | # -----
F0592 0584 | @cli.group("stability")
F0593 0585 | def stability_cmd() -> None:
F0594 0586 |     """Generate optional preliminary stability-screening inputs."""
F0595 0587 |
F0596 0588 |
F0597 0589 | @stability_cmd.command("sqc-generate")
F0598 0590 | @click.option(
F0599 0591 |     "--pairs",
F0600 0592 |     "pairs_path",
```

```
src/sssreen/cli/app.py
```

```
F0601 0593 | type=click.Path(exists=True, path_type=Path),
F0602 0594 | required=True,
F0603 0595 | help="Final pair CSV from `ss-screen pair`.",
F0604 0596 | )
F0605 0597 | @click.option(
F0606 0598 |     "--dataset",
F0607 0599 |     "dataset_path",
F0608 0600 |     type=click.Path(exists=True, path_type=Path),
F0609 0601 |     required=True,
F0610 0602 |     help="Normalized dataset pickle containing endpoint structures.",
F0611 0603 | )
F0612 0604 | @click.option(
F0613 0605 |     "--target-fraction",
F0614 0606 |     "target_fractions",
F0615 0607 |     multiple=True,
F0616 0608 |     type=click.FloatRange(0.0, 1.0),
F0617 0609 |     default=(0.5,),
F0618 0610 |     show_default=True,
F0619 0611 |     help="Target fraction of endpoint B on the substituted site.",
F0620 0612 | )
F0621 0613 | @click.option(
F0622 0614 |     "--supercell",
F0623 0615 |     callback=lambda _ctx, _param, value: _parse_supercell(value),
F0624 0616 |     default="2,2,2",
F0625 0617 |     show_default=True,
F0626 0618 |     help="Diagonal supercell as 'a,b,c' or 'axbxc'.",
F0627 0619 | )
F0628 0620 | @click.option(
F0629 0621 |     "--backend",
F0630 0622 |     type=click.Choice(["random", "icet"]),
F0631 0623 |     default="random",
F0632 0624 |     show_default=True,
F0633 0625 |     help="Alloy structure generator backend.",
F0634 0626 | )
F0635 0627 | @click.option(
F0636 0628 |     "--cutoff",
F0637 0629 |     "cutoffs",
F0638 0630 |     multiple=True,
F0639 0631 |     type=float,
F0640 0632 |     default=(4.0,),
F0641 0633 |     show_default=True,
F0642 0634 |     help="icet cluster-space cutoff in Angstrom; repeat for higher orders.",
F0643 0635 | )
F0644 0636 | @click.option(
F0645 0637 |     "--sqc-steps",
F0646 0638 |     type=int,
F0647 0639 |     help="Number of icet/mchammer Monte Carlo steps; icet chooses a default if omitted.",
F0648 0640 | )
F0649 0641 | @click.option("--seed", type=int, default=0, show_default=True)
F0650 0642 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
```

src/ssscreen/cli/app.py

F0651 0643 | @click.option("--manifest", "manifest_path", type=click.Path(path_type=Path), required=True)

F0652 0644 | def stability_sqs_generate_cmd(

F0653 0645 | pairs_path: Path,

F0654 0646 | dataset_path: Path,

F0655 0647 | target_fractions: tuple[float, ...],

F0656 0648 | supercell: tuple[int, int, int],

F0657 0649 | backend: str,

F0658 0650 | cutoffs: tuple[float, ...],

F0659 0651 | sqs_steps: int | None,

F0660 0652 | seed: int,

F0661 0653 | output_dir: Path,

F0662 0654 | manifest_path: Path,

F0663 0655 |) -> None:

F0664 0656 | """Generate representative random-substitution alloy structures."""

F0665 0657 | from ..stability.sqs import generate_sqs_inputs

F0666 0658 |

F0667 0659 | records = generate_sqs_inputs(

F0668 0660 | pairs_path=pairs_path,

F0669 0661 | dataset_path=dataset_path,

F0670 0662 | output_dir=output_dir,

F0671 0663 | manifest_path=manifest_path,

F0672 0664 | target_fractions=target_fractions,

F0673 0665 | supercell=supercell,

F0674 0666 | seed=seed,

F0675 0667 | backend=backend,

F0676 0668 | cutoffs=cutoffs,

F0677 0669 | sqs_steps=sqs_steps,

F0678 0670 |)

F0679 0671 | statuses = pd.Series([record["status"] for record in records]).value_counts().to_dict()

F0680 0672 | click.echo(

F0681 0673 | f"SQS input summary: written={statuses.get('written', 0)} "

F0682 0674 | f"skipped={statuses.get('skipped', 0)} manifest={manifest_path}"

F0683 0675 |)

F0684 0676 |

F0685 0677 |

F0686 0678 | @stability_cmd.command("relax")

F0687 0679 | @click.option(

F0688 0680 | "--manifest",

F0689 0681 | "manifest_path",

F0690 0682 | type=click.Path(exists=True, path_type=Path),

F0691 0683 | required=True,

F0692 0684 | help="Stage 6 SQS JSONL manifest.",

F0693 0685 |)

F0694 0686 | @click.option(

F0695 0687 | "--include-endmembers/--no-include-endmembers",

F0696 0688 | default=False,

F0697 0689 | show_default=True,

F0698 0690 | help="Also relax de-duplicated endpoint A/B structures.",

F0699 0691 |)

F0700 0692 | @click.option(

```
src/ssscreen/cli/app.py
```

```
F0701 0693 |     "--dataset",
F0702 0694 |     "dataset_path",
F0703 0695 |     type=click.Path(exists=True, path_type=Path),
F0704 0696 |     help="Dataset fallback for older manifests without endpoint structure paths.",
F0705 0697 | )
F0706 0698 | @click.option(
F0707 0699 |     "--backend",
F0708 0700 |     type=click.Choice(["mace"]),
F0709 0701 |     default="mace",
F0710 0702 |     show_default=True,
F0711 0703 | )
F0712 0704 | @click.option(
F0713 0705 |     "--model-path",
F0714 0706 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F0715 0707 |     required=True,
F0716 0708 |     help="Local MACE checkpoint path.",
F0717 0709 | )
F0718 0710 | @click.option("--model-name", help="Optional stable model label stored in provenance.")
F0719 0711 | @click.option("--device", default="cpu", show_default=True, help="MACE device, e.g. cpu or cuda:0.")
F0720 0712 | @click.option(
F0721 0713 |     "--dtype",
F0722 0714 |     type=click.Choice(["float32", "float64"]),
F0723 0715 |     default="float32",
F0724 0716 |     show_default=True,
F0725 0717 | )
F0726 0718 | @click.option(
F0727 0719 |     "--fmax",
F0728 0720 |     type=click.FloatRange(min=0.0, min_open=True),
F0729 0721 |     default=MLPRelaxationSettings.force_tolerance,
F0730 0722 |     show_default=True,
F0731 0723 |     help="Force convergence tolerance in eV/Angstrom.",
F0732 0724 | )
F0733 0725 | @click.option(
F0734 0726 |     "--max-steps",
F0735 0727 |     type=click.IntRange(min=1),
F0736 0728 |     default=MLPRelaxationSettings.max_steps,
F0737 0729 |     show_default=True,
F0738 0730 | )
F0739 0731 | @click.option(
F0740 0732 |     "--relax-cell/--positions-only",
F0741 0733 |     default=True,
F0742 0734 |     show_default=True,
F0743 0735 |     help="Relax positions and periodic cell, or positions only.",
F0744 0736 | )
F0745 0737 | @click.option("--limit", type=click.IntRange(min=1), help="Process at most this many tasks.")
F0746 0738 | @click.option("--overwrite", is_flag=True, help="Replace results even when fingerprints differ.")
F0747 0739 | @click.option("--retry-failed", is_flag=True, help="Retry matching failed task records.")
F0748 0740 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F0749 0741 | @click.option("--results", "results_path", type=click.Path(path_type=Path), required=True)
F0750 0742 | def stability_relax_cmd(
```

```
src/ssscreen/cli/app.py
```

```
F0751 0743 | manifest_path: Path,
F0752 0744 | include_endmembers: bool,
F0753 0745 | dataset_path: Path | None,
F0754 0746 | backend: str,
F0755 0747 | model_path: Path,
F0756 0748 | model_name: str | None,
F0757 0749 | device: str,
F0758 0750 | dtype: str,
F0759 0751 | fmax: float,
F0760 0752 | max_steps: int,
F0761 0753 | relax_cell: bool,
F0762 0754 | limit: int | None,
F0763 0755 | overwrite: bool,
F0764 0756 | retry_failed: bool,
F0765 0757 | output_dir: Path,
F0766 0758 | results_path: Path,
F0767 0759 | ) -> None:
F0768 0760 | """Relax SQS and optional endpoint structures with a configured MLP."""
F0769 0761 | from ..stability.mlp import MaceRelaxationBackend
F0770 0762 | from ..stability.relax import relax_manifest
F0771 0763 |
F0772 0764 | if backend != "mace": # The Click choice keeps this explicit for future backends.
F0773 0765 |     raise click.ClickException(f"unsupported MLP backend: {backend}")
F0774 0766 | calculator = MaceRelaxationBackend(
F0775 0767 |     model_path,
F0776 0768 |     device=device,
F0777 0769 |     dtype=dtype,
F0778 0770 |     model_name=model_name,
F0779 0771 | )
F0780 0772 | records = relax_manifest(
F0781 0773 |     manifest_path=manifest_path,
F0782 0774 |     output_dir=output_dir,
F0783 0775 |     results_path=results_path,
F0784 0776 |     backend=calculator,
F0785 0777 |     include_endmembers=include_endmembers,
F0786 0778 |     dataset_path=dataset_path,
F0787 0779 |     fmax=fmax,
F0788 0780 |     max_steps=max_steps,
F0789 0781 |     relax_cell=relax_cell,
F0790 0782 |     overwrite=overwrite,
F0791 0783 |     retry_failed=retry_failed,
F0792 0784 |     limit=limit,
F0793 0785 | )
F0794 0786 | statuses = pd.Series([record["status"] for record in records]).value_counts().to_dict()
F0795 0787 | click.echo(
F0796 0788 |     "MLP relaxation summary: "
F0797 0789 |     f"success={statuses.get('success', 0)} "
F0798 0790 |     f"not_converged={statuses.get('not_converged', 0)} "
F0799 0791 |     f"failed={statuses.get('failed', 0)} results={results_path}"
F0800 0792 | )
```

```
src/sssreen/cli/app.py
```

```
F0801 0793 |
```

```
F0802 0794 |
```

```
F0803 0795 | @stability_cmd.command("mixing-enthalpy")
```

```
F0804 0796 | @click.option(
```

```
F0805 0797 |     "--pairs",
```

```
F0806 0798 |     "pairs_path",
```

```
F0807 0799 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
```

```
F0808 0800 |     required=True,
```

```
F0809 0801 |     help="Final pair CSV used to generate the Stage 6 SQS manifest.",
```

```
F0810 0802 | )
```

```
F0811 0803 | @click.option(
```

```
F0812 0804 |     "--relax-results",
```

```
F0813 0805 |     "relax_results_path",
```

```
F0814 0806 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
```

```
F0815 0807 |     required=True,
```

```
F0816 0808 |     help="Stage 7 relaxation-results JSONL containing SQS and endmembers.",
```

```
F0817 0809 | )
```

```
F0818 0810 | @click.option("--output", "output_path", type=click.Path(path_type=Path), required=True)
```

```
F0819 0811 | @click.option("--summary", "summary_path", type=click.Path(path_type=Path), required=True)
```

```
F0820 0812 | def stability_mixing_enthalpy_cmd(
```

```
F0821 0813 |     pairs_path: Path,
```

```
F0822 0814 |     relax_results_path: Path,
```

```
F0823 0815 |     output_path: Path,
```

```
F0824 0816 |     summary_path: Path,
```

```
F0825 0817 | ) -> None:
```

```
F0826 0818 |     """Calculate preliminary per-composition mixing enthalpies."""
```

```
F0827 0819 |     from ..stability.thermodynamics import calculate_mixing_enthalpies
```

```
F0828 0820 |
```

```
F0829 0821 |     _frame, summary = calculate_mixing_enthalpies(
```

```
F0830 0822 |         pairs_path=pairs_path,
```

```
F0831 0823 |         relax_results_path=relax_results_path,
```

```
F0832 0824 |         output_path=output_path,
```

```
F0833 0825 |         summary_path=summary_path,
```

```
F0834 0826 |     )
```

```
F0835 0827 |     click.echo(
```

```
F0836 0828 |         "Mixing enthalpy summary: "
```

```
F0837 0829 |         f"success={summary['success_count']} " "
```

```
F0838 0830 |         f"failed={summary['failed_count']} output={output_path} summary={summary_path}"
```

```
F0839 0831 |     )
```

```
F0840 0832 |
```

```
F0841 0833 |
```

```
F0842 0834 | @stability_cmd.command("phonon-export")
```

```
F0843 0835 | @click.option(
```

```
F0844 0836 |     "--relax-results",
```

```
F0845 0837 |     "relax_results_path",
```

```
F0846 0838 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
```

```
F0847 0839 |     required=True,
```

```
F0848 0840 |     help="Stage 7 relaxation-results JSONL.",
```

```
F0849 0841 | )
```

```
F0850 0842 | @click.option(
```

```
src/ssscreen/cli/app.py
```

```
F0851 0843 | "--structure-id",
F0852 0844 | "structure_ids",
F0853 0845 | multiple=True,
F0854 0846 | help="Process only this structure ID; repeat to select more than one.",
F0855 0847 | )
F0856 0848 | @click.option("--include-endmembers/--sqs-only", default=False, show_default=True)
F0857 0849 | @click.option(
F0858 0850 | "--supercell",
F0859 0851 | callback=_parse_optional_supercell,
F0860 0852 | help="Optional diagonal supercell as 'a,b,c'; otherwise choose from cell lengths.",
F0861 0853 | )
F0862 0854 | @click.option(
F0863 0855 | "--min-supercell-length",
F0864 0856 | type=click.FloatRange(min=0.0, min_open=True),
F0865 0857 | default=PhononSettings.min_supercell_length,
F0866 0858 | show_default=True,
F0867 0859 | help="Automatic supercells target this lattice-vector length in Angstrom.",
F0868 0860 | )
F0869 0861 | @click.option(
F0870 0862 | "--max-supercell-atoms",
F0871 0863 | type=click.IntRange(min=1),
F0872 0864 | default=PhononSettings.max_supercell_atoms,
F0873 0865 | show_default=True,
F0874 0866 | )
F0875 0867 | @click.option(
F0876 0868 | "--displacement",
F0877 0869 | "displacement_distance",
F0878 0870 | type=click.FloatRange(min=0.0, min_open=True),
F0879 0871 | default=PhononSettings.displacement_distance,
F0880 0872 | show_default=True,
F0881 0873 | help="Finite-displacement amplitude in Angstrom.",
F0882 0874 | )
F0883 0875 | @click.option(
F0884 0876 | "--symprec",
F0885 0877 | "symmetry_tolerance",
F0886 0878 | type=click.FloatRange(min=0.0, min_open=True),
F0887 0879 | default=PhononSettings.symmetry_tolerance,
F0888 0880 | show_default=True,
F0889 0881 | )
F0890 0882 | @click.option(
F0891 0883 | "--max-input-force",
F0892 0884 | type=click.FloatRange(min=0.0, min_open=True),
F0893 0885 | default=PhononSettings.max_input_force,
F0894 0886 | show_default=True,
F0895 0887 | help="Maximum accepted Stage 7 residual force in eV/Angstrom.",
F0896 0888 | )
F0897 0889 | @click.option(
F0898 0890 | "--allow-loose-input",
F0899 0891 | is_flag=True,
F0900 0892 | help="Generate tasks even when the Stage 7 residual force exceeds the limit.",
```

```
src/ssscreen/cli/app.py
```

```
F0901 0893 | )
F0902 0894 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F0903 0895 | @click.option("--manifest", "manifest_path", type=click.Path(path_type=Path), required=True)
F0904 0896 | def stability_phonon_export_cmd(
F0905 0897 |     relax_results_path: Path,
F0906 0898 |     structure_ids: tuple[str, ...],
F0907 0899 |     include_endmembers: bool,
F0908 0900 |     supercell: tuple[int, int, int] | None,
F0909 0901 |     min_supercell_length: float,
F0910 0902 |     max_supercell_atoms: int,
F0911 0903 |     displacement_distance: float,
F0912 0904 |     symmetry_tolerance: float,
F0913 0905 |     max_input_force: float,
F0914 0906 |     allow_loose_input: bool,
F0915 0907 |     output_dir: Path,
F0916 0908 |     manifest_path: Path,
F0917 0909 | ) -> None:
F0918 0910 |     """Generate Phonopy reference and displaced-supercell tasks."""
F0919 0911 |     from ..stability.phonon import export_phonon_tasks
F0920 0912 |
F0921 0913 |     records = export_phonon_tasks(
F0922 0914 |         relax_results_path=relax_results_path,
F0923 0915 |         output_dir=output_dir,
F0924 0916 |         manifest_path=manifest_path,
F0925 0917 |         structure_ids=structure_ids,
F0926 0918 |         include_endmembers=include_endmembers,
F0927 0919 |         supercell=supercell,
F0928 0920 |         displacement_distance=displacement_distance,
F0929 0921 |         symmetry_tolerance=symmetry_tolerance,
F0930 0922 |         min_supercell_length=min_supercell_length,
F0931 0923 |         max_supercell_atoms=max_supercell_atoms,
F0932 0924 |         max_input_force=max_input_force,
F0933 0925 |         allow_loose_input=allow_loose_input,
F0934 0926 |     )
F0935 0927 |     written = sum(record.get("status") == "written" for record in records)
F0936 0928 |     skipped = sum(record.get("status") == "skipped" for record in records)
F0937 0929 |     structures = len({record.get("phonon_id") for record in records})
F0938 0930 |     click.echo(
F0939 0931 |         f"Phonon export summary: structures={structures} tasks={written} "
F0940 0932 |         f"skipped={skipped} manifest={manifest_path}"
F0941 0933 |     )
F0942 0934 |
F0943 0935 |
F0944 0936 | @stability_cmd.command("phonon-forces")
F0945 0937 | @click.option(
F0946 0938 |     "--manifest",
F0947 0939 |     "manifest_path",
F0948 0940 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F0949 0941 |     required=True,
F0950 0942 | )
```

```
src/ssscreen/cli/app.py
```

```
F0951 0943 | @click.option("--backend", type=click.Choice(["mace"]), default="mace", show_default=True)
F0952 0944 | @click.option(
F0953 0945 |     "--model-path",
F0954 0946 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F0955 0947 |     required=True,
F0956 0948 | )
F0957 0949 | @click.option("--model-name", help="Optional stable model label stored in provenance.")
F0958 0950 | @click.option("--device", default="cpu", show_default=True)
F0959 0951 | @click.option(
F0960 0952 |     "--dtype",
F0961 0953 |     type=click.Choice(["float32", "float64"]),
F0962 0954 |     default="float64",
F0963 0955 |     show_default=True,
F0964 0956 | )
F0965 0957 | @click.option("--limit", type=click.IntRange(min=1), help="Evaluate at most this many tasks.")
F0966 0958 | @click.option("--overwrite", is_flag=True)
F0967 0959 | @click.option("--retry-failed", is_flag=True)
F0968 0960 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F0969 0961 | @click.option("--results", "results_path", type=click.Path(path_type=Path), required=True)
F0970 0962 | def stability_phonon_forces_cmd(
F0971 0963 |     manifest_path: Path,
F0972 0964 |     backend: str,
F0973 0965 |     model_path: Path,
F0974 0966 |     model_name: str | None,
F0975 0967 |     device: str,
F0976 0968 |     dtype: str,
F0977 0969 |     limit: int | None,
F0978 0970 |     overwrite: bool,
F0979 0971 |     retry_failed: bool,
F0980 0972 |     output_dir: Path,
F0981 0973 |     results_path: Path,
F0982 0974 | ) -> None:
F0983 0975 |     """Evaluate Phonopy task energies, forces, and stresses with MACE."""
F0984 0976 |     from ..stability.mlp import MaceRelaxationBackend
F0985 0977 |     from ..stability.phonon import evaluate_phonon_forces
F0986 0978 |
F0987 0979 |     if backend != "mace":
F0988 0980 |         raise click.ClickException(f"unsupported phonon force backend: {backend}")
F0989 0981 |     calculator = MaceRelaxationBackend(
F0990 0982 |         model_path,
F0991 0983 |         device=device,
F0992 0984 |         dtype=dtype,
F0993 0985 |         model_name=model_name,
F0994 0986 |     )
F0995 0987 |     records = evaluate_phonon_forces(
F0996 0988 |         manifest_path=manifest_path,
F0997 0989 |         output_dir=output_dir,
F0998 0990 |         results_path=results_path,
F0999 0991 |         backend=calculator,
F1000 0992 |         overwrite=overwrite,
```

```
src/ssscren/cli/app.py
```

```
F1001 0993 |     retry_failed=retry_failed,
F1002 0994 |     limit=limit,
F1003 0995 | )
F1004 0996 | statuses = pd.Series([record["status"] for record in records]).value_counts().to_dict()
F1005 0997 | click.echo(
F1006 0998 |     "Phonon MACE force summary: "
F1007 0999 |     f"success={statuses.get('success', 0)} failed={statuses.get('failed', 0)} "
F1008 1000 |     f"results={results_path}"
F1009 1001 | )
F1010 1002 |
F1011 1003 |
F1012 1004 | @stability_cmd.command("phonon-collect")
F1013 1005 | @click.option(
F1014 1006 |     "--manifest",
F1015 1007 |     "manifest_path",
F1016 1008 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1017 1009 |     required=True,
F1018 1010 | )
F1019 1011 | @click.option(
F1020 1012 |     "--force-results",
F1021 1013 |     "force_results_path",
F1022 1014 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1023 1015 |     required=True,
F1024 1016 | )
F1025 1017 | @click.option(
F1026 1018 |     "--mesh",
F1027 1019 |     callback=lambda _ctx, _param, value: _parse_supercell(value),
F1028 1020 |     default="", join(map(str, PhononSettings.mesh)),
F1029 1021 |     show_default=True,
F1030 1022 |     help="Gamma-centered q mesh as 'a,b,c'.",
F1031 1023 | )
F1032 1024 | @click.option(
F1033 1025 |     "--band-points",
F1034 1026 |     type=click.IntRange(min=2),
F1035 1027 |     default=PhononSettings.band_points,
F1036 1028 |     show_default=True,
F1037 1029 | )
F1038 1030 | @click.option(
F1039 1031 |     "--imaginary-tolerance",
F1040 1032 |     type=click.FloatRange(min=0.0),
F1041 1033 |     default=PhononSettings.imaginary_tolerance_thz,
F1042 1034 |     show_default=True,
F1043 1035 |     help="Magnitude threshold for significant imaginary frequencies in THz.",
F1044 1036 | )
F1045 1037 | @click.option("--subtract-reference-forces/--raw-forces", default=True, show_default=True)
F1046 1038 | @click.option("--symmetrize-fc/--raw-force-constants", default=True, show_default=True)
F1047 1039 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F1048 1040 | @click.option("--summary", "summary_path", type=click.Path(path_type=Path), required=True)
F1049 1041 | @click.option("--report", "report_path", type=click.Path(path_type=Path), required=True)
F1050 1042 | def stability_phonon_collect_cmd(
```

```
src/ssscreen/cli/app.py
```

```
F1051 1043 | manifest_path: Path,
F1052 1044 | force_results_path: Path,
F1053 1045 | mesh: tuple[int, int, int],
F1054 1046 | band_points: int,
F1055 1047 | imaginary_tolerance: float,
F1056 1048 | subtract_reference_forces: bool,
F1057 1049 | symmetrize_fc: bool,
F1058 1050 | output_dir: Path,
F1059 1051 | summary_path: Path,
F1060 1052 | report_path: Path,
F1061 1053 | ) -> None:
F1062 1054 | """Build force constants, phonon bands, DOS, and stability summaries."""
F1063 1055 | from ..stability.phonon import collect_phonon_results
F1064 1056 |
F1065 1057 | frame, report = collect_phonon_results(
F1066 1058 |     manifest_path=manifest_path,
F1067 1059 |     force_results_path=force_results_path,
F1068 1060 |     output_dir=output_dir,
F1069 1061 |     summary_path=summary_path,
F1070 1062 |     report_path=report_path,
F1071 1063 |     mesh=mesh,
F1072 1064 |     band_points=band_points,
F1073 1065 |     imaginary_tolerance=imaginary_tolerance,
F1074 1066 |     subtract_reference_forces=subtract_reference_forces,
F1075 1067 |     symmetrize_force_constants=symmetrize_fc,
F1076 1068 | )
F1077 1069 | counts = report["dynamical_status_counts"]
F1078 1070 | click.echo(
F1079 1071 |     "Phonon collection summary: "
F1080 1072 |     f"structures={len(frame)} stable={counts.get('stable', 0)} "
F1081 1073 |     f"unstable={counts.get('unstable', 0)} uncertain={counts.get('uncertain', 0)} "
F1082 1074 |     f"failed={counts.get('failed', 0)} summary={summary_path}"
F1083 1075 | )
F1084 1076 |
F1085 1077 |
F1086 1078 | @stability_cmd.command("phonon-run")
F1087 1079 | @click.option(
F1088 1080 |     "--relax-results",
F1089 1081 |     "relax_results_path",
F1090 1082 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1091 1083 |     required=True,
F1092 1084 | )
F1093 1085 | @click.option("--structure-id", "structure_ids", multiple=True)
F1094 1086 | @click.option("--include-endmembers/--sqs-only", default=False, show_default=True)
F1095 1087 | @click.option("--supercell", callback=_parse_optional_supercell)
F1096 1088 | @click.option(
F1097 1089 |     "--min-supercell-length",
F1098 1090 |     type=click.FloatRange(min=0.0, min_open=True),
F1099 1091 |     default=PhononSettings.min_supercell_length,
F1100 1092 |     show_default=True,
```

```
src/ssscreen/cli/app.py
```

```
F1101 1093 | )
F1102 1094 | @click.option(
F1103 1095 |     "--max-supercell-atoms",
F1104 1096 |     type=click.IntRange(min=1),
F1105 1097 |     default=PhononSettings.max_supercell_atoms,
F1106 1098 |     show_default=True,
F1107 1099 | )
F1108 1100 | @click.option(
F1109 1101 |     "--displacement",
F1110 1102 |     type=click.FloatRange(min=0.0, min_open=True),
F1111 1103 |     default=PhononSettings.displacement_distance,
F1112 1104 |     show_default=True,
F1113 1105 | )
F1114 1106 | @click.option(
F1115 1107 |     "--symprec",
F1116 1108 |     type=click.FloatRange(min=0.0, min_open=True),
F1117 1109 |     default=PhononSettings.symmetry_tolerance,
F1118 1110 |     show_default=True,
F1119 1111 | )
F1120 1112 | @click.option(
F1121 1113 |     "--max-input-force",
F1122 1114 |     type=click.FloatRange(min=0.0, min_open=True),
F1123 1115 |     default=PhononSettings.max_input_force,
F1124 1116 |     show_default=True,
F1125 1117 | )
F1126 1118 | @click.option("--allow-loose-input", is_flag=True)
F1127 1119 | @click.option(
F1128 1120 |     "--mesh",
F1129 1121 |     callback=lambda _ctx, _param, value: _parse_supercell(value),
F1130 1122 |     default=",".join(map(str, PhononSettings.mesh)),
F1131 1123 |     show_default=True,
F1132 1124 | )
F1133 1125 | @click.option(
F1134 1126 |     "--band-points",
F1135 1127 |     type=click.IntRange(min=2),
F1136 1128 |     default=PhononSettings.band_points,
F1137 1129 |     show_default=True,
F1138 1130 | )
F1139 1131 | @click.option(
F1140 1132 |     "--imaginary-tolerance",
F1141 1133 |     type=click.FloatRange(min=0.0),
F1142 1134 |     default=PhononSettings.imaginary_tolerance_thz,
F1143 1135 |     show_default=True,
F1144 1136 | )
F1145 1137 | @click.option(
F1146 1138 |     "--model-path",
F1147 1139 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1148 1140 |     required=True,
F1149 1141 | )
F1150 1142 | @click.option("--model-name")
```

```
src/ssscreen/cli/app.py
```

```
F1151 1143 | @click.option("--device", default="cpu", show_default=True)
F1152 1144 | @click.option(
F1153 1145 |     "--dtype",
F1154 1146 |     type=click.Choice(["float32", "float64"]),
F1155 1147 |     default="float64",
F1156 1148 |     show_default=True,
F1157 1149 | )
F1158 1150 | @click.option("--overwrite", is_flag=True)
F1159 1151 | @click.option("--retry-failed", is_flag=True)
F1160 1152 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F1161 1153 | def stability_phonon_run_cmd(
F1162 1154 |     relax_results_path: Path,
F1163 1155 |     structure_ids: tuple[str, ...],
F1164 1156 |     include_endmembers: bool,
F1165 1157 |     supercell: tuple[int, int, int] | None,
F1166 1158 |     min_supercell_length: float,
F1167 1159 |     max_supercell_atoms: int,
F1168 1160 |     displacement: float,
F1169 1161 |     symprec: float,
F1170 1162 |     max_input_force: float,
F1171 1163 |     allow_loose_input: bool,
F1172 1164 |     mesh: tuple[int, int, int],
F1173 1165 |     band_points: int,
F1174 1166 |     imaginary_tolerance: float,
F1175 1167 |     model_path: Path,
F1176 1168 |     model_name: str | None,
F1177 1169 |     device: str,
F1178 1170 |     dtype: str,
F1179 1171 |     overwrite: bool,
F1180 1172 |     retry_failed: bool,
F1181 1173 |     output_dir: Path,
F1182 1174 | ) -> None:
F1183 1175 |     """Run harmonic finite-displacement phonons end to end with MACE forces."""
F1184 1176 |     from ..stability.mlp import MaceRelaxationBackend
F1185 1177 |     from ..stability.phonon import run_phonon_pipeline
F1186 1178 |
F1187 1179 |     calculator = MaceRelaxationBackend(
F1188 1180 |         model_path,
F1189 1181 |         device=device,
F1190 1182 |         dtype=dtype,
F1191 1183 |         model_name=model_name,
F1192 1184 |     )
F1193 1185 |     settings = PhononSettings(
F1194 1186 |         displacement_distance=displacement,
F1195 1187 |         symmetry_tolerance=symprec,
F1196 1188 |         min_supercell_length=min_supercell_length,
F1197 1189 |         max_supercell_atoms=max_supercell_atoms,
F1198 1190 |         mesh=mesh,
F1199 1191 |         band_points=band_points,
F1200 1192 |         imaginary_tolerance_thz=imaginary_tolerance,
```

```
src/sssreen/cli/app.py
```

```
F1201 1193 |     max_input_force=max_input_force,
F1202 1194 | )
F1203 1195 | frame, report = run_phonon_pipeline(
F1204 1196 |     relax_results_path=relax_results_path,
F1205 1197 |     output_dir=output_dir,
F1206 1198 |     backend=calculator,
F1207 1199 |     settings=settings,
F1208 1200 |     structure_ids=structure_ids,
F1209 1201 |     include_endmembers=include_endmembers,
F1210 1202 |     supercell=supercell,
F1211 1203 |     allow_loose_input=allow_loose_input,
F1212 1204 |     overwrite=overwrite,
F1213 1205 |     retry_failed=retry_failed,
F1214 1206 | )
F1215 1207 | counts = report["dynamical_status_counts"]
F1216 1208 | click.echo(
F1217 1209 |     "MACE phonon summary: "
F1218 1210 |     f"structures={len(frame)} stable={counts.get('stable', 0)} "
F1219 1211 |     f"unstable={counts.get('unstable', 0)} uncertain={counts.get('uncertain', 0)} "
F1220 1212 |     f"failed={counts.get('failed', 0)} output={output_dir}"
F1221 1213 | )
F1222 1214 |
F1223 1215 |
F1224 1216 | @stability_cmd.command("competing-export")
F1225 1217 | @click.option(
F1226 1218 |     "--relax-results",
F1227 1219 |     "relax_results_path",
F1228 1220 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1229 1221 |     required=True,
F1230 1222 |     help="Stage 7 relaxation-results JSONL containing SQS candidates.",
F1231 1223 | )
F1232 1224 | @click.option("--structure-id", "structure_ids", multiple=True)
F1233 1225 | @click.option(
F1234 1226 |     "--mp-backend",
F1235 1227 |     type=click.Choice(["api", "offline"]),
F1236 1228 |     default="api",
F1237 1229 |     show_default=True,
F1238 1230 |     help="Materials Project source used only to discover competing structures.",
F1239 1231 | )
F1240 1232 | @click.option(
F1241 1233 |     "--offline-db",
F1242 1234 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1243 1235 |     help="Explicit mp_offline SQLite snapshot; required for the offline backend.",
F1244 1236 | )
F1245 1237 | @click.option(
F1246 1238 |     "--mp-max-e-hull",
F1247 1239 |     type=click.FloatRange(min=0.0),
F1248 1240 |     default=CompetingPhaseSettings.max_mp_energy_above_hull,
F1249 1241 |     show_default=True,
F1250 1242 |     help="Maximum MP hull distance used to select competing structures (eV/atom).",
```

```
src/ssscreen/cli/app.py
```

```
F1251 1243 | )
```

```
F1252 1244 | @click.option(
```

```
F1253 1245 |     "--thermo-type",
```

```
F1254 1246 |     type=click.Choice(["GGA_GGA+U", "GGA_GGA+U_R2SCAN", "R2SCAN"]),
```

```
F1255 1247 |     default=CompetingPhaseSettings.thermo_type,
```

```
F1256 1248 |     show_default=True,
```

```
F1257 1249 |     help="MP thermo filter for the API backend; not available in offline summaries.",
```

```
F1258 1250 | )
```

```
F1259 1251 | @click.option(
```

```
F1260 1252 |     "--max-phase-atoms",
```

```
F1261 1253 |     type=click.IntRange(min=1),
```

```
F1262 1254 |     default=CompetingPhaseSettings.max_competing_phase_atoms,
```

```
F1263 1255 |     show_default=True,
```

```
F1264 1256 |     help="Maximum atom count after primitive-cell reduction.",
```

```
F1265 1257 | )
```

```
F1266 1258 | @click.option(
```

```
F1267 1259 |     "--api-timeout",
```

```
F1268 1260 |     type=click.FloatRange(min=0.0, min_open=True),
```

```
F1269 1261 |     default=CompetingPhaseSettings.api_timeout_seconds,
```

```
F1270 1262 |     show_default=True,
```

```
F1271 1263 |     help="Per-request and per-system Materials Project timeout in seconds.",
```

```
F1272 1264 | )
```

```
F1273 1265 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
```

```
F1274 1266 | @click.option("--manifest", "manifest_path", type=click.Path(path_type=Path), required=True)
```

```
F1275 1267 | @click.option("--report", "report_path", type=click.Path(path_type=Path), required=True)
```

```
F1276 1268 | def stability_competing_export_cmd(
```

```
F1277 1269 |     relax_results_path: Path,
```

```
F1278 1270 |     structure_ids: tuple[str, ...],
```

```
F1279 1271 |     mp_backend: str,
```

```
F1280 1272 |     offline_db: Path | None,
```

```
F1281 1273 |     mp_max_e_hull: float,
```

```
F1282 1274 |     thermo_type: str,
```

```
F1283 1275 |     max_phase_atoms: int,
```

```
F1284 1276 |     api_timeout: float,
```

```
F1285 1277 |     output_dir: Path,
```

```
F1286 1278 |     manifest_path: Path,
```

```
F1287 1279 |     report_path: Path,
```

```
F1288 1280 | ) -> None:
```

```
F1289 1281 |     """Export competing structures from MP API or an offline snapshot."""
```

```
F1290 1282 |     from ...stability.competing import export_competing_phases
```

```
F1291 1283 |
```

```
F1292 1284 |     if mp_backend == "offline" and offline_db is None:
```

```
F1293 1285 |         raise click.UsageError("--offline-db is required when --mp-backend=offline")
```

```
F1294 1286 |     api_key = _prompt_mp_api_key() if mp_backend == "api" else None
```

```
F1295 1287 |     records, report = export_competing_phases(
```

```
F1296 1288 |         relax_results_path=relax_results_path,
```

```
F1297 1289 |         output_dir=output_dir,
```

```
F1298 1290 |         manifest_path=manifest_path,
```

```
F1299 1291 |         report_path=report_path,
```

```
F1300 1292 |         api_key=api_key,
```

```
src/ssscreen/cli/app.py
```

```
F1301 1293 |     mp_backend=mp_backend,
F1302 1294 |     offline_db=offline_db,
F1303 1295 |     structure_ids=structure_ids,
F1304 1296 |     settings=CompetingPhaseSettings(
F1305 1297 |         thermo_type=thermo_type,
F1306 1298 |         max_mp_energy_above_hull=mp_max_e_hull,
F1307 1299 |         max_competing_phase_atoms=max_phase_atoms,
F1308 1300 |         api_timeout_seconds=api_timeout,
F1309 1301 |     ),
F1310 1302 | )
F1311 1303 | counts = report["status_counts"]
F1312 1304 | click.echo(
F1313 1305 |     f"MP competing-phase export ({mp_backend}): "
F1314 1306 |     f"systems={len(report['chemical_systems'])} fetched={report['fetched_entry_count']} "
F1315 1307 |     f"written={counts.get('written', 0)} skipped={counts.get('skipped', 0)} "
F1316 1308 |     f"query_failed={report['query_failure_count']} manifest={manifest_path}"
F1317 1309 | )
F1318 1310 |
F1319 1311 |
F1320 1312 | @stability_cmd.command("competing-relax")
F1321 1313 | @click.option(
F1322 1314 |     "--manifest",
F1323 1315 |     "manifest_path",
F1324 1316 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1325 1317 |     required=True,
F1326 1318 | )
F1327 1319 | @click.option(
F1328 1320 |     "--model-path",
F1329 1321 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1330 1322 |     required=True,
F1331 1323 | )
F1332 1324 | @click.option("--model-name")
F1333 1325 | @click.option("--device", default="cpu", show_default=True)
F1334 1326 | @click.option(
F1335 1327 |     "--dtype",
F1336 1328 |     type=click.Choice(["float32", "float64"]),
F1337 1329 |     default="float32",
F1338 1330 |     show_default=True,
F1339 1331 | )
F1340 1332 | @click.option(
F1341 1333 |     "--fmax",
F1342 1334 |     type=click.FloatRange(min=0.0, min_open=True),
F1343 1335 |     default=MLPRelaxationSettings.force_tolerance,
F1344 1336 |     show_default=True,
F1345 1337 | )
F1346 1338 | @click.option(
F1347 1339 |     "--max-steps",
F1348 1340 |     type=click.IntRange(min=1),
F1349 1341 |     default=MLPRelaxationSettings.max_steps,
F1350 1342 |     show_default=True,
```

```
src/ssscreen/cli/app.py
```

```
F1351 1343 | )
F1352 1344 | @click.option("--relax-cell/--positions-only", default=True, show_default=True)
F1353 1345 | @click.option("--limit", type=click.IntRange(min=1))
F1354 1346 | @click.option("--overwrite", is_flag=True)
F1355 1347 | @click.option("--retry-failed", is_flag=True)
F1356 1348 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F1357 1349 | @click.option("--results", "results_path", type=click.Path(path_type=Path), required=True)
F1358 1350 | def stability_competing_relax_cmd(
F1359 1351 |     manifest_path: Path,
F1360 1352 |     model_path: Path,
F1361 1353 |     model_name: str | None,
F1362 1354 |     device: str,
F1363 1355 |     dtype: str,
F1364 1356 |     fmax: float,
F1365 1357 |     max_steps: int,
F1366 1358 |     relax_cell: bool,
F1367 1359 |     limit: int | None,
F1368 1360 |     overwrite: bool,
F1369 1361 |     retry_failed: bool,
F1370 1362 |     output_dir: Path,
F1371 1363 |     results_path: Path,
F1372 1364 | ) -> None:
F1373 1365 |     """Relax MP competing structures with MACE and the Stage 7 contract."""
F1374 1366 |     from ..stability.competing import relax_competing_phases
F1375 1367 |     from ..stability.mlp import MaceRelaxationBackend
F1376 1368 |
F1377 1369 |     calculator = MaceRelaxationBackend(
F1378 1370 |         model_path,
F1379 1371 |         device=device,
F1380 1372 |         dtype=dtype,
F1381 1373 |         model_name=model_name,
F1382 1374 |     )
F1383 1375 |     records = relax_competing_phases(
F1384 1376 |         manifest_path=manifest_path,
F1385 1377 |         output_dir=output_dir,
F1386 1378 |         results_path=results_path,
F1387 1379 |         backend=calculator,
F1388 1380 |         fmax=fmax,
F1389 1381 |         max_steps=max_steps,
F1390 1382 |         relax_cell=relax_cell,
F1391 1383 |         overwrite=overwrite,
F1392 1384 |         retry_failed=retry_failed,
F1393 1385 |         limit=limit,
F1394 1386 |     )
F1395 1387 |     counts = pd.Series([record["status"] for record in records]).value_counts().to_dict()
F1396 1388 |     click.echo(
F1397 1389 |         "Competing-phase MACE relaxation: "
F1398 1390 |         f"success={counts.get('success', 0)} "
F1399 1391 |         f"not_converged={counts.get('not_converged', 0)} "
F1400 1392 |         f"failed={counts.get('failed', 0)} results={results_path}"
```

```
src/ssscreen/cli/app.py
```

```
F1401 1393 | )
F1402 1394 |
F1403 1395 |
F1404 1396 | @stability_cmd.command("convex-hull")
F1405 1397 | @click.option(
F1406 1398 |     "--relax-results",
F1407 1399 |     "candidate_relax_results_path",
F1408 1400 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1409 1401 |     required=True,
F1410 1402 | )
F1411 1403 | @click.option(
F1412 1404 |     "--competing-manifest",
F1413 1405 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1414 1406 |     required=True,
F1415 1407 | )
F1416 1408 | @click.option(
F1417 1409 |     "--competing-results",
F1418 1410 |     "competing_relax_results_path",
F1419 1411 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1420 1412 |     required=True,
F1421 1413 | )
F1422 1414 | @click.option("--structure-id", "structure_ids", multiple=True)
F1423 1415 | @click.option(
F1424 1416 |     "--screening-cutoff",
F1425 1417 |     type=click.FloatRange(min=0.0),
F1426 1418 |     default=CompetingPhaseSettings.screening_cutoff_ev_per_atom,
F1427 1419 |     show_default=True,
F1428 1420 |     help="Maximum e_above_hull retained for screening (eV/atom).",
F1429 1421 | )
F1430 1422 | @click.option(
F1431 1423 |     "--numerical-tolerance",
F1432 1424 |     type=click.FloatRange(min=0.0),
F1433 1425 |     default=CompetingPhaseSettings.numerical_tolerance_ev_per_atom,
F1434 1426 |     show_default=True,
F1435 1427 | )
F1436 1428 | @click.option(
F1437 1429 |     "--allow-incomplete",
F1438 1430 |     is_flag=True,
F1439 1431 |     help="Calculate an explicitly uncertain hull despite missing competing phases.",
F1440 1432 | )
F1441 1433 | @click.option("--output", "output_path", type=click.Path(path_type=Path), required=True)
F1442 1434 | @click.option(
F1443 1435 |     "--entries-output",
F1444 1436 |     "entries_output_path",
F1445 1437 |     type=click.Path(path_type=Path),
F1446 1438 |     required=True,
F1447 1439 | )
F1448 1440 | @click.option("--summary", "summary_path", type=click.Path(path_type=Path), required=True)
F1449 1441 | def stability_convex_hull_cmd(
F1450 1442 |     candidate_relax_results_path: Path,
```

src/ssscreen/cli/app.py

```

F1451 1443 |     competing_manifest: Path,
F1452 1444 |     competing_relax_results_path: Path,
F1453 1445 |     structure_ids: tuple[str, ...],
F1454 1446 |     screening_cutoff: float,
F1455 1447 |     numerical_tolerance: float,
F1456 1448 |     allow_incomplete: bool,
F1457 1449 |     output_path: Path,
F1458 1450 |     entries_output_path: Path,
F1459 1451 |     summary_path: Path,
F1460 1452 | ) -> None:
F1461 1453 |     """Build strict same-MLIP convex hulls for relaxed SQS candidates."""
F1462 1454 |     from ..stability.competing import collect_convex_hulls
F1463 1455 |
F1464 1456 |     frame, _entries, summary = collect_convex_hulls(
F1465 1457 |         candidate_relax_results_path=candidate_relax_results_path,
F1466 1458 |         competing_manifest_path=competing_manifest,
F1467 1459 |         competing_relax_results_path=competing_relax_results_path,
F1468 1460 |         output_path=output_path,
F1469 1461 |         entries_output_path=entries_output_path,
F1470 1462 |         summary_path=summary_path,
F1471 1463 |         structure_ids=structure_ids,
F1472 1464 |         screening_cutoff=screening_cutoff,
F1473 1465 |         numerical_tolerance=numerical_tolerance,
F1474 1466 |         allow_incomplete=allow_incomplete,
F1475 1467 |     )
F1476 1468 |     click.echo(
F1477 1469 |         "MLIP convex-hull summary: "
F1478 1470 |         f"candidates={len(frame)} success={summary['success_count']} "
F1479 1471 |         f"uncertain={summary['uncertain_count']} failed={summary['failed_count']} "
F1480 1472 |         f"output={output_path}"
F1481 1473 |     )
F1482 1474 |
F1483 1475 |
F1484 1476 | @stability_cmd.command("phase-diagram")
F1485 1477 | @click.option(
F1486 1478 |     "--relax-results",
F1487 1479 |     "candidate_relax_results_path",
F1488 1480 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1489 1481 |     required=True,
F1490 1482 | )
F1491 1483 | @click.option("--structure-id", "structure_ids", multiple=True)
F1492 1484 | @click.option(
F1493 1485 |     "--mp-backend",
F1494 1486 |     type=click.Choice(["api", "offline"]),
F1495 1487 |     default="api",
F1496 1488 |     show_default=True,
F1497 1489 |     help="Materials Project source used only to discover competing structures.",
F1498 1490 | )
F1499 1491 | @click.option(
F1500 1492 |     "--offline-db",

```

```
src/ssscreen/cli/app.py
```

```
F1501 1493 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1502 1494 | help="Explicit mp_offline SQLite snapshot; required for the offline backend.",
F1503 1495 | )
F1504 1496 | @click.option(
F1505 1497 |     "--mp-max-e-hull",
F1506 1498 |     type=click.FloatRange(min=0.0),
F1507 1499 |     default=CompetingPhaseSettings.max_mp_energy_above_hull,
F1508 1500 |     show_default=True,
F1509 1501 | )
F1510 1502 | @click.option(
F1511 1503 |     "--thermo-type",
F1512 1504 |     type=click.Choice(["GGA_GGA+U", "GGA_GGA+U_R2SCAN", "R2SCAN"]),
F1513 1505 |     default=CompetingPhaseSettings.thermo_type,
F1514 1506 |     show_default=True,
F1515 1507 |     help="MP thermo filter for the API backend; not available in offline summaries.",
F1516 1508 | )
F1517 1509 | @click.option(
F1518 1510 |     "--max-phase-atoms",
F1519 1511 |     type=click.IntRange(min=1),
F1520 1512 |     default=CompetingPhaseSettings.max_competing_phase_atoms,
F1521 1513 |     show_default=True,
F1522 1514 | )
F1523 1515 | @click.option(
F1524 1516 |     "--api-timeout",
F1525 1517 |     type=click.FloatRange(min=0.0, min_open=True),
F1526 1518 |     default=CompetingPhaseSettings.api_timeout_seconds,
F1527 1519 |     show_default=True,
F1528 1520 | )
F1529 1521 | @click.option(
F1530 1522 |     "--model-path",
F1531 1523 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1532 1524 |     required=True,
F1533 1525 | )
F1534 1526 | @click.option("--model-name")
F1535 1527 | @click.option("--device", default="cpu", show_default=True)
F1536 1528 | @click.option(
F1537 1529 |     "--dtype",
F1538 1530 |     type=click.Choice(["float32", "float64"]),
F1539 1531 |     default="float32",
F1540 1532 |     show_default=True,
F1541 1533 | )
F1542 1534 | @click.option(
F1543 1535 |     "--fmax",
F1544 1536 |     type=click.FloatRange(min=0.0, min_open=True),
F1545 1537 |     default=MLPRelaxationSettings.force_tolerance,
F1546 1538 |     show_default=True,
F1547 1539 | )
F1548 1540 | @click.option(
F1549 1541 |     "--max-steps",
F1550 1542 |     type=click.IntRange(min=1),
```

```
src/sssreen/cli/app.py
```

```
F1551 1543 |     default=MLPRelaxationSettings.max_steps,
F1552 1544 |     show_default=True,
F1553 1545 | )
F1554 1546 | @click.option("--relax-cell/--positions-only", default=True, show_default=True)
F1555 1547 | @click.option(
F1556 1548 |     "--screening-cutoff",
F1557 1549 |     type=click.FloatRange(min=0.0),
F1558 1550 |     default=CompetingPhaseSettings.screening_cutoff_ev_per_atom,
F1559 1551 |     show_default=True,
F1560 1552 | )
F1561 1553 | @click.option(
F1562 1554 |     "--numerical-tolerance",
F1563 1555 |     type=click.FloatRange(min=0.0),
F1564 1556 |     default=CompetingPhaseSettings.numerical_tolerance_ev_per_atom,
F1565 1557 |     show_default=True,
F1566 1558 | )
F1567 1559 | @click.option("--allow-incomplete", is_flag=True)
F1568 1560 | @click.option("--overwrite", is_flag=True)
F1569 1561 | @click.option("--retry-failed", is_flag=True)
F1570 1562 | @click.option("--output-dir", type=click.Path(path_type=Path), required=True)
F1571 1563 | def stability_phase_diagram_cmd(
F1572 1564 |     candidate_relax_results_path: Path,
F1573 1565 |     structure_ids: tuple[str, ...],
F1574 1566 |     mp_backend: str,
F1575 1567 |     offline_db: Path | None,
F1576 1568 |     mp_max_e_hull: float,
F1577 1569 |     thermo_type: str,
F1578 1570 |     max_phase_atoms: int,
F1579 1571 |     api_timeout: float,
F1580 1572 |     model_path: Path,
F1581 1573 |     model_name: str | None,
F1582 1574 |     device: str,
F1583 1575 |     dtype: str,
F1584 1576 |     fmax: float,
F1585 1577 |     max_steps: int,
F1586 1578 |     relax_cell: bool,
F1587 1579 |     screening_cutoff: float,
F1588 1580 |     numerical_tolerance: float,
F1589 1581 |     allow_incomplete: bool,
F1590 1582 |     overwrite: bool,
F1591 1583 |     retry_failed: bool,
F1592 1584 |     output_dir: Path,
F1593 1585 | ) -> None:
F1594 1586 |     """Discover, relax, and compare MP phases from API or offline data."""
F1595 1587 |     from ..stability.competing import run_phase_diagram_pipeline
F1596 1588 |     from ..stability.mlp import MaceRelaxationBackend
F1597 1589 |
F1598 1590 |     if mp_backend == "offline" and offline_db is None:
F1599 1591 |         raise click.UsageError("--offline-db is required when --mp-backend=offline")
F1600 1592 |     api_key = _prompt_mp_api_key() if mp_backend == "api" else None
```

```
src/sssreen/cli/app.py
```

```
F1601 1593 |     calculator = MaceRelaxationBackend(
F1602 1594 |         model_path,
F1603 1595 |         device=device,
F1604 1596 |         dtype=dtype,
F1605 1597 |         model_name=model_name,
F1606 1598 |     )
F1607 1599 |     settings = CompetingPhaseSettings(
F1608 1600 |         thermo_type=thermo_type,
F1609 1601 |         max_mp_energy_above_hull=mp_max_e_hull,
F1610 1602 |         max_competing_phase_atoms=max_phase_atoms,
F1611 1603 |         api_timeout_seconds=api_timeout,
F1612 1604 |         screening_cutoff_ev_per_atom=screening_cutoff,
F1613 1605 |         numerical_tolerance_ev_per_atom=numerical_tolerance,
F1614 1606 |     )
F1615 1607 |     frame, _entries, summary = run_phase_diagram_pipeline(
F1616 1608 |         candidate_relax_results_path=candidate_relax_results_path,
F1617 1609 |         output_dir=output_dir,
F1618 1610 |         api_key=api_key,
F1619 1611 |         mp_backend=mp_backend,
F1620 1612 |         offline_db=offline_db,
F1621 1613 |         backend=calculator,
F1622 1614 |         settings=settings,
F1623 1615 |         structure_ids=structure_ids,
F1624 1616 |         fmax=fmax,
F1625 1617 |         max_steps=max_steps,
F1626 1618 |         relax_cell=relax_cell,
F1627 1619 |         allow_incomplete=allow_incomplete,
F1628 1620 |         overwrite=overwrite,
F1629 1621 |         retry_failed=retry_failed,
F1630 1622 |     )
F1631 1623 |     click.echo(
F1632 1624 |         "Stage 10 phase-diagram summary: "
F1633 1625 |         f"candidates={len(frame)} success={summary['success_count']} "
F1634 1626 |         f"uncertain={summary['uncertain_count']} failed={summary['failed_count']} "
F1635 1627 |         f"output={output_dir}"
F1636 1628 |     )
F1637 1629 |
F1638 1630 |
F1639 1631 | # -----
F1640 1632 | # Stage 11 recommendation
F1641 1633 | # -----
F1642 1634 | @cli.command("recommend")
F1643 1635 | @click.option(
F1644 1636 |     "--pairs",
F1645 1637 |     "pairs_path",
F1646 1638 |     type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1647 1639 |     required=True,
F1648 1640 |     help="Stage 5 final-pairs CSV.",
F1649 1641 | )
F1650 1642 | @click.option(
```

```
src/ssscreen/cli/app.py
```

```
F1651 1643 | "--gap-results",
F1652 1644 | "gap_results_path",
F1653 1645 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1654 1646 | required=True,
F1655 1647 | help="Normalized external high-fidelity gap results.",
F1656 1648 | )
F1657 1649 | @click.option(
F1658 1650 | "--mixing-enthalpy",
F1659 1651 | "mixing_enthalpy_path",
F1660 1652 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1661 1653 | required=True,
F1662 1654 | help="Stage 8 mixing-enthalpy CSV.",
F1663 1655 | )
F1664 1656 | @click.option(
F1665 1657 | "--phonons",
F1666 1658 | "phonons_path",
F1667 1659 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1668 1660 | required=True,
F1669 1661 | help="Stage 9 phonon-summary CSV.",
F1670 1662 | )
F1671 1663 | @click.option(
F1672 1664 | "--phase-stability",
F1673 1665 | "phase_stability_path",
F1674 1666 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1675 1667 | required=True,
F1676 1668 | help="Stage 10 phase-stability CSV.",
F1677 1669 | )
F1678 1670 | @click.option(
F1679 1671 | "--defects",
F1680 1672 | "defects_path",
F1681 1673 | type=click.Path(exists=True, dir_okay=False, path_type=Path),
F1682 1674 | help="Optional defect evidence with pair_index, status, and defect_signal.",
F1683 1675 | )
F1684 1676 | @click.option(
F1685 1677 | "--gap-method",
F1686 1678 | help="Gap method override for legacy pair tables without a gap_method column.",
F1687 1679 | )
F1688 1680 | @click.option(
F1689 1681 | "--require-defects",
F1690 1682 | is_flag=True,
F1691 1683 | help="Downgrade missing defect evidence to uncertain.",
F1692 1684 | )
F1693 1685 | @click.option(
F1694 1686 | "--promising-max-mixing",
F1695 1687 | type=click.FloatRange(min=0.0),
F1696 1688 | default=RecommendationSettings.promising_max_mixing_enthalpy_mev_per_atom,
F1697 1689 | show_default=True,
F1698 1690 | help="Maximum promising mixing enthalpy in meV/atom.",
F1699 1691 | )
F1700 1692 | @click.option(
```

```
src/sssreen/cli/app.py
```

```
F1701 1693 | "--low-priority-mixing",
F1702 1694 | type=click.FloatRange(min=0.0),
F1703 1695 | default=RecommendationSettings.low_priority_mixing_enthalpy_mev_per_atom,
F1704 1696 | show_default=True,
F1705 1697 | help="Mixing enthalpy above which a candidate is low priority (meV/atom).",
F1706 1698 | )
F1707 1699 | @click.option(
F1708 1700 | "--promising-max-hull",
F1709 1701 | type=click.FloatRange(min=0.0),
F1710 1702 | default=RecommendationSettings.promising_max_hull_ev_per_atom,
F1711 1703 | show_default=True,
F1712 1704 | help="Maximum promising same-MLIP hull distance in eV/atom.",
F1713 1705 | )
F1714 1706 | @click.option(
F1715 1707 | "--low-priority-hull",
F1716 1708 | type=click.FloatRange(min=0.0),
F1717 1709 | default=RecommendationSettings.low_priority_hull_ev_per_atom,
F1718 1710 | show_default=True,
F1719 1711 | help="Hull distance above which a candidate is low priority (eV/atom).",
F1720 1712 | )
F1721 1713 | @click.option(
F1722 1714 | "--gap-consistency-tolerance",
F1723 1715 | type=click.FloatRange(min=0.0),
F1724 1716 | default=RecommendationSettings.gap_consistency_tolerance_ev,
F1725 1717 | show_default=True,
F1726 1718 | help="Allowed endpoint-gap difference between pair and gap-result tables (eV).",
F1727 1719 | )
F1728 1720 | @click.option("--output", "output_path", type=click.Path(path_type=Path), required=True)
F1729 1721 | @click.option("--report", "report_path", type=click.Path(path_type=Path), required=True)
F1730 1722 | @click.option("--summary", "summary_path", type=click.Path(path_type=Path))
F1731 1723 | def recommend_cmd(
F1732 1724 |     pairs_path: Path,
F1733 1725 |     gap_results_path: Path,
F1734 1726 |     mixing_enthalpy_path: Path,
F1735 1727 |     phonons_path: Path,
F1736 1728 |     phase_stability_path: Path,
F1737 1729 |     defects_path: Path | None,
F1738 1730 |     gap_method: str | None,
F1739 1731 |     require_defects: bool,
F1740 1732 |     promising_max_mixing: float,
F1741 1733 |     low_priority_mixing: float,
F1742 1734 |     promising_max_hull: float,
F1743 1735 |     low_priority_hull: float,
F1744 1736 |     gap_consistency_tolerance: float,
F1745 1737 |     output_path: Path,
F1746 1738 |     report_path: Path,
F1747 1739 |     summary_path: Path | None,
F1748 1740 | ) -> None:
F1749 1741 |     """Rank candidates for further DFT or experimental study."""
F1750 1742 |     from ..stability.recommendation import generate_recommendations
```

```
src/ssscreen/cli/app.py
```

```
F1751 1743 |
F1752 1744 |     try:
F1753 1745 |         frame, summary = generate_recommendations(
F1754 1746 |             pairs_path=pairs_path,
F1755 1747 |             gap_results_path=gap_results_path,
F1756 1748 |             mixing_enthalpy_path=mixing_enthalpy_path,
F1757 1749 |             phonons_path=phonons_path,
F1758 1750 |             phase_stability_path=phase_stability_path,
F1759 1751 |             defects_path=defects_path,
F1760 1752 |             output_path=output_path,
F1761 1753 |             report_path=report_path,
F1762 1754 |             summary_path=summary_path,
F1763 1755 |             gap_method=gap_method,
F1764 1756 |             require_defects=require_defects,
F1765 1757 |             settings=RecommendationSettings(
F1766 1758 |                 promising_max_mixing_enthalpy_mev_per_atom=promising_max_mixing,
F1767 1759 |                 low_priority_mixing_enthalpy_mev_per_atom=low_priority_mixing,
F1768 1760 |                 promising_max_hull_ev_per_atom=promising_max_hull,
F1769 1761 |                 low_priority_hull_ev_per_atom=low_priority_hull,
F1770 1762 |                 gap_consistency_tolerance_ev=gap_consistency_tolerance,
F1771 1763 |             ),
F1772 1764 |         )
F1773 1765 |     except ValueError as exc:
F1774 1766 |         raise click.ClickException(str(exc)) from exc
F1775 1767 |     counts = summary["classification_counts"]
F1776 1768 |     click.echo(
F1777 1769 |         "Stage 11 recommendation summary: "
F1778 1770 |         f"candidates={len(frame)} promising={counts.get('promising', 0)} "
F1779 1771 |         f"uncertain={counts.get('uncertain', 0)} "
F1780 1772 |         f"low-priority={counts.get('low-priority', 0)} "
F1781 1773 |         f"output={output_path} report={report_path}"
F1782 1774 |     )
F1783 1775 |
F1784 1776 |
F1785 1777 | # -----
F1786 1778 | # composition-screen
F1787 1779 | # -----
F1788 1780 | @cli.command("composition-screen")
F1789 1781 | @click.option(
F1790 1782 |     "--df",
F1791 1783 |     "df_paths",
F1792 1784 |     multiple=True,
F1793 1785 |     required=True,
F1794 1786 |     type=click.Path(exists=True, path_type=Path),
F1795 1787 |     help="Pickled normalized dataset DataFrame (repeatable).",
F1796 1788 | )
F1797 1789 | @click.option(
F1798 1790 |     "--valence-ids",
F1799 1791 |     "valence_ids",
F1800 1792 |     type=click.Path(exists=True, path_type=Path),
```

```
src/sssreen/cli/app.py
```

```
F1801 1793 | help="Optional JSON list of valence-valid IDs (from `valence-filter`).",
F1802 1794 | )
F1803 1795 | @click.option("--nelems", type=int, default=2, show_default=True)
F1804 1796 | @click.option("--max-bandgap", type=float, default=1.0, show_default=True)
F1805 1797 | @click.option("--max-e-hull", "max_e_hull", type=float, default=0.0, show_default=True)
F1806 1798 | @click.option("--min-group-size", type=int, default=2, show_default=True)
F1807 1799 | @click.option("--min-x-elements", type=int, default=2, show_default=True)
F1808 1800 | @click.option(
F1809 1801 |     "--excluded-elements",
F1810 1802 |     "excluded_elements",
F1811 1803 |     multiple=True,
F1812 1804 |     default=[],
F1813 1805 |     help="Extra elements to exclude (added to the default list).",
F1814 1806 | )
F1815 1807 | @click.option(
F1816 1808 |     "--output",
F1817 1809 |     "output",
F1818 1810 |     type=click.Path(path_type=Path),
F1819 1811 |     required=True,
F1820 1812 |     help="CSV table of composition-template candidates.",
F1821 1813 | )
F1822 1814 | @click.option(
F1823 1815 |     "--summary",
F1824 1816 |     "summary",
F1825 1817 |     type=click.Path(path_type=Path),
F1826 1818 |     help="Optional JSON screening summary.",
F1827 1819 | )
F1828 1820 | def composition_screen_cmd(
F1829 1821 |     df_paths: tuple[Path, ...],
F1830 1822 |     valence_ids: Path | None,
F1831 1823 |     nelems: int,
F1832 1824 |     max_bandgap: float,
F1833 1825 |     max_e_hull: float,
F1834 1826 |     min_group_size: int,
F1835 1827 |     min_x_elements: int,
F1836 1828 |     excluded_elements: tuple[str, ...],
F1837 1829 |     output: Path,
F1838 1830 |     summary: Path | None,
F1839 1831 | ) -> None:
F1840 1832 |     """Screen composition-template candidates before structure matching."""
F1841 1833 |     from monty.serialization import dumpfn, loadfn
F1842 1834 |
F1843 1835 |     frames = [pd.read_pickle(path) for path in df_paths]
F1844 1836 |     df = pd.concat(frames, axis=0)
F1845 1837 |     if valence_ids:
F1846 1838 |         valid = loadfn(str(valence_ids))
F1847 1839 |         df = df.loc[valid]
F1848 1840 |
F1849 1841 |     thresholds = Thresholds.default(nelems).with_nelems(nelems)
F1850 1842 |     excluded = (
```

```
src/ssscreen/cli/app.py
```

```
F1851 1843 |     coerce_excluded(excluded_elements) if excluded_elements else thresholds.excluded_elements
F1852 1844 | )
F1853 1845 | candidates, report = screen_composition_candidates(
F1854 1846 |     df,
F1855 1847 |     nelems=nelems,
F1856 1848 |     max_bandgap=max_bandgap,
F1857 1849 |     max_e_hull=max_e_hull,
F1858 1850 |     excluded_elements=excluded,
F1859 1851 |     min_group_size=min_group_size,
F1860 1852 |     min_x_elements=min_x_elements,
F1861 1853 | )
F1862 1854 | output.parent.mkdir(parents=True, exist_ok=True)
F1863 1855 | candidates.to_csv(output, index=False)
F1864 1856 | if summary:
F1865 1857 |     summary.parent.mkdir(parents=True, exist_ok=True)
F1866 1858 |     dumpfn(report, str(summary))
F1867 1859 |
F1868 1860 | click.echo(
F1869 1861 |     f"Wrote {len(candidates)} composition-candidate rows "
F1870 1862 |     f"across {report['template_count']} templates to {output}"
F1871 1863 | )
F1872 1864 |
F1873 1865 |
F1874 1866 | # -----
F1875 1867 | # group
F1876 1868 | # -----
F1877 1869 | @cli.command("group")
F1878 1870 | @click.option("--df-mp", "df_mp", type=click.Path(exists=True, path_type=Path))
F1879 1871 | @click.option("--df-wbm", "df_wbm", type=click.Path(exists=True, path_type=Path))
F1880 1872 | @click.option(
F1881 1873 |     "--valence-ids",
F1882 1874 |     "valence_ids",
F1883 1875 |     type=click.Path(exists=True, path_type=Path),
F1884 1876 |     help="Optional JSON list of valence-valid IDs (from `valence-filter`).",
F1885 1877 | )
F1886 1878 | @click.option(
F1887 1879 |     "--nelems",
F1888 1880 |     type=int,
F1889 1881 |     default=2,
F1890 1882 |     show_default=True,
F1891 1883 |     help="Number of elements in the reduced composition (2 or 3).",
F1892 1884 | )
F1893 1885 | @click.option(
F1894 1886 |     "--condensed-dirs",
F1895 1887 |     "condensed_dirs",
F1896 1888 |     multiple=True,
F1897 1889 |     required=True,
F1898 1890 |     type=click.Path(exists=True, file_okay=False),
F1899 1891 |     help="Directories of robocrys condensed JSONs (repeatable).",
F1900 1892 | )
```

```
src/ssscreen/cli/app.py
```

```
F1901 1893 | @click.option(
F1902 1894 |     "--max-bandgap",
F1903 1895 |     type=float,
F1904 1896 |     default=1.0,
F1905 1897 |     show_default=True,
F1906 1898 |     help="Keep materials with PBE band gap below this (eV).",
F1907 1899 | )
F1908 1900 | @click.option(
F1909 1901 |     "--max-e-hull",
F1910 1902 |     "max_e_hull",
F1911 1903 |     type=float,
F1912 1904 |     default=0.0,
F1913 1905 |     show_default=True,
F1914 1906 |     help="Keep materials with energy above hull at most this (eV/atom).",
F1915 1907 | )
F1916 1908 | @click.option(
F1917 1909 |     "--max-natoms",
F1918 1910 |     type=int,
F1919 1911 |     default=None,
F1920 1912 |     help="Primitive-cell atom cap for the *output* structures (default 30 binary / 45 ternary).",
F1921 1913 | )
F1922 1914 | @click.option(
F1923 1915 |     "--excluded-elements",
F1924 1916 |     "excluded_elements",
F1925 1917 |     multiple=True,
F1926 1918 |     default=[],
F1927 1919 |     help="Extra elements to exclude (added to the default list).",
F1928 1920 | )
F1929 1921 | @click.option(
F1930 1922 |     "--output",
F1931 1923 |     "output",
F1932 1924 |     type=click.Path(path_type=Path),
F1933 1925 |     required=True,
F1934 1926 |     help="Output group JSON (array of StructureGroup records).",
F1935 1927 | )
F1936 1928 | @click.option("--no-verbose", is_flag=True, default=False)
F1937 1929 | def group_cmd(
F1938 1930 |     df_mp,
F1939 1931 |     df_wbm,
F1940 1932 |     valence_ids,
F1941 1933 |     nelems,
F1942 1934 |     condensed_dirs,
F1943 1935 |     max_bandgap,
F1944 1936 |     max_e_hull,
F1945 1937 |     max_natoms,
F1946 1938 |     excluded_elements,
F1947 1939 |     output,
F1948 1940 |     no_verbose,
F1949 1941 | ):
F1950 1942 |     """Group materials by composition template + environment fingerprint."""
```

```
src/ssscreen/cli/app.py
```

```
F1951 1943 |     from tqdm.auto import tqdm
F1952 1944 |
F1953 1945 |     # Load + concatenate data.
F1954 1946 |     frames = []
F1955 1947 |     if df_mp:
F1956 1948 |         frames.append(pd.read_pickle(df_mp))
F1957 1949 |     if df_wbm:
F1958 1950 |         frames.append(pd.read_pickle(df_wbm))
F1959 1951 |     if not frames:
F1960 1952 |         raise click.UsageError("Provide at least one of --df-mp / --df-wbm.")
F1961 1953 |     df_all = pd.concat(frames, axis=0)
F1962 1954 |
F1963 1955 |     # Optional valence pre-filter.
F1964 1956 |     if valence_ids:
F1965 1957 |         from monty.serialization import loadfn
F1966 1958 |
F1967 1959 |         valid = loadfn(str(valence_ids))
F1968 1960 |         df_all = df_all.loc[valid]
F1969 1961 |
F1970 1962 |     # Selection thresholds (stage 1).
F1971 1963 |     t = Thresholds.default(nelems).with_nelems(nelems)
F1972 1964 |     sel = t.selection
F1973 1965 |     if max_bandgap != sel.max_bandgap or max_e_hull != sel.max_e_hull:
F1974 1966 |         from dataclasses import replace
F1975 1967 |
F1976 1968 |         t = replace(t, selection=replace(sel, max_bandgap=max_bandgap, max_e_hull=max_e_hull))
F1977 1969 |     excluded = coerce_excluded(excluded_elements) if excluded_elements else t.excluded_elements
F1978 1970 |     if max_natoms is not None:
F1979 1971 |         from dataclasses import replace
F1980 1972 |
F1981 1973 |         t = replace(t, max_natoms=max_natoms)
F1982 1974 |
F1983 1975 |     # Stage 1: select by nelems, gap, hull.
F1984 1976 |     df_t = df_all.loc[
F1985 1977 |         (df_all["nelems"] == nelems)
F1986 1978 |         & (df_all["e_hull"] <= max_e_hull)
F1987 1979 |         & (df_all["band_gap"] < max_bandgap),
F1988 1980 |         :,
F1989 1981 |     ]
F1990 1982 |     click.echo(
F1991 1983 |         f"Selected {len(df_t)} materials (nelems={nelems}, gap<={max_bandgap}, e_hull<={max_e_hull})"
F1992 1984 |     )
F1993 1985 |
F1994 1986 |     # Stage 2: composition-template grouping.
F1995 1987 |     templates = group_by_composition_template(df_t, nelems=nelems)
F1996 1988 |     click.echo(f"{len(templates)} composition templates with >=2 members")
F1997 1989 |
F1998 1990 |     # Stage 3: environment grouping within each template.
F1999 1991 |     loader = CondenseLoader(list(condensed_dirs))
F2000 1992 |     all_groups = []
```

```
src/ssscreen/cli/app.py
```

```
F2001 1993 |     iterator = tqdm(  
F2002 1994 |         templates.items(), disable=no_verbose, desc="env grouping", total=len(templates)  
F2003 1995 |     )  
F2004 1996 |     for key, entries in iterator:  
F2005 1997 |         # entries: list of [mp_id, x_elem, band_gap]  
F2006 1998 |         mp_ids = [e[0] for e in entries]  
F2007 1999 |         condensed = [loader.get_condensed(mid) for mid in mp_ids]  
F2008 2000 |         envs = find_unique_envs(condensed)  
F2009 2001 |         groups = group_similar_structures(envs, key, mp_ids)  
F2010 2002 |         attach_band_gaps(groups, entries)  
F2011 2003 |         all_groups.extend(groups)  
F2012 2004 |  
F2013 2005 |     # Keep groups with X-element diversity.  
F2014 2006 |     import numpy as np  
F2015 2007 |  
F2016 2008 |     all_groups = [g for g in all_groups if len(np.unique(g.X_element)) > 1]  
F2017 2009 |     click.echo(f"{len(all_groups)} groups after X-diversity filter")  
F2018 2010 |  
F2019 2011 |     # Stage 4: element exclusion + gap-diversity gate.  
F2020 2012 |     valid_groups = []  
F2021 2013 |     for g in all_groups:  
F2022 2014 |         kept = apply_element_exclusion(g, excluded)  
F2023 2015 |         if kept is None:  
F2024 2016 |             continue  
F2025 2017 |         if not has_gap_diversity(kept, require_zero=t.group.require_zero_gap_member):  
F2026 2018 |             continue  
F2027 2019 |         valid_groups.append(kept)  
F2028 2020 |     click.echo(f"{len(valid_groups)} groups after element-exclusion + gap gate")  
F2029 2021 |  
F2030 2022 |     # Stage 4b (output preparation): apply size gate to the referenced structures.  
F2031 2023 |     all_ids = {mid for g in valid_groups for mid in g.mp_ids}  
F2032 2024 |     if all_ids:  
F2033 2025 |         calc_df = df_t.loc[list(all_ids)].copy()  
F2034 2026 |         calc_df = apply_size_gate(calc_df, t.max_natoms)  
F2035 2027 |         surviving = set(calc_df.index.astype(str))  
F2036 2028 |         valid_groups = [g for g in valid_groups if all(m in surviving for m in g.mp_ids)]  
F2037 2029 |     click.echo(f"{len(valid_groups)} groups survive the size gate (natoms<{t.max_natoms})")  
F2038 2030 |  
F2039 2031 |     dump_group_df(valid_groups, output)  
F2040 2032 |     click.echo(f"Wrote {len(valid_groups)} groups to {output}")  
F2041 2033 |  
F2042 2034 |  
F2043 2035 | # -----  
F2044 2036 | # pair  
F2045 2037 | # -----  
F2046 2038 | @cli.command("pair")  
F2047 2039 | @click.option(  
F2048 2040 |     "--groups",  
F2049 2041 |     "groups_path",  
F2050 2042 |     type=click.Path(exists=True, path_type=Path),
```

```
src/ssscreen/cli/app.py
```

```
F2051 2043 |     required=True,
F2052 2044 |     help="Group JSON from `ss-screen group`.",
F2053 2045 | )
F2054 2046 | @click.option(
F2055 2047 |     "--gaps",
F2056 2048 |     "gaps_path",
F2057 2049 |     type=click.Path(exists=True, path_type=Path),
F2058 2050 |     help="Legacy mbj_gaps_*_pmg_info.csv with computed gaps.",
F2059 2051 | )
F2060 2052 | @click.option(
F2061 2053 |     "--gap-results",
F2062 2054 |     "gap_results",
F2063 2055 |     multiple=True,
F2064 2056 |     type=click.Path(exists=True, path_type=Path),
F2065 2057 |     help="External gap-result CSV using the stable schema from `gap-validate`.",
F2066 2058 | )
F2067 2059 | @click.option("--low-gap", "low_gap", type=float, default=0.15, show_default=True)
F2068 2060 | @click.option("--direct-min", "direct_min", type=float, default=0.15, show_default=True)
F2069 2061 | @click.option("--direct-max", "direct_max", type=float, default=1.5, show_default=True)
F2070 2062 | @click.option("--pair-any-below", "pair_any_below", type=float, default=0.3, show_default=True)
F2071 2063 | @click.option("--pair-any-above", "pair_any_above", type=float, default=0.2, show_default=True)
F2072 2064 | @click.option("--pair-both-below", "pair_both_below", type=float, default=0.8, show_default=True)
F2073 2065 | @click.option("--method", help="Method to select from --gap-results when multiple are present.")
F2074 2066 | @click.option("--summary", type=click.Path(path_type=Path), help="Optional coverage JSON summary.")
F2075 2067 | @click.option("--output", "output", type=click.Path(path_type=Path), required=True)
F2076 2068 | def pair_cmd(
F2077 2069 |     groups_path,
F2078 2070 |     gaps_path,
F2079 2071 |     gap_results,
F2080 2072 |     low_gap,
F2081 2073 |     direct_min,
F2082 2074 |     direct_max,
F2083 2075 |     pair_any_below,
F2084 2076 |     pair_any_above,
F2085 2077 |     pair_both_below,
F2086 2078 |     method,
F2087 2079 |     summary,
F2088 2080 |     output,
F2089 2081 | ):
F2090 2082 |     """Enumerate promising alloying pairs from computed gaps."""
F2091 2083 |     from ..config import PairThresholds
F2092 2084 |     from ..pair.pairing import attach_gaps_and_compress, gaps_valid
F2093 2085 |
F2094 2086 |     t = PairThresholds(
F2095 2087 |         low_gap=low_gap,
F2096 2088 |         direct_min=direct_min,
F2097 2089 |         direct_max=direct_max,
F2098 2090 |         pair_any_below=pair_any_below,
F2099 2091 |         pair_any_above=pair_any_above,
F2100 2092 |         pair_both_below=pair_both_below,
```

```
src/ssscreen/cli/app.py -> src/ssscreen/config.py
```

```
F2101 2093 | )
F2102 2094 | if bool(gaps_path) == bool(gap_results):
F2103 2095 |     raise click.UsageError("Provide exactly one of --gaps or --gap-results.")
F2104 2096 |
F2105 2097 | if gap_results:
F2106 2098 |     from ..pair.gap_feedback import generate_pairs_from_gap_results
F2107 2099 |
F2108 2100 |     pairs_df, report = generate_pairs_from_gap_results(
F2109 2101 |         groups_path=groups_path,
F2110 2102 |         gap_paths=gap_results,
F2111 2103 |         output=output,
F2112 2104 |         summary=summary,
F2113 2105 |         method=method,
F2114 2106 |         thresholds=t,
F2115 2107 |     )
F2116 2108 |     click.echo(
F2117 2109 |         f"Wrote {len(pairs_df)} pairs to {output} "
F2118 2110 |         f"({coverage={report['covered_materials']}/{report['materials_in_groups']}})"
F2119 2111 |     )
F2120 2112 |     return
F2121 2113 |
F2122 2114 | groups = load_group_df(groups_path)
F2123 2115 | gap_map = read_mbj_gaps(gaps_path)
F2124 2116 |
F2125 2117 | all_pairs = []
F2126 2118 | for g in groups:
F2127 2119 |     compressed = attach_gaps_and_compress(g, gap_map)
F2128 2120 |     if compressed is None:
F2129 2121 |         continue
F2130 2122 |     g_aligned, gaps = compressed
F2131 2123 |     if not gaps_valid(gaps, t):
F2132 2124 |         continue
F2133 2125 |     all_pairs.extend(enumerate_pairs(g_aligned, gaps, t))
F2134 2126 |
F2135 2127 | pairs_df = pairs_to_dataframe(all_pairs)
F2136 2128 | write_pairs_csv(pairs_df, output)
F2137 2129 | click.echo(f"Wrote {len(pairs_df)} pairs to {output}")
F2138 2130 |
F2139 2131 |
F2140 2132 | if __name__ == "__main__":
F2141 2133 |     cli()
F2142 0001 | """Central configuration: numerical thresholds and element-exclusion lists.
F2143 0002 |
F2144 0003 | All magic numbers that were scattered as inline literals across the source
F2145 0004 | notebooks live here as named, documented attributes of :class:`Thresholds`.
F2146 0005 | This is the single place to audit or tune the screening criteria.
F2147 0006 |
F2148 0007 | Default values reproduce the research workspace defaults
F2149 0008 | (``pair-screening/screening-binary.ipynb`` and ``pairing_mbj_gaps_binary.ipynb``).
F2150 0009 | """
```

```
src/ssscreen/config.py
```

```
F2151 0010 |
```

```
F2152 0011 | from __future__ import annotations
```

```
F2153 0012 |
```

```
F2154 0013 | from collections.abc import Sequence
```

```
F2155 0014 | from dataclasses import dataclass, field, replace
```

```
F2156 0015 |
```

```
F2157 0016 | # -----
```

```
F2158 0017 | # Element exclusion
```

```
F2159 0018 | # -----
```

```
F2160 0019 | # Default excluded elements: radioactive/toxic, common transition metals that
```

```
F2161 0020 | # rarely form the target narrow-gap semiconductors, and the lanthanide +
```

```
F2162 0021 | # actinide series (f-electron complexity, magnetism).
```

```
F2163 0022 | # Source: pair-screening/screening-binary.ipynb cell 29.
```

```
F2164 0023 | _DEFAULT_EXCLUDED = [
```

```
F2165 0024 |     # radioactive / toxic
```

```
F2166 0025 |     "U",
```

```
F2167 0026 |     "Th",
```

```
F2168 0027 |     "Po",
```

```
F2169 0028 |     "Tl",
```

```
F2170 0029 |     "Hg",
```

```
F2171 0030 |     "Np",
```

```
F2172 0031 |     "Pu",
```

```
F2173 0032 |     "Pa",
```

```
F2174 0033 |     "Pr",
```

```
F2175 0034 |     # transition metals (excluded in the binary screen)
```

```
F2176 0035 |     "Ti",
```

```
F2177 0036 |     "Fe",
```

```
F2178 0037 |     "Co",
```

```
F2179 0038 |     "Ni",
```

```
F2180 0039 |     "Mn",
```

```
F2181 0040 |     "Cr",
```

```
F2182 0041 |     "V",
```

```
F2183 0042 |     # non-metals that distort the chemistry
```

```
F2184 0043 |     "H",
```

```
F2185 0044 |     # lanthanides
```

```
F2186 0045 |     "La",
```

```
F2187 0046 |     "Ce",
```

```
F2188 0047 |     "Nd",
```

```
F2189 0048 |     "Pm",
```

```
F2190 0049 |     "Sm",
```

```
F2191 0050 |     "Eu",
```

```
F2192 0051 |     "Gd",
```

```
F2193 0052 |     "Tb",
```

```
F2194 0053 |     "Dy",
```

```
F2195 0054 |     "Ho",
```

```
F2196 0055 |     "Er",
```

```
F2197 0056 |     "Tm",
```

```
F2198 0057 |     "Yb",
```

```
F2199 0058 |     "Lu",
```

```
F2200 0059 | ]
```

```
src/ssscren/config.py
```

```
F2201 0060 |
```

```
F2202 0061 | #: Frozen set used by :func:`ssscren.pair.filters.apply_element_exclusion`.
```

```
F2203 0062 | DEFAULT_EXCLUDED_ELEMENTS: frozenset[str] = frozenset(_DEFAULT_EXCLUDED)
```

```
F2204 0063 |
```

```
F2205 0064 |
```

```
F2206 0065 | # -----
```

```
F2207 0066 | # Permutation indices for composition-template grouping
```

```
F2208 0067 | # -----
```

```
F2209 0068 | # For a reduced composition with ``nelems`` elements, the orderings in which
```

```
F2210 0069 | # each element is treated as the variable `X` site. Mirrors
```

```
F2211 0070 | # screening-binary.ipynb cell 15 ([[0,1],[1,0]]) and
```

```
F2212 0071 | # screening-tenary.ipynb cell 5 ([[0,1,2],[0,2,1],[1,2,0]]).
```

```
F2213 0072 | _PERMUTATIONS: dict[int, list[list[int]]] = {
```

```
F2214 0073 |     2: [[0, 1], [1, 0]],
```

```
F2215 0074 |     3: [[0, 1, 2], [0, 2, 1], [1, 2, 0]],
```

```
F2216 0075 | }
```

```
F2217 0076 |
```

```
F2218 0077 |
```

```
F2219 0078 | def composition_permutations(nelems: int) -> list[list[int]]:
```

```
F2220 0079 |     """Return the per-element-as-X permutations for a reduced composition.
```

```
F2221 0080 |
```

```
F2222 0081 |     The last index in each permutation is the variable element; the leading
```

```
F2223 0082 |     indices are the fixed (spectator) elements.
```

```
F2224 0083 |     """
```

```
F2225 0084 |     if nelems not in _PERMUTATIONS:
```

```
F2226 0085 |         raise ValueError(
```

```
F2227 0086 |             f"Composition-template grouping only supports nelems in {sorted(_PERMUTATIONS)}; "
```

```
F2228 0087 |             f"got nelems={nelems}."
```

```
F2229 0088 |         )
```

```
F2230 0089 |     return [list(p) for p in _PERMUTATIONS[nelems]]
```

```
F2231 0090 |
```

```
F2232 0091 |
```

```
F2233 0092 | # -----
```

```
F2234 0093 | # Thresholds
```

```
F2235 0094 | # -----
```

```
F2236 0095 | @dataclass(frozen=True)
```

```
F2237 0096 | class SelectionThresholds:
```

```
F2238 0097 |     """Thresholds for the initial data selection (stage 1).
```

```
F2239 0098 |
```

```
F2240 0099 |     Attributes mirror the filters in ``screening-binary.ipynb`` cells 9-12.
```

```
F2241 0100 |     """
```

```
F2242 0101 |
```

```
F2243 0102 | #: Number of elements in the reduced composition (2=binary, 3=ternary).
```

```
F2244 0103 | nelems: int = 2
```

```
F2245 0104 | #: Maximum PBE band gap (eV) to keep a material. Default 1.0 (cell 12).
```

```
F2246 0105 | max_bandgap: float = 1.0
```

```
F2247 0106 | #: Maximum energy above hull (eV/atom). Use a small negative value to keep
```

```
F2248 0107 | #: only on-hull structures (``e_hull <= 0``), matching cell 12's ``<= 0.0``;
```

```
F2249 0108 | #: use ``0.01`` to mirror ``MPOffline``'s query predicate.
```

```
F2250 0109 | max_e_hull: float = 0.0
```

```
src/ssscreen/config.py
```

```
F2251 0110 |
```

```
F2252 0111 |
```

```
F2253 0112 | @dataclass(frozen=True)
```

```
F2254 0113 | class GroupThresholds:
```

```
F2255 0114 |     """Thresholds applied to environment groups (stages 3-4).
```

```
F2256 0115 |
```

```
F2257 0116 |     Mirrors ``screening-binary.ipynb`` cells 26-29.
```

```
F2258 0117 |     """
```

```
F2259 0118 |
```

```
F2260 0119 |     #: Minimum number of distinct X elements required in a group after
```

```
F2261 0120 |     #: environment matching (cell 26: ``len(np.unique(group.X_element)) > 1``).
```

```
F2262 0121 |     min_x_diversity: int = 2
```

```
F2263 0122 |     #: Require at least one zero-gap member (cell 29: ``any(x == 0 ...)``).
```

```
F2264 0123 |     require_zero_gap_member: bool = True
```

```
F2265 0124 |
```

```
F2266 0125 |
```

```
F2267 0126 | @dataclass(frozen=True)
```

```
F2268 0127 | class PairThresholds:
```

```
F2269 0128 |     """Thresholds for the pair-enumeration stage (stage 5).
```

```
F2270 0129 |
```

```
F2271 0130 |     Mirrors ``pairing_mbj_gaps_binary.ipynb`` cells 6 and 8. Two distinct sets:
```

```
F2272 0131 |     the *group validity* rule (``gaps_valid``) and the *pair* rule.
```

```
F2273 0132 |     """
```

```
F2274 0133 |
```

```
F2275 0134 |     # --- gaps_valid (cell 6): group must contain both kinds of member ---
```

```
F2276 0135 |     #: A "low" member has gap below this (eV). Default 0.15.
```

```
F2277 0136 |     low_gap: float = 0.15
```

```
F2278 0137 |     #: A "direct small-gap" member has gap in (direct_min, direct_max) eV.
```

```
F2279 0138 |     direct_min: float = 0.15
```

```
F2280 0139 |     direct_max: float = 1.5
```

```
F2281 0140 |
```

```
F2282 0141 |     # --- pair enumeration (cell 8) ---
```

```
F2283 0142 |     #: A pair is kept if some member is below this (eV). Default 0.3.
```

```
F2284 0143 |     pair_any_below: float = 0.3
```

```
F2285 0144 |     #: ...and some member is above this (eV). Default 0.2.
```

```
F2286 0145 |     pair_any_above: float = 0.2
```

```
F2287 0146 |     #: ...and neither member exceeds this (eV). Default 0.8.
```

```
F2288 0147 |     pair_both_below: float = 0.8
```

```
F2289 0148 |
```

```
F2290 0149 |
```

```
F2291 0150 | @dataclass(frozen=True)
```

```
F2292 0151 | class MLPRelaxationSettings:
```

```
F2293 0152 |     """Numerical settings and quality-control bounds for Stage 7 MLP relaxation."""
```

```
F2294 0153 |
```

```
F2295 0154 |     #: Atomic-force convergence tolerance in eV/Angstrom.
```

```
F2296 0155 |     force_tolerance: float = 0.03
```

```
F2297 0156 |     #: Maximum number of geometry-optimization steps.
```

```
F2298 0157 |     max_steps: int = 500
```

```
F2299 0158 |     #: Warn when the final volume is less than this fraction of the initial volume.
```

```
F2300 0159 |     min_volume_ratio: float = 0.5
```

src/ssscreen/config.py

```
F2301 0160 | #: Warn when the final volume exceeds this multiple of the initial volume.
F2302 0161 | max_volume_ratio: float = 2.0
F2303 0162 | #: Warn when distinct atoms are closer than this distance in Angstrom.
F2304 0163 | min_distance: float = 0.5
F2305 0164 |
F2306 0165 |
F2307 0166 | @dataclass(frozen=True)
F2308 0167 | class PhononSettings:
F2309 0168 |     """Numerical defaults for Stage 9 finite-displacement phonons."""
F2310 0169 |
F2311 0170 | #: Cartesian displacement amplitude in Angstrom.
F2312 0171 | displacement_distance: float = 0.01
F2313 0172 | #: Phonopy symmetry tolerance.
F2314 0173 | symmetry_tolerance: float = 1e-5
F2315 0174 | #: Automatic diagonal supercells target at least this lattice-vector length.
F2316 0175 | min_supercell_length: float = 10.0
F2317 0176 | #: Refuse automatically or explicitly generated supercells above this size.
F2318 0177 | max_supercell_atoms: int = 300
F2319 0178 | #: Gamma-centered q-point mesh used for the dynamical-stability screen.
F2320 0179 | mesh: tuple[int, int, int] = (20, 20, 20)
F2321 0180 | #: Number of q points per high-symmetry path segment.
F2322 0181 | band_points: int = 101
F2323 0182 | #: Frequencies below minus this tolerance are significant imaginary modes.
F2324 0183 | imaginary_tolerance_thz: float = 0.1
F2325 0184 | #: Maximum accepted Stage 7 residual force before phonon generation.
F2326 0185 | max_input_force: float = 0.01
F2327 0186 |
F2328 0187 |
F2329 0188 | @dataclass(frozen=True)
F2330 0189 | class CompetingPhaseSettings:
F2331 0190 |     """Defaults for Stage 10 MP competing-phase and convex-hull screening."""
F2332 0191 |
F2333 0192 | #: Materials Project thermo scheme used only to select source structures.
F2334 0193 | thermo_type: str = "GGA_GGA+U_R2SCAN"
F2335 0194 | #: Fetch MP structures no farther than this from the selected MP hull (eV/atom).
F2336 0195 | max_mp_energy_above_hull: float = 0.1
F2337 0196 | #: Refuse a primitive competing-phase structure above this atom count.
F2338 0197 | max_competing_phase_atoms: int = 200
F2339 0198 | #: Candidate energies at or below this MLP hull distance remain screening candidates.
F2340 0199 | screening_cutoff_ev_per_atom: float = 0.1
F2341 0200 | #: Numerical tolerance used to distinguish an on-hull candidate from metastability.
F2342 0201 | numerical_tolerance_ev_per_atom: float = 1e-6
F2343 0202 | #: Current Materials Project helper supports at most nine elements per parent system.
F2344 0203 | max_query_elements: int = 9
F2345 0204 | #: Per-request and per-chemical-system wall-clock timeout for MP API calls.
F2346 0205 | api_timeout_seconds: float = 30.0
F2347 0206 |
F2348 0207 |
F2349 0208 | @dataclass(frozen=True)
F2350 0209 | class RecommendationSettings:
```

src/ssscreen/config.py

```
F2351 0210 | """Conservative Stage 11 decision-support thresholds.
F2352 0211 |
F2353 0212 | These values are triage defaults rather than universal stability criteria.
F2354 0213 | Stage 11 records them in every summary and exposes them as CLI options.
F2355 0214 | """
F2356 0215 |
F2357 0216 | #: Mixing enthalpies at or below this remain a positive screening signal.
F2358 0217 | promising_max_mixing_enthalpy_mev_per_atom: float = 25.0
F2359 0218 | #: Mixing enthalpies above this are an explicit low-priority signal.
F2360 0219 | low_priority_mixing_enthalpy_mev_per_atom: float = 50.0
F2361 0220 | #: Same-MLIP hull distances at or below this remain a positive signal.
F2362 0221 | promising_max_hull_ev_per_atom: float = 0.025
F2363 0222 | #: Hull distances above this are an explicit low-priority signal.
F2364 0223 | low_priority_hull_ev_per_atom: float = 0.1
F2365 0224 | #: Tolerance for checking pair-table gaps against normalized gap results.
F2366 0225 | gap_consistency_tolerance_ev: float = 1e-6
F2367 0226 |
F2368 0227 |
F2369 0228 | @dataclass(frozen=True)
F2370 0229 | class Thresholds:
F2371 0230 |     """Aggregate of all screening thresholds.
F2372 0231 |
F2373 0232 |     Build with :meth:`default` for the research defaults, or :meth:`with_nelems`
F2374 0233 |     to switch binary <-> ternary while keeping the rest.
F2375 0234 |     """
F2376 0235 |
F2377 0236 |     selection: SelectionThresholds = field(default_factory=SelectionThresholds)
F2378 0237 |     group: GroupThresholds = field(default_factory=GroupThresholds)
F2379 0238 |     pair: PairThresholds = field(default_factory=PairThresholds)
F2380 0239 |     #: Maximum number of atoms in the primitive cell kept for DFT (cell 36/41:
F2381 0240 |     #: binary uses 30, ternary uses 45).
F2382 0241 |     max_natoms: int = 30
F2383 0242 |     #: Elements to exclude from A-positions and the X-position.
F2384 0243 |     excluded_elements: frozenset[str] = DEFAULT_EXCLUDED_ELEMENTS
F2385 0244 |
F2386 0245 |     @staticmethod
F2387 0246 |     def default(nelems: int = 2) -> Thresholds:
F2388 0247 |         """Research defaults for the given element count."""
F2389 0248 |         max_natoms = 30 if nelems == 2 else 45
F2390 0249 |         return Thresholds(
F2391 0250 |             selection=SelectionThresholds(nelems=nelems),
F2392 0251 |             max_natoms=max_natoms,
F2393 0252 |         )
F2394 0253 |
F2395 0254 |     def with_nelems(self, nelems: int) -> Thresholds:
F2396 0255 |         """Return a copy with ``nelems`` (and the matching natoms cap) set."""
F2397 0256 |         return replace(
F2398 0257 |             self,
F2399 0258 |             selection=replace(self.selection, nelems=nelems),
F2400 0259 |             max_natoms=30 if nelems == 2 else 45,
```

```
src/ssscreen/config.py -> src/ssscreen/data/condense.py
```

```
F2401 0260 |     )
F2402 0261 |
F2403 0262 |     @property
F2404 0263 |     def excluded(self) -> set[str]:
F2405 0264 |         """The excluded-element set as a plain ``set`` (for easy membership tests)."""
F2406 0265 |         return set(self.excluded_elements)
F2407 0266 |
F2408 0267 |
F2409 0268 | def coerce_excluded(extra: Sequence[str] | None) -> frozenset[str]:
F2410 0269 |     """Helper for the CLI: merge user-supplied extras into the default set."""
F2411 0270 |     base = set(DEFAULT_EXCLUDED_ELEMENTS)
F2412 0271 |     if extra:
F2413 0272 |         base.update(extra)
F2414 0273 |     return frozenset(base)
F2415 0001 | """Data layer: I/O for condensed structures, group tables, and gap tables."""
F2416 0002 |
F2417 0003 | from .condense import CondenseSummary, condense_paths
F2418 0004 | from .io import (
F2419 0005 |     CondenseLoader,
F2420 0006 |     dump_group_df,
F2421 0007 |     groups_to_records,
F2422 0008 |     load_group_df,
F2423 0009 |     read_mbj_gaps,
F2424 0010 |     write_pairs_csv,
F2425 0011 | )
F2426 0012 |
F2427 0013 | __all__ = [
F2428 0014 |     "CondenseSummary",
F2429 0015 |     "CondenseLoader",
F2430 0016 |     "condense_paths",
F2431 0017 |     "dump_group_df",
F2432 0018 |     "groups_to_records",
F2433 0019 |     "load_group_df",
F2434 0020 |     "read_mbj_gaps",
F2435 0021 |     "write_pairs_csv",
F2436 0022 | ]
F2437 0001 | """Robocrys condensation helpers for ``ss-screen condense``.
F2438 0002 |
F2439 0003 | This ports the single-structure / batch condensation pattern from the read-only
F2440 0004 | reference scripts ``mp-condense/condense_one.py`` and ``mp-condense/condense.py``.
F2441 0005 | The optional ``robocrys`` dependency is imported only inside the runtime path so
F2442 0006 | the core package and default tests do not need it installed.
F2443 0007 | """
F2444 0008 |
F2445 0009 | from __future__ import annotations
F2446 0010 |
F2447 0011 | import hashlib
F2448 0012 | import json
F2449 0013 | import os
F2450 0014 | import tempfile
```

```
src/ssscreen/data/condense.py
```

```
F2451 0015 | from collections.abc import Sequence
```

```
F2452 0016 | from dataclasses import asdict, dataclass
```

```
F2453 0017 | from pathlib import Path
```

```
F2454 0018 | from typing import Any
```

```
F2455 0019 |
```

```
F2456 0020 | import pandas as pd
```

```
F2457 0021 | from monty.serialization import dumpfn, loadfn
```

```
F2458 0022 | from pymatgen.core import Structure
```

```
F2459 0023 |
```

```
F2460 0024 |
```

```
F2461 0025 | @dataclass(frozen=True)
```

```
F2462 0026 | class CondenseSummary:
```

```
F2463 0027 |     """Counts from a condensation run."""
```

```
F2464 0028 |
```

```
F2465 0029 |     written: int = 0
```

```
F2466 0030 |     skipped: int = 0
```

```
F2467 0031 |     failed: int = 0
```

```
F2468 0032 |
```

```
F2469 0033 |
```

```
F2470 0034 | @dataclass(frozen=True)
```

```
F2471 0035 | class CondenseRecord:
```

```
F2472 0036 |     """Metadata for one attempted condensed-structure archive member."""
```

```
F2473 0037 |
```

```
F2474 0038 |     material_id: str
```

```
F2475 0039 |     status: str
```

```
F2476 0040 |     output_path: str
```

```
F2477 0041 |     source_path: str = ""
```

```
F2478 0042 |     formula: str = ""
```

```
F2479 0043 |     error: str = ""
```

```
F2480 0044 |     file_size: int = 0
```

```
F2481 0045 |     sha256: str = ""
```

```
F2482 0046 |
```

```
F2483 0047 |
```

```
F2484 0048 | REQUIRED_CONDENSED_KEYS = {
```

```
F2485 0049 |     "formula",
```

```
F2486 0050 |     "spg_symbol",
```

```
F2487 0051 |     "crystal_system",
```

```
F2488 0052 |     "mineral",
```

```
F2489 0053 |     "dimensionality",
```

```
F2490 0054 |     "sites",
```

```
F2491 0055 |     "distances",
```

```
F2492 0056 |     "angles",
```

```
F2493 0057 |     "nnn_distances",
```

```
F2494 0058 |     "components",
```

```
F2495 0059 |     "component_makeup",
```

```
F2496 0060 |     "vdw_heterostructure_info",
```

```
F2497 0061 | }
```

```
F2498 0062 |
```

```
F2499 0063 |
```

```
F2500 0064 | def expand_input_paths(inputs: Sequence[str | Path]) -> list[Path]:
```

```
src/ssscreen/stability/phonon.py
```

```
B0001 0903 |         rows.append(row)
B0002 0904 |         continue
B0003 0905 |     try:
B0004 0906 |         row = _collect_one(
B0005 0907 |             tasks=written,
B0006 0908 |             force_records=force_by_task,
B0007 0909 |             manifest_path=manifest_path,
B0008 0910 |             output_dir=output_dir,
B0009 0911 |             mesh=mesh,
B0010 0912 |             band_points=band_points,
B0011 0913 |             imaginary_tolerance=imaginary_tolerance,
B0012 0914 |             subtract_reference_forces=subtract_reference_forces,
B0013 0915 |             symmetrize_force_constants=symmetrize_force_constants,
B0014 0916 |         )
B0015 0917 |     except Exception as exc:
B0016 0918 |         row = _base_summary(written[0])
B0017 0919 |         expected = len(written)
B0018 0920 |         success_count = sum(
B0019 0921 |             force_by_task.get(str(task["task_id"]), {}).get("status") == "success"
B0020 0922 |             for task in written
B0021 0923 |         )
B0022 0924 |         row.update(
B0023 0925 |             {
B0024 0926 |                 "force_success_count": success_count,
B0025 0927 |                 "force_coverage": success_count / expected if expected else 0.0,
B0026 0928 |                 "mesh": json.dumps(list(mesh)),
B0027 0929 |                 "imaginary_tolerance_THz": imaginary_tolerance,
B0028 0930 |                 "error": f"{type(exc).__name__}: {exc}":["2000],
B0029 0931 |             }
B0030 0932 |         )
B0031 0933 |         rows.append(row)
B0032 0934 |
B0033 0935 |     frame = pd.DataFrame(rows, columns=SUMMARY_COLUMNS)
B0034 0936 |     status_counts = Counter(str(value) for value in frame["status"])
B0035 0937 |     dynamical_counts = Counter(str(value) for value in frame["dynamical_status"])
B0036 0938 |     report = {
B0037 0939 |         "schema_version": PHONON_SCHEMA_VERSION,
B0038 0940 |         "manifest_path": str(manifest_path),
B0039 0941 |         "force_results_path": str(force_results_path),
B0040 0942 |         "output_dir": str(output_dir),
B0041 0943 |         "summary_path": str(summary_path),
B0042 0944 |         "structure_count": len(frame),
B0043 0945 |         "status_counts": dict(sorted(status_counts.items())),
B0044 0946 |         "dynamical_status_counts": dict(sorted(dynamical_counts.items())),
B0045 0947 |         "settings": {
B0046 0948 |             "mesh": list(mesh),
B0047 0949 |             "band_points": band_points,
B0048 0950 |             "imaginary_tolerance_THz": imaginary_tolerance,
B0049 0951 |             "subtract_reference_forces": subtract_reference_forces,
B0050 0952 |             "symmetrize_force_constants": symmetrize_force_constants,
```

```
src/ssscreen/stability/phonon.py

B0051 0953 |     },
B0052 0954 |     "scientific_scope": (
B0053 0955 |         "harmonic finite-displacement MLIP screen; no significant imaginary modes "
B0054 0956 |         "does not establish finite-temperature miscibility or thermodynamic stability"
B0055 0957 |     ),
B0056 0958 | }
B0057 0959 | _atomic_write_dataframe(frame, summary_path)
B0058 0960 | _atomic_write_text(report_path, json.dumps(report, indent=2, sort_keys=True) + "\n")
B0059 0961 | return frame, report
B0060 0962 |
B0061 0963 |
B0062 0964 | def run_phonon_pipeline(
B0063 0965 |     *,
B0064 0966 |     relax_results_path: str | Path,
B0065 0967 |     output_dir: str | Path,
B0066 0968 |     backend: ForceBackend,
B0067 0969 |     settings: PhononSettings | None = None,
B0068 0970 |     structure_ids: tuple[str, ...] | list[str] = (),
B0069 0971 |     include_endmembers: bool = False,
B0070 0972 |     supercell: tuple[int, int, int] | None = None,
B0071 0973 |     allow_loose_input: bool = False,
B0072 0974 |     overwrite: bool = False,
B0073 0975 |     retry_failed: bool = False,
B0074 0976 | ) -> tuple[pd.DataFrame, dict[str, Any]]:
B0075 0977 |     """Run export, MACE force evaluation, and Phonopy collection end to end."""
B0076 0978 |     settings = settings or PhononSettings()
B0077 0979 |     output_dir = Path(output_dir)
B0078 0980 |     manifest_path = output_dir / "phonon_jobs.jsonl"
B0079 0981 |     force_results_path = output_dir / "phonon_force_results.jsonl"
B0080 0982 |     summary_path = output_dir / "phonon_summary.csv"
B0081 0983 |     report_path = output_dir / "phonon_summary.json"
B0082 0984 |     export_phonon_tasks(
B0083 0985 |         relax_results_path=relax_results_path,
B0084 0986 |         output_dir=output_dir / "inputs",
B0085 0987 |         manifest_path=manifest_path,
B0086 0988 |         structure_ids=structure_ids,
B0087 0989 |         include_endmembers=include_endmembers,
B0088 0990 |         supercell=supercell,
B0089 0991 |         displacement_distance=settings.displacement_distance,
B0090 0992 |         symmetry_tolerance=settings.symmetry_tolerance,
B0091 0993 |         min_supercell_length=settings.min_supercell_length,
B0092 0994 |         max_supercell_atoms=settings.max_supercell_atoms,
B0093 0995 |         max_input_force=settings.max_input_force,
B0094 0996 |         allow_loose_input=allow_loose_input,
B0095 0997 |     )
B0096 0998 |     evaluate_phonon_forces(
B0097 0999 |         manifest_path=manifest_path,
B0098 1000 |         output_dir=output_dir / "forces",
B0099 1001 |         results_path=force_results_path,
B0100 1002 |         backend=backend,
```

```
src/ssscreen/stability/phonon.py -> src/ssscreen/stability/recommendation.py
```

```
B0101 1003 |     overwrite=overwrite,
B0102 1004 |     retry_failed=retry_failed,
B0103 1005 | )
B0104 1006 | return collect_phonon_results(
B0105 1007 |     manifest_path=manifest_path,
B0106 1008 |     force_results_path=force_results_path,
B0107 1009 |     output_dir=output_dir / "results",
B0108 1010 |     summary_path=summary_path,
B0109 1011 |     report_path=report_path,
B0110 1012 |     mesh=settings.mesh,
B0111 1013 |     band_points=settings.band_points,
B0112 1014 |     imaginary_tolerance=settings.imaginary_tolerance_thz,
B0113 1015 | )
B0114 0001 | """Auditable Stage 11 recommendations from Stage 5 and Stage 8--10 evidence."""
B0115 0002 |
B0116 0003 | from __future__ import annotations
B0117 0004 |
B0118 0005 | import hashlib
B0119 0006 | import json
B0120 0007 | import math
B0121 0008 | import os
B0122 0009 | import tempfile
B0123 0010 | from collections import Counter
B0124 0011 | from dataclasses import dataclass, field
B0125 0012 | from datetime import UTC, datetime
B0126 0013 | from pathlib import Path
B0127 0014 | from typing import Any
B0128 0015 |
B0129 0016 | import pandas as pd
B0130 0017 |
B0131 0018 | from ..config import RecommendationSettings
B0132 0019 |
B0133 0020 | RECOMMENDATION_SCHEMA_VERSION = 1
B0134 0021 | CLASSIFICATION_ORDER = {"promising": 0, "uncertain": 1, "low-priority": 2}
B0135 0022 | EVIDENCE_LEVEL_LABELS = {
B0136 0023 |     "L2": "structure-matched pair",
B0137 0024 |     "L3": "high-fidelity endpoint gaps",
B0138 0025 |     "L4": "SQS/MLP mixing evidence",
B0139 0026 |     "L5": "phonon and competing-phase evidence",
B0140 0027 |     "L6": "defect evidence",
B0141 0028 | }
B0142 0029 | REQUIRED_PAIR_COLUMNS = ("comp_a", "comp_b", "mp_id_a", "mp_id_b")
B0143 0030 | REQUIRED_GAP_COLUMNS = ("material_id", "method", "band_gap", "is_direct", "status")
B0144 0031 | REQUIRED_MIXING_COLUMNS = (
B0145 0032 |     "pair_index",
B0146 0033 |     "sqs_structure_id",
B0147 0034 |     "mixing_enthalpy_meV_per_atom",
B0148 0035 |     "status",
B0149 0036 |     "usable_for_screening",
B0150 0037 | )
```

```
src/ssscreen/stability/recommendation.py
```

```
B0151 0038 | REQUIRED_PHONON_COLUMNS = (  
B0152 0039 |     "pair_index",  
B0153 0040 |     "structure_id",  
B0154 0041 |     "status",  
B0155 0042 |     "dynamical_status",  
B0156 0043 | )  
  
B0157 0044 | REQUIRED_PHASE_COLUMNS = (  
B0158 0045 |     "pair_index",  
B0159 0046 |     "structure_id",  
B0160 0047 |     "energy_above_hull_eV_per_atom",  
B0161 0048 |     "hull_signal",  
B0162 0049 |     "status",  
B0163 0050 |     "usable_for_screening",  
B0164 0051 | )  
  
B0165 0052 | REQUIRED_DEFECT_COLUMNS = ("pair_index", "status", "defect_signal")  
  
B0166 0053 | OUTPUT_COLUMNS = [  
B0167 0054 |     "schema_version",  
B0168 0055 |     "recommendation_id",  
B0169 0056 |     "rank",  
B0170 0057 |     "classification",  
B0171 0058 |     "evidence_level",  
B0172 0059 |     "evidence_level_label",  
B0173 0060 |     "pair_index",  
B0174 0061 |     "source_group_index",  
B0175 0062 |     "structure_id",  
B0176 0063 |     "target_fraction_b",  
B0177 0064 |     "actual_fraction_b",  
B0178 0065 |     "fraction_basis",  
B0179 0066 |     "mp_id_a",  
B0180 0067 |     "mp_id_b",  
B0181 0068 |     "comp_a",  
B0182 0069 |     "comp_b",  
B0183 0070 |     "gap_method",  
B0184 0071 |     "gap_a_eV",  
B0185 0072 |     "gap_b_eV",  
B0186 0073 |     "gap_direct_a",  
B0187 0074 |     "gap_direct_b",  
B0188 0075 |     "gap_evidence_status",  
B0189 0076 |     "mixing_status",  
B0190 0077 |     "mixing_enthalpy_meV_per_atom",  
B0191 0078 |     "mixing_signal",  
B0192 0079 |     "phonon_status",  
B0193 0080 |     "dynamical_status",  
B0194 0081 |     "minimum_mesh_frequency_THz",  
B0195 0082 |     "phonon_nac_applied",  
B0196 0083 |     "phase_status",  
B0197 0084 |     "hull_signal",  
B0198 0085 |     "energy_above_hull_eV_per_atom",  
B0199 0086 |     "competing_set_complete",  
B0200 0087 |     "mp_source_backend",
```

```
src/ssscreen/stability/recommendation.py
```

```
B0201 0088 | "mp_source_scope",
B0202 0089 | "defect_status",
B0203 0090 | "defect_signal",
B0204 0091 | "defect_record_count",
B0205 0092 | "model_sha256",
B0206 0093 | "model_compatibility_status",
B0207 0094 | "positive_evidence_count",
B0208 0095 | "risk_count",
B0209 0096 | "missing_data_count",
B0210 0097 | "positive_evidence",
B0211 0098 | "risks",
B0212 0099 | "missing_data",
B0213 0100 | "source_warnings",
B0214 0101 | "recommended_next_steps",
B0215 0102 | "rationale",
B0216 0103 | ]
B0217 0104 |
B0218 0105 |
B0219 0106 | @dataclass
B0220 0107 | class _Assessment:
B0221 0108 |     positive: list[str] = field(default_factory=list)
B0222 0109 |     risks: list[str] = field(default_factory=list)
B0223 0110 |     missing: list[str] = field(default_factory=list)
B0224 0111 |     uncertain: list[str] = field(default_factory=list)
B0225 0112 |     hard_negative: list[str] = field(default_factory=list)
B0226 0113 |     source_warnings: list[str] = field(default_factory=list)
B0227 0114 |     model_sha256_values: set[str] = field(default_factory=set)
B0228 0115 |
B0229 0116 |     def add_positive(self, code: str) -> None:
B0230 0117 |         _append_unique(self.positive, code)
B0231 0118 |
B0232 0119 |     def add_risk(self, code: str, *, uncertain: bool = False, hard: bool = False) -> None:
B0233 0120 |         _append_unique(self.risks, code)
B0234 0121 |         if uncertain:
B0235 0122 |             _append_unique(self.uncertain, code)
B0236 0123 |         if hard:
B0237 0124 |             _append_unique(self.hard_negative, code)
B0238 0125 |
B0239 0126 |     def add_missing(self, code: str, *, required: bool = True) -> None:
B0240 0127 |         _append_unique(self.missing, code)
B0241 0128 |         if required:
B0242 0129 |             _append_unique(self.uncertain, code)
B0243 0130 |
B0244 0131 |     def add_warning(self, stage: str, value: object) -> None:
B0245 0132 |         text = _clean_text(value)
B0246 0133 |         if text:
B0247 0134 |             _append_unique(self.source_warnings, f"{stage}: {text}")
B0248 0135 |
B0249 0136 |     def add_model(self, value: object) -> None:
B0250 0137 |         text = _clean_text(value)
```

```
src/ssscreen/stability/recommendation.py
```

```
B0251 0138 |         if text:
B0252 0139 |             self.model_sha256_values.add(text)
B0253 0140 |
B0254 0141 |
B0255 0142 | def _append_unique(values: list[str], value: str) -> None:
B0256 0143 |     if value not in values:
B0257 0144 |         values.append(value)
B0258 0145 |
B0259 0146 |
B0260 0147 | def _atomic_write_text(path: Path, text: str) -> None:
B0261 0148 |     path.parent.mkdir(parents=True, exist_ok=True)
B0262 0149 |     descriptor, temporary = tempfile.mkstemp(prefix=f"{path.name}.", dir=path.parent)
B0263 0150 |     try:
B0264 0151 |         with os.fdopen(descriptor, "w", encoding="utf-8") as handle:
B0265 0152 |             handle.write(text)
B0266 0153 |             os.replace(temporary, path)
B0267 0154 |     except Exception:
B0268 0155 |         Path(temporary).unlink(missing_ok=True)
B0269 0156 |         raise
B0270 0157 |
B0271 0158 |
B0272 0159 | def _atomic_write_dataframe(frame: pd.DataFrame, path: Path) -> None:
B0273 0160 |     path.parent.mkdir(parents=True, exist_ok=True)
B0274 0161 |     descriptor, temporary = tempfile.mkstemp(
B0275 0162 |         prefix=f"{path.stem}.", suffix=path.suffix or ".csv", dir=path.parent
B0276 0163 |     )
B0277 0164 |     os.close(descriptor)
B0278 0165 |     try:
B0279 0166 |         frame.to_csv(temporary, index=False)
B0280 0167 |         os.replace(temporary, path)
B0281 0168 |     except Exception:
B0282 0169 |         Path(temporary).unlink(missing_ok=True)
B0283 0170 |         raise
B0284 0171 |
B0285 0172 |
B0286 0173 | def _clean_text(value: object) -> str | None:
B0287 0174 |     if value is None:
B0288 0175 |         return None
B0289 0176 |     try:
B0290 0177 |         if bool(pd.isna(value)):
B0291 0178 |             return None
B0292 0179 |     except (TypeError, ValueError):
B0293 0180 |         pass
B0294 0181 |     text = str(value).strip()
B0295 0182 |     return text or None
B0296 0183 |
B0297 0184 |
B0298 0185 | def _finite_float(value: object) -> float | None:
B0299 0186 |     if value is None:
B0300 0187 |         return None
```

```
src/ssscreen/stability/recommendation.py
```

```
B0301 0188 | try:
B0302 0189 |     number = float(value)
B0303 0190 | except (TypeError, ValueError):
B0304 0191 |     return None
B0305 0192 | return number if math.isfinite(number) else None
B0306 0193 |
B0307 0194 |
B0308 0195 | def _as_bool(value: object) -> bool | None:
B0309 0196 |     if value is None:
B0310 0197 |         return None
B0311 0198 | try:
B0312 0199 |     if bool(pd.isna(value)):
B0313 0200 |         return None
B0314 0201 | except (TypeError, ValueError):
B0315 0202 |     pass
B0316 0203 | if isinstance(value, bool):
B0317 0204 |     return value
B0318 0205 | if isinstance(value, (int, float)) and value in {0, 1}:
B0319 0206 |     return bool(value)
B0320 0207 | text = str(value).strip().lower()
B0321 0208 | if text in {"true", "t", "yes", "y", "1"}:
B0322 0209 |     return True
B0323 0210 | if text in {"false", "f", "no", "n", "0"}:
B0324 0211 |     return False
B0325 0212 | return None
B0326 0213 |
B0327 0214 |
B0328 0215 | def _json_list(values: list[str]) -> str:
B0329 0216 |     return json.dumps(values, ensure_ascii=True)
B0330 0217 |
B0331 0218 |
B0332 0219 | def _load_csv(path: Path, *, label: str, required: tuple[str, ...]) -> pd.DataFrame:
B0333 0220 |     frame = pd.read_csv(path)
B0334 0221 |     unnamed = [column for column in frame.columns if str(column).startswith("Unnamed:")]
B0335 0222 |     if unnamed:
B0336 0223 |         frame = frame.drop(columns=unnamed)
B0337 0224 |     missing = [column for column in required if column not in frame.columns]
B0338 0225 |     if missing:
B0339 0226 |         raise ValueError(f"{label} is missing required columns: {' ', ' '.join(missing)}")
B0340 0227 |     return frame
B0341 0228 |
B0342 0229 |
B0343 0230 | def _prepare_pair_index(frame: pd.DataFrame, *, label: str, pair_count: int) -> pd.DataFrame:
B0344 0231 |     prepared = frame.copy()
B0345 0232 |     prepared["__pair_index"] = pd.Series(pd.NA, index=prepared.index, dtype="Int64")
B0346 0233 |     present = prepared["pair_index"].notna()
B0347 0234 |     if not present.any():
B0348 0235 |         return prepared
B0349 0236 |     numeric = pd.to_numeric(prepared.loc[present, "pair_index"], errors="coerce")
B0350 0237 |     if numeric.isna().any() or any(float(value) != int(value) for value in numeric):
```

```
src/ssscreen/stability/recommendation.py
```

```
B0351 0238 |     raise ValueError(f"{label} contains a non-integer pair_index")
B0352 0239 | indices = numeric.astype(int)
B0353 0240 | invalid = sorted({int(value) for value in indices if value < 0 or value >= pair_count})
B0354 0241 | if invalid:
B0355 0242 |     raise ValueError(f"{label} contains out-of-range pair_index values: {invalid}")
B0356 0243 | prepared.loc[present, "__pair_index"] = indices.to_numpy()
B0357 0244 | return prepared
B0358 0245 |
B0359 0246 |
B0360 0247 | def _validate_settings(settings: RecommendationSettings) -> None:
B0361 0248 |     values = {
B0362 0249 |         "promising mixing threshold": settings.promising_max_mixing_enthalpy_mev_per_atom,
B0363 0250 |         "low-priority mixing threshold": settings.low_priority_mixing_enthalpy_mev_per_atom,
B0364 0251 |         "promising hull threshold": settings.promising_max_hull_ev_per_atom,
B0365 0252 |         "low-priority hull threshold": settings.low_priority_hull_ev_per_atom,
B0366 0253 |         "gap consistency tolerance": settings.gap_consistency_tolerance_ev,
B0367 0254 |     }
B0368 0255 |     invalid = [name for name, value in values.items() if value < 0 or not math.isfinite(value)]
B0369 0256 |     if invalid:
B0370 0257 |         raise ValueError(f"recommendation settings must be finite and non-negative: {invalid}")
B0371 0258 |     if (
B0372 0259 |         settings.promising_max_mixing_enthalpy_mev_per_atom
B0373 0260 |         > settings.low_priority_mixing_enthalpy_mev_per_atom
B0374 0261 |     ):
B0375 0262 |         raise ValueError("promising mixing threshold cannot exceed the low-priority threshold")
B0376 0263 |     if settings.promising_max_hull_ev_per_atom > settings.low_priority_hull_ev_per_atom:
B0377 0264 |         raise ValueError("promising hull threshold cannot exceed the low-priority threshold")
B0378 0265 |
B0379 0266 |
B0380 0267 | def _rows_for_pair_and_structure(
B0381 0268 |     frame: pd.DataFrame,
B0382 0269 |     *,
B0383 0270 |     pair_index: int,
B0384 0271 |     structure_column: str,
B0385 0272 |     structure_id: str | None,
B0386 0273 | ) -> pd.DataFrame:
B0387 0274 |     rows = frame.loc[frame["__pair_index"] == pair_index]
B0388 0275 |     if structure_column not in rows.columns:
B0389 0276 |         return rows if structure_id is None else rows.iloc[0:0]
B0390 0277 |     normalized = rows[structure_column].map(_clean_text)
B0391 0278 |     if structure_id is None:
B0392 0279 |         return rows.loc[normalized.isna()]
B0393 0280 |     return rows.loc[normalized == structure_id]
B0394 0281 |
B0395 0282 |
B0396 0283 | def _single_row(
B0397 0284 |     rows: pd.DataFrame,
B0398 0285 |     *,
B0399 0286 |     stage: str,
B0400 0287 |     assessment: _Assessment,
```

src/ssscreen/stability/recommendation.py

```

B0401 0288 | ) -> tuple[pd.Series | None, str]:
B0402 0289 |     if rows.empty:
B0403 0290 |         assessment.add_missing(f"{stage}:missing_source")
B0404 0291 |         return None, "missing_source"
B0405 0292 |     if len(rows) != 1:
B0406 0293 |         assessment.add_risk(f"{stage}:ambiguous_records", uncertain=True)
B0407 0294 |         return None, "incompatible"
B0408 0295 |     return rows.iloc[0], "available"
B0409 0296 |
B0410 0297 |
B0411 0298 | def _recommendation_id(pair_index: int, structure_id: str | None, pair: pd.Series) -> str:
B0412 0299 |     payload = {
B0413 0300 |         "pair_index": pair_index,
B0414 0301 |         "structure_id": structure_id,
B0415 0302 |         "mp_id_a": _clean_text(pair.get("mp_id_a")),
B0416 0303 |         "mp_id_b": _clean_text(pair.get("mp_id_b")),
B0417 0304 |         "gap_method": _clean_text(pair.get("gap_method")),
B0418 0305 |     }
B0419 0306 |     digest = hashlib.sha256(
B0420 0307 |         json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
B0421 0308 |     ).hexdigest()
B0422 0309 |     return f"recommendation-{digest[:20]}"
B0423 0310 |
B0424 0311 |
B0425 0312 | def _evaluate_gap_evidence(
B0426 0313 |     pair: pd.Series,
B0427 0314 |     gaps: pd.DataFrame,
B0428 0315 |     *,
B0429 0316 |     method_override: str | None,
B0430 0317 |     settings: RecommendationSettings,
B0431 0318 |     assessment: _Assessment,
B0432 0319 | ) -> dict[str, Any]:
B0433 0320 |     pair_method = _clean_text(pair.get("gap_method"))
B0434 0321 |     method = _clean_text(method_override) or pair_method
B0435 0322 |     if method_override and pair_method and method_override != pair_method:
B0436 0323 |         assessment.add_risk("gap:pair_method_conflict", uncertain=True)
B0437 0324 |         return {
B0438 0325 |             "gap_method": method_override,
B0439 0326 |             "gap_evidence_status": "incompatible",
B0440 0327 |             "gap_a_eV": _finite_float(pair.get("gap_a")),
B0441 0328 |             "gap_b_eV": _finite_float(pair.get("gap_b")),
B0442 0329 |             "gap_direct_a": _as_bool(pair.get("gap_direct_a")),
B0443 0330 |             "gap_direct_b": _as_bool(pair.get("gap_direct_b")),
B0444 0331 |             "complete": False,
B0445 0332 |         }
B0446 0333 |
B0447 0334 |     endpoint_ids = [_clean_text(pair.get("mp_id_a")), _clean_text(pair.get("mp_id_b"))]
B0448 0335 |     if method is None:
B0449 0336 |         endpoint_rows = gaps.loc[gaps["material_id"].astype(str).isin(endpoint_ids)]
B0450 0337 |         methods = sorted(

```

src/sssreen/stability/recommendation.py

```
B0451 0338 |         {text for value in endpoint_rows["method"] if (text := _clean_text(value))}
B0452 0339 |     )
B0453 0340 |     if len(methods) == 1:
B0454 0341 |         method = methods[0]
B0455 0342 |     else:
B0456 0343 |         code = "gap:missing_method" if not methods else "gap:ambiguous_method"
B0457 0344 |     (
B0458 0345 |         assessment.add_missing(code)
B0459 0346 |         if not methods
B0460 0347 |         else assessment.add_risk(code, uncertain=True)
B0461 0348 |     )
B0462 0349 |     return {
B0463 0350 |         "gap_method": None,
B0464 0351 |         "gap_evidence_status": "missing_source" if not methods else "incompatible",
B0465 0352 |         "gap_a_eV": _finite_float(pair.get("gap_a")),
B0466 0353 |         "gap_b_eV": _finite_float(pair.get("gap_b")),
B0467 0354 |         "gap_direct_a": _as_bool(pair.get("gap_direct_a")),
B0468 0355 |         "gap_direct_b": _as_bool(pair.get("gap_direct_b")),
B0469 0356 |         "complete": False,
B0470 0357 |     }
B0471 0358 |
B0472 0359 |     selected = gaps.loc[gaps["method"].astype(str) == method]
B0473 0360 |     result: dict[str, Any] = {"gap_method": method, "complete": True}
B0474 0361 |     result_status = "complete"
B0475 0362 |     direct_values: list[bool | None] = []
B0476 0363 |     for endpoint, material_id in zip(("a", "b"), endpoint_ids, strict=True):
B0477 0364 |         rows = selected.loc[selected["material_id"].astype(str) == str(material_id)]
B0478 0365 |         if rows.empty:
B0479 0366 |             assessment.add_missing(f"gap:{endpoint}:missing_source")
B0480 0367 |             result["complete"] = False
B0481 0368 |             result_status = "missing_source"
B0482 0369 |             result[f"gap_{endpoint}_eV"] = _finite_float(pair.get(f"gap_{endpoint}"))
B0483 0370 |             direct = _as_bool(pair.get(f"gap_direct_{endpoint}"))
B0484 0371 |             result[f"gap_direct_{endpoint}"] = direct
B0485 0372 |             direct_values.append(direct)
B0486 0373 |             continue
B0487 0374 |         if len(rows) != 1:
B0488 0375 |             assessment.add_risk(f"gap:{endpoint}:ambiguous_records", uncertain=True)
B0489 0376 |             result["complete"] = False
B0490 0377 |             result_status = "incompatible"
B0491 0378 |             result[f"gap_{endpoint}_eV"] = _finite_float(pair.get(f"gap_{endpoint}"))
B0492 0379 |             direct = _as_bool(pair.get(f"gap_direct_{endpoint}"))
B0493 0380 |             result[f"gap_direct_{endpoint}"] = direct
B0494 0381 |             direct_values.append(direct)
B0495 0382 |             continue
B0496 0383 |
B0497 0384 |     row = rows.iloc[0]
B0498 0385 |     source_status = (_clean_text(row.get("status"))) or "missing").lower()
B0499 0386 |     gap = _finite_float(row.get("band_gap"))
B0500 0387 |     direct = _as_bool(row.get("is_direct"))
```

```
src/ssscreen/stability/recommendation.py
```

```
B0501 0388 |     result[f"gap_{endpoint}_eV"] = gap
B0502 0389 |     result[f"gap_direct_{endpoint}"] = direct
B0503 0390 |     direct_values.append(direct)
B0504 0391 |     if source_status != "success" or gap is None or gap < 0:
B0505 0392 |         assessment.add_missing(f"gap:{endpoint}:{source_status}")
B0506 0393 |     result["complete"] = False
B0507 0394 |     result_status = "failed"
B0508 0395 |     continue
B0509 0396 |
B0510 0397 |     pair_gap = _finite_float(pair.get(f"gap_{endpoint}"))
B0511 0398 |     if pair_gap is not None and abs(pair_gap - gap) > settings.gap_consistency_tolerance_ev:
B0512 0399 |         assessment.add_risk(f"gap:{endpoint}:value_mismatch", uncertain=True)
B0513 0400 |     result["complete"] = False
B0514 0401 |     result_status = "incompatible"
B0515 0402 |     pair_direct = _as_bool(pair.get(f"gap_direct_{endpoint}"))
B0516 0403 |     if pair_direct is not None and direct is not None and pair_direct != direct:
B0517 0404 |         assessment.add_risk(f"gap:{endpoint}:directness_mismatch", uncertain=True)
B0518 0405 |     result["complete"] = False
B0519 0406 |     result_status = "incompatible"
B0520 0407 |
B0521 0408 |     result["gap_evidence_status"] = result_status
B0522 0409 |     if result["complete"]:
B0523 0410 |         assessment.add_positive(f"gap:endpoints_verified:{method}")
B0524 0411 |     if direct_values == [False, False]:
B0525 0412 |         assessment.add_risk("gap:both_endmembers_indirect")
B0526 0413 |     return result
B0527 0414 |
B0528 0415 |
B0529 0416 | def _evaluate_mixing(
B0530 0417 |     rows: pd.DataFrame,
B0531 0418 |     *,
B0532 0419 |     settings: RecommendationSettings,
B0533 0420 |     assessment: _Assessment,
B0534 0421 | ) -> dict[str, Any]:
B0535 0422 |     row, availability = _single_row(rows, stage="mixing", assessment=assessment)
B0536 0423 |     result = {
B0537 0424 |         "mixing_status": availability,
B0538 0425 |         "mixing_enthalpy_meV_per_atom": None,
B0539 0426 |         "mixing_signal": None,
B0540 0427 |         "complete": False,
B0541 0428 |         "target_fraction_b": None,
B0542 0429 |         "actual_fraction_b": None,
B0543 0430 |         "fraction_basis": None,
B0544 0431 |     }
B0545 0432 |     if row is None:
B0546 0433 |         return result
B0547 0434 |     assessment.add_model(row.get("model_sha256"))
B0548 0435 |     assessment.add_warning("mixing", row.get("warnings"))
B0549 0436 |     status = (_clean_text(row.get("status")) or "missing").lower()
B0550 0437 |     usable = _as_bool(row.get("usable_for_screening"))
```

```
src/ssscreen/stability/recommendation.py
```

```
B0551 0438 | value = _finite_float(row.get("mixing_enthalpy_meV_per_atom"))
B0552 0439 | result.update(
B0553 0440 |     {
B0554 0441 |         "mixing_status": status,
B0555 0442 |         "mixing_enthalpy_meV_per_atom": value,
B0556 0443 |         "mixing_signal": _clean_text(row.get("mixing_signal")),
B0557 0444 |         "target_fraction_b": _finite_float(row.get("target_fraction_b")),
B0558 0445 |         "actual_fraction_b": _finite_float(row.get("actual_fraction_b")),
B0559 0446 |         "fraction_basis": _clean_text(row.get("fraction_basis")),
B0560 0447 |     }
B0561 0448 | )
B0562 0449 | if status != "success" or usable is not True or value is None:
B0563 0450 |     assessment.add_missing(f"mixing:{status}")
B0564 0451 |     return result
B0565 0452 | result["complete"] = True
B0566 0453 | if value > settings.low_priority_mixing_enthalpy_mev_per_atom:
B0567 0454 |     assessment.add_risk("mixing:above_low_priority_threshold", hard=True)
B0568 0455 | elif value > settings.promising_max_mixing_enthalpy_mev_per_atom:
B0569 0456 |     assessment.add_risk("mixing:borderline", uncertain=True)
B0570 0457 | else:
B0571 0458 |     assessment.add_positive("mixing:within_promising_threshold")
B0572 0459 | return result
B0573 0460 |
B0574 0461 |
B0575 0462 | def _evaluate_phonon(rows: pd.DataFrame, *, assessment: _Assessment) -> dict[str, Any]:
B0576 0463 |     row, availability = _single_row(rows, stage="phonon", assessment=assessment)
B0577 0464 |     result = {
B0578 0465 |         "phonon_status": availability,
B0579 0466 |         "dynamical_status": None,
B0580 0467 |         "minimum_mesh_frequency_THz": None,
B0581 0468 |         "phonon_nac_applied": None,
B0582 0469 |         "complete": False,
B0583 0470 |     }
B0584 0471 |     if row is None:
B0585 0472 |         return result
B0586 0473 |     assessment.add_model(row.get("model_sha256"))
B0587 0474 |     assessment.add_warning("phonon", row.get("warnings"))
B0588 0475 |     status = (_clean_text(row.get("status")) or "missing").lower()
B0589 0476 |     dynamical = (_clean_text(row.get("dynamical_status")) or "missing").lower()
B0590 0477 |     nac_applied = _as_bool(row.get("nac_applied"))
B0591 0478 |     result.update(
B0592 0479 |         {
B0593 0480 |             "phonon_status": status,
B0594 0481 |             "dynamical_status": dynamical,
B0595 0482 |             "minimum_mesh_frequency_THz": _finite_float(row.get("minimum_mesh_frequency_THz")),
B0596 0483 |             "phonon_nac_applied": nac_applied,
B0597 0484 |         }
B0598 0485 |     )
B0599 0486 |     if status != "success":
B0600 0487 |         assessment.add_missing(f"phonon:{status}")
```

src/ssscreen/stability/recommendation.py

```

B0601 0488 |         return result
B0602 0489 |     if dynamical == "unstable":
B0603 0490 |         result["complete"] = True
B0604 0491 |         assessment.add_risk("phonon:unstable", hard=True)
B0605 0492 |     elif dynamical == "stable":
B0606 0493 |         result["complete"] = True
B0607 0494 |         assessment.add_positive("phonon:no_significant_imaginary_modes")
B0608 0495 |     else:
B0609 0496 |         assessment.add_risk(f"phonon:{dynamical}", uncertain=True)
B0610 0497 |     if nac_applied is False:
B0611 0498 |         assessment.add_risk("phonon:nac_not_applied")
B0612 0499 |     return result
B0613 0500 |
B0614 0501 |
B0615 0502 | def _evaluate_phase(
B0616 0503 |     rows: pd.DataFrame,
B0617 0504 |     *,
B0618 0505 |     settings: RecommendationSettings,
B0619 0506 |     assessment: _Assessment,
B0620 0507 | ) -> dict[str, Any]:
B0621 0508 |     row, availability = _single_row(rows, stage="phase", assessment=assessment)
B0622 0509 |     result = {
B0623 0510 |         "phase_status": availability,
B0624 0511 |         "hull_signal": None,
B0625 0512 |         "energy_above_hull_eV_per_atom": None,
B0626 0513 |         "competing_set_complete": None,
B0627 0514 |         "mp_source_backend": None,
B0628 0515 |         "mp_source_scope": None,
B0629 0516 |         "complete": False,
B0630 0517 |     }
B0631 0518 |     if row is None:
B0632 0519 |         return result
B0633 0520 |     assessment.add_model(row.get("model_sha256"))
B0634 0521 |     assessment.add_warning("phase", row.get("warnings"))
B0635 0522 |     status = (_clean_text(row.get("status")) or "missing").lower()
B0636 0523 |     signal = (_clean_text(row.get("hull_signal")) or "missing").lower()
B0637 0524 |     usable = _as_bool(row.get("usable_for_screening"))
B0638 0525 |     complete = _as_bool(row.get("competing_set_complete"))
B0639 0526 |     value = _finite_float(row.get("energy_above_hull_eV_per_atom"))
B0640 0527 |     backend = _clean_text(row.get("mp_source_backend"))
B0641 0528 |     scope = _clean_text(row.get("mp_source_scope"))
B0642 0529 |     result.update(
B0643 0530 |         {
B0644 0531 |             "phase_status": status,
B0645 0532 |             "hull_signal": signal,
B0646 0533 |             "energy_above_hull_eV_per_atom": value,
B0647 0534 |             "competing_set_complete": complete,
B0648 0535 |             "mp_source_backend": backend,
B0649 0536 |             "mp_source_scope": scope,
B0650 0537 |         }

```

```
src/sssreen/stability/recommendation.py
```

```
B0651 0538 | )
B0652 0539 | if status != "success" or usable is not True or complete is not True or value is None:
B0653 0540 |     code = "phase:incomplete_competing_set" if complete is False else f"phase:{status}"
B0654 0541 |     assessment.add_missing(code)
B0655 0542 |     return result
B0656 0543 | result["complete"] = True
B0657 0544 | if signal == "unstable" or value > settings.low_priority_hull_ev_per_atom:
B0658 0545 |     assessment.add_risk("phase:above_low_priority_threshold", hard=True)
B0659 0546 | elif value > settings.promising_max_hull_ev_per_atom:
B0660 0547 |     assessment.add_risk("phase:borderline", uncertain=True)
B0661 0548 | else:
B0662 0549 |     assessment.add_positive("phase:within_promising_threshold")
B0663 0550 | if backend == "offline" or scope == "offline_summary_snapshot":
B0664 0551 |     assessment.add_risk("phase:offline_snapshot_scope")
B0665 0552 | return result
B0666 0553 |
B0667 0554 |
B0668 0555 | def _evaluate_defects(
B0669 0556 |     defects: pd.DataFrame | None,
B0670 0557 |     *,
B0671 0558 |     pair_index: int,
B0672 0559 |     structure_id: str | None,
B0673 0560 |     require_defects: bool,
B0674 0561 |     assessment: _Assessment,
B0675 0562 | ) -> dict[str, Any]:
B0676 0563 |     result = {"defect_status": "not_requested", "defect_signal": None, "record_count": 0}
B0677 0564 |     if defects is None:
B0678 0565 |         assessment.add_missing("defects:not_requested", required=require_defects)
B0679 0566 |         return result
B0680 0567 |
B0681 0568 |     pair_rows = defects.loc[defects["__pair_index"] == pair_index]
B0682 0569 |     if "structure_id" in pair_rows.columns:
B0683 0570 |         normalized = pair_rows["structure_id"].map(_clean_text)
B0684 0571 |         exact = (
B0685 0572 |             pair_rows.loc[normalized == structure_id]
B0686 0573 |             if structure_id
B0687 0574 |             else pair_rows.loc[normalized.isna()]
B0688 0575 |         )
B0689 0576 |         rows = exact if not exact.empty else pair_rows.loc[normalized.isna()]
B0690 0577 |     else:
B0691 0578 |         rows = pair_rows
B0692 0579 |     if rows.empty:
B0693 0580 |         assessment.add_missing("defects:missing_source", required=require_defects)
B0694 0581 |         result["defect_status"] = "missing_source"
B0695 0582 |         return result
B0696 0583 |
B0697 0584 |     result["record_count"] = int(len(rows))
B0698 0585 |     statuses = {(_clean_text(value) or "missing").lower() for value in rows["status"]}
B0699 0586 |     signals = {(_clean_text(value) or "missing").lower() for value in rows["defect_signal"]}
B0700 0587 |     if statuses != {"success"}:
```

src/ssscreen/stability/recommendation.py

```

B0701 0588 |     assessment.add_missing("defects:failed_or_incomplete", required=require_defects)
B0702 0589 |     result["defect_status"] = "failed"
B0703 0590 |     result["defect_signal"] = ";;".join(sorted(signals))
B0704 0591 |     return result
B0705 0592 |
B0706 0593 |     allowed = {"favorable", "neutral", "low_risk", "risky", "uncertain"}
B0707 0594 |     unknown = signals - allowed
B0708 0595 |     if unknown:
B0709 0596 |         assessment.add_risk("defects:unknown_signal", uncertain=True)
B0710 0597 |         result["defect_status"] = "incompatible"
B0711 0598 |     else:
B0712 0599 |         result["defect_status"] = "complete"
B0713 0600 |     result["defect_signal"] = ";;".join(sorted(signals))
B0714 0601 |     if "risky" in signals:
B0715 0602 |         assessment.add_risk("defects:risky", hard=True)
B0716 0603 |     elif "uncertain" in signals:
B0717 0604 |         assessment.add_risk("defects:uncertain", uncertain=True)
B0718 0605 |     elif not unknown:
B0719 0606 |         assessment.add_positive("defects:no_high-risk_signal")
B0720 0607 |     return result
B0721 0608 |
B0722 0609 |
B0723 0610 | def _candidate_structure_ids(
B0724 0611 |     pair_index: int,
B0725 0612 |     mixing: pd.DataFrame,
B0726 0613 |     phonons: pd.DataFrame,
B0727 0614 |     phases: pd.DataFrame,
B0728 0615 |     defects: pd.DataFrame | None,
B0729 0616 | ) -> list[str | None]:
B0730 0617 |     values: set[str] = set()
B0731 0618 |     for frame, column in (
B0732 0619 |         (mixing, "sqs_structure_id"),
B0733 0620 |         (phonons, "structure_id"),
B0734 0621 |         (phases, "structure_id"),
B0735 0622 |     ):
B0736 0623 |         rows = frame.loc[frame["__pair_index"] == pair_index]
B0737 0624 |         values.update(text for value in rows[column] if (text := _clean_text(value)))
B0738 0625 |     if defects is not None and "structure_id" in defects.columns:
B0739 0626 |         rows = defects.loc[defects["__pair_index"] == pair_index]
B0740 0627 |         values.update(text for value in rows["structure_id"] if (text := _clean_text(value)))
B0741 0628 |     return sorted(values) if values else [None]
B0742 0629 |
B0743 0630 |
B0744 0631 | def _recommended_steps(
B0745 0632 |     classification: str,
B0746 0633 |     assessment: _Assessment,
B0747 0634 |     *,
B0748 0635 |     require_defects: bool,
B0749 0636 | ) -> list[str]:
B0750 0637 |     steps: list[str] = []

```

```
src/ssscreen/stability/recommendation.py
```

```
B0751 0638 | incomplete = [*assessment.missing]
B0752 0639 | incompatible = [
B0753 0640 |     code
B0754 0641 |     for code in assessment.uncertain
B0755 0642 |     if any(token in code for token in ("mismatch", "ambiguous", "conflict", "unknown"))
B0756 0643 | ]
B0757 0644 | if any(code.startswith("gap:") for code in [*incomplete, *incompatible]):
B0758 0645 |     _append_unique(steps, "complete_or_revalidate_high_fidelity_endpoint_gaps")
B0759 0646 | if any(code.startswith("mixing:") for code in [*incomplete, *incompatible]):
B0760 0647 |     _append_unique(steps, "run_compatible_sqs_relaxation_and_mixing_enthalpy")
B0761 0648 | if any(code.startswith("phonon:") for code in [*incomplete, *incompatible]):
B0762 0649 |     _append_unique(steps, "run_converged_phonon_screening")
B0763 0650 | if any(code.startswith("phase:") for code in [*incomplete, *incompatible]):
B0764 0651 |     _append_unique(steps, "complete_same_mlip_competing_phase_hull")
B0765 0652 | if (
B0766 0653 |     any(code.startswith("defects:") for code in [*incomplete, *incompatible])
B0767 0654 |     and require_defects
B0768 0655 | ):
B0769 0656 |     _append_unique(steps, "complete_consistent_defect_screening")
B0770 0657 | if "mlp:model_sha256_mismatch" in assessment.uncertain:
B0771 0658 |     _append_unique(steps, "recompute_stability_evidence_with_one_mlp_model")
B0772 0659 | if "phonon:unstable" in assessment.hard_negative:
B0773 0660 |     _append_unique(steps, "verify_imaginary_modes_with_larger_supercells_or_dft")
B0774 0661 | if "mixing:above_low_priority_threshold" in assessment.hard_negative:
B0775 0662 |     _append_unique(steps, "verify_high_mixing_penalty_before_further_investment")
B0776 0663 | if "mixing:borderline" in assessment.uncertain:
B0777 0664 |     _append_unique(steps, "test_additional_sqs_compositions_and_energy_convergence")
B0778 0665 | if "phase:above_low_priority_threshold" in assessment.hard_negative:
B0779 0666 |     _append_unique(steps, "verify_competing_phases_and_hull_with_consistent_dft")
B0780 0667 | if "phase:borderline" in assessment.uncertain:
B0781 0668 |     _append_unique(steps, "verify_borderline_hull_distance_with_consistent_dft")
B0782 0669 | if classification == "promising":
B0783 0670 |     _append_unique(steps, "perform_consistent_high_fidelity_dft_validation")
B0784 0671 |     _append_unique(steps, "test_multiple_sqs_compositions_and_supercell_convergence")
B0785 0672 |     if not require_defects:
B0786 0673 |         _append_unique(steps, "consider_defect_screening_for_priority_candidates")
B0787 0674 |     elif classification == "low-priority":
B0788 0675 |         _append_unique(steps, "deprioritize_unless_the_negative_signal_is_resolved")
B0789 0676 |     else:
B0790 0677 |         _append_unique(steps, "resolve_missing_or_conflicting_evidence_before_ranking")
B0791 0678 |     return steps
B0792 0679 |
B0793 0680 |
B0794 0681 | def _rationale(classification: str, assessment: _Assessment) -> str:
B0795 0682 |     if classification == "low-priority":
B0796 0683 |         return "Low priority because explicit negative signals were found: " + " ", ".join(
B0797 0684 |             assessment.hard_negative
B0798 0685 |         )
B0799 0686 |     if classification == "uncertain":
B0800 0687 |         reasons = [*assessment.uncertain]
```

```
src/ssscreen/stability/recommendation.py
```

```
B0801 0688 |     return "Uncertain because evidence is missing, borderline, or incompatible: " + ", ".join(
B0802 0689 |         reasons
B0803 0690 |     )
B0804 0691 |     return (
B0805 0692 |         "Promising for further study because the required Stage 3--10 evidence is complete "
B0806 0693 |         "and no configured low-priority signal was found."
B0807 0694 |     )
B0808 0695 |
B0809 0696 |
B0810 0697 | def _evidence_level(
B0811 0698 |     *,
B0812 0699 |     gap_complete: bool,
B0813 0700 |     mixing_complete: bool,
B0814 0701 |     phonon_complete: bool,
B0815 0702 |     phase_complete: bool,
B0816 0703 |     defect_complete: bool,
B0817 0704 | ) -> str:
B0818 0705 |     level = "L2"
B0819 0706 |     if gap_complete:
B0820 0707 |         level = "L3"
B0821 0708 |         if mixing_complete:
B0822 0709 |             level = "L4"
B0823 0710 |             if phonon_complete and phase_complete:
B0824 0711 |                 level = "L5"
B0825 0712 |                 if defect_complete:
B0826 0713 |                     level = "L6"
B0827 0714 |     return level
B0828 0715 |
B0829 0716 |
B0830 0717 | def _sort_number(value: object) -> float:
B0831 0718 |     number = _finite_float(value)
B0832 0719 |     return number if number is not None else math.inf
B0833 0720 |
B0834 0721 |
B0835 0722 | def _render_markdown_report(
B0836 0723 |     frame: pd.DataFrame,
B0837 0724 |     summary: dict[str, Any],
B0838 0725 | ) -> str:
B0839 0726 |     def display(value: object, digits: int = 4) -> str:
B0840 0727 |         number = _finite_float(value)
B0841 0728 |         if number is not None:
B0842 0729 |             return f"{number:.{digits}f}"
B0843 0730 |         text = _clean_text(value)
B0844 0731 |         return text.replace("|", "\\|") if text else "not available"
B0845 0732 |
B0846 0733 |     lines = [
B0847 0734 |         "# SS-Screen Stage 11 Recommendation Report",
B0848 0735 |         "",
B0849 0736 |         f"Generated: `{summary['generated_at']}`",
B0850 0737 |         "",
```

src/sssreen/stability/recommendation.py

```

B0851 0738 |     "> Decision-support output only. `promising` means worthy of further consistent",
B0852 0739 |     "> DFT or experimental study; it does not establish thermodynamic stability,",
B0853 0740 |     "> synthesizability, or device performance.",
B0854 0741 |     "",
B0855 0742 |     "## Summary",
B0856 0743 |     "",
B0857 0744 |     f"- Input pairs: {summary['input_pair_count']}",
B0858 0745 |     f"- Recommendation rows: {summary['recommendation_count']}",
B0859 0746 |     f"- Classification counts: `{json.dumps(summary['classification_counts'], sort_keys=True)}`",
B0860 0747 |     f"- Evidence levels: `{json.dumps(summary['evidence_level_counts'], sort_keys=True)}`",
B0861 0748 |     f"- Defects required: `{str(summary['require_defects']).lower()}`",
B0862 0749 |     "",
B0863 0750 |     "## Configured triage thresholds",
B0864 0751 |     "",
B0865 0752 |     f"- Promising mixing enthalpy maximum: {summary['settings']['promising_max_mixing_enthalpy_meV_per_atom']} meV/atom",
B0866 0753 |     f"- Low-priority mixing enthalpy threshold: {summary['settings']['low_priority_mixing_enthalpy_meV_per_atom']} meV/atom",
B0867 0754 |     f"- Promising same-MLIP hull maximum: {summary['settings']['promising_max_hull_eV_per_atom']} eV/atom",
B0868 0755 |     f"- Low-priority hull threshold: {summary['settings']['low_priority_hull_eV_per_atom']} eV/atom",
B0869 0756 |     "",
B0870 0757 |     "These are configurable triage thresholds, not universal physical constants.",
B0871 0758 |     "",
B0872 0759 |     "## Ranked candidates",
B0873 0760 |     "",
B0874 0761 |     "| Rank | Class | Evidence | Pair | Structure | x(B) | Mixing (meV/atom) | Hull (eV/atom) | Phonon |",
B0875 0762 |     "|---:|---:|---:|---:|---:|---:|---:|---:|---:|",
B0876 0763 | ]
B0877 0764 | for row in frame.to_dict(orient="records"):
B0878 0765 |     pair = f"{row['comp_a']} / {row['comp_b']}"
B0879 0766 |     fraction = row.get("actual_fraction_b")
B0880 0767 |     if _finite_float(fraction) is None:
B0881 0768 |         fraction = row.get("target_fraction_b")
B0882 0769 |     lines.append(
B0883 0770 |         "| "
B0884 0771 |         f"{row['rank']} | {row['classification']} | {row['evidence_level']} | "
B0885 0772 |         f"{display(pair)} | {display(row.get('structure_id'))} | {display(fraction, 3)} | "
B0886 0773 |         f"{display(row.get('mixing_enthalpy_meV_per_atom'), 3)} | "
B0887 0774 |         f"{display(row.get('energy_above_hull_eV_per_atom'), 5)} | "
B0888 0775 |         f"{display(row.get('dynamical_status'))} |"
B0889 0776 |     )
B0890 0777 | if frame.empty:
B0891 0778 |     lines.append("| - | - | - | No candidates | - | - | - |")
B0892 0779 |
B0893 0780 | lines.extend(["", "## Candidate audits", ""])
B0894 0781 | for row in frame.to_dict(orient="records"):
B0895 0782 |     lines.extend(
B0896 0783 |         [
B0897 0784 |             f"### {row['rank']}. {row['recommendation_id']}",
B0898 0785 |             "",
B0899 0786 |             f"- Classification: `{row['classification']}`",
B0900 0787 |             f"- Evidence level: `{row['evidence_level']}` — {row['evidence_level_label']}",

```

```
src/ssscreen/stability/recommendation.py
```

```
B0901 0788 |         f"- Pair: `{row['mp_id_a']}` ({row['comp_a']}) / `{row['mp_id_b']}` ({row['comp_b']})",
B0902 0789 |         f"- Structure: `{row['structure_id']}` or 'not available'`",
B0903 0790 |         f"- Rationale: {row['rationale']}",
B0904 0791 |         f"- Positive evidence: `{row['positive_evidence']}`",
B0905 0792 |         f"- Risks: `{row['risks']}`",
B0906 0793 |         f"- Missing data: `{row['missing_data']}`",
B0907 0794 |         f"- Source warnings: `{row['source_warnings']}`",
B0908 0795 |         f"- Recommended next steps: `{row['recommended_next_steps']}`",
B0909 0796 |         "",
B0910 0797 |     ]
B0911 0798 | )
B0912 0799 |
B0913 0800 | lines.extend(
B0914 0801 |     [
B0915 0802 |         "## Ranking policy",
B0916 0803 |         "",
B0917 0804 |         "Rows are ordered by classification, then higher evidence level, fewer risks and missing",
B0918 0805 |         "items, lower mixing enthalpy, lower same-MLIP hull distance, pair index, and structure ID.",
B0919 0806 |         "No opaque aggregate scientific score is used.",
B0920 0807 |         "",
B0921 0808 |         "## Scientific scope",
B0922 0809 |         "",
B0923 0810 |         summary["scientific_scope"],
B0924 0811 |         "",
B0925 0812 |     ]
B0926 0813 | )
B0927 0814 | return "\n".join(lines)
B0928 0815 |
B0929 0816 |
B0930 0817 | def generate_recommendations(
B0931 0818 |     *,
B0932 0819 |     pairs_path: str | Path,
B0933 0820 |     gap_results_path: str | Path,
B0934 0821 |     mixing_enthalpy_path: str | Path,
B0935 0822 |     phonons_path: str | Path,
B0936 0823 |     phase_stability_path: str | Path,
B0937 0824 |     output_path: str | Path,
B0938 0825 |     report_path: str | Path,
B0939 0826 |     defects_path: str | Path | None = None,
B0940 0827 |     summary_path: str | Path | None = None,
B0941 0828 |     gap_method: str | None = None,
B0942 0829 |     require_defects: bool = False,
B0943 0830 |     settings: RecommendationSettings | None = None,
B0944 0831 | ) -> tuple[pd.DataFrame, dict[str, Any]]:
B0945 0832 |     """Join Stage 5 and Stage 8--10 artifacts into auditable recommendations."""
B0946 0833 |     settings = settings or RecommendationSettings()
B0947 0834 |     _validate_settings(settings)
B0948 0835 |     pairs_path = Path(pairs_path)
B0949 0836 |     gaps_path = Path(gap_results_path)
B0950 0837 |     mixing_path = Path(mixing_enthalpy_path)
```

src/sssreen/stability/recommendation.py

```

B0951 0838 | phonons_path = Path(phonons_path)
B0952 0839 | phase_path = Path(phase_stability_path)
B0953 0840 | output_path = Path(output_path)
B0954 0841 | report_path = Path(report_path)
B0955 0842 | defects_file = Path(defects_path) if defects_path is not None else None
B0956 0843 |
B0957 0844 | pairs = _load_csv(pairs_path, label="pairs", required=REQUIRED_PAIR_COLUMNS).reset_index(
B0958 0845 |     drop=True
B0959 0846 | )
B0960 0847 | gaps = _load_csv(gaps_path, label="gap results", required=REQUIRED_GAP_COLUMNS)
B0961 0848 | mixing = _load_csv(mixing_path, label="mixing enthalpy", required=REQUIRED_MIXING_COLUMNS)
B0962 0849 | phonons = _load_csv(phonons_path, label="phonon summary", required=REQUIRED_PHONON_COLUMNS)
B0963 0850 | phases = _load_csv(phase_path, label="phase stability", required=REQUIRED_PHASE_COLUMNS)
B0964 0851 | defects = (
B0965 0852 |     _load_csv(defects_file, label="defect results", required=REQUIRED_DEFECT_COLUMNS)
B0966 0853 |     if defects_file is not None
B0967 0854 |     else None
B0968 0855 | )
B0969 0856 |
B0970 0857 | pair_count = len(pairs)
B0971 0858 | mixing = _prepare_pair_index(mixing, label="mixing enthalpy", pair_count=pair_count)
B0972 0859 | phonons = _prepare_pair_index(phonons, label="phonon summary", pair_count=pair_count)
B0973 0860 | phases = _prepare_pair_index(phases, label="phase stability", pair_count=pair_count)
B0974 0861 | if "structure_role" in phonons.columns:
B0975 0862 |     phonons = phonons.loc[phonons["structure_role"].fillna("sqs") == "sqs"].copy()
B0976 0863 | if defects is not None:
B0977 0864 |     defects = _prepare_pair_index(defects, label="defect results", pair_count=pair_count)
B0978 0865 |
B0979 0866 | rows: list[dict[str, Any]] = []
B0980 0867 | for pair_index, pair in pairs.iterrows():
B0981 0868 |     structure_ids = _candidate_structure_ids(pair_index, mixing, phonons, phases, defects)
B0982 0869 |     for structure_id in structure_ids:
B0983 0870 |         assessment = _Assessment()
B0984 0871 |         mixing_rows = _rows_for_pair_and_structure(
B0985 0872 |             mixing,
B0986 0873 |             pair_index=pair_index,
B0987 0874 |             structure_column="sqs_structure_id",
B0988 0875 |             structure_id=structure_id,
B0989 0876 |         )
B0990 0877 |         phonon_rows = _rows_for_pair_and_structure(
B0991 0878 |             phonons,
B0992 0879 |             pair_index=pair_index,
B0993 0880 |             structure_column="structure_id",
B0994 0881 |             structure_id=structure_id,
B0995 0882 |         )
B0996 0883 |         phase_rows = _rows_for_pair_and_structure(
B0997 0884 |             phases,
B0998 0885 |             pair_index=pair_index,
B0999 0886 |             structure_column="structure_id",
B1000 0887 |             structure_id=structure_id,

```

```
src/ssscreen/stability/recommendation.py
```

```
B1001 0888 |         )
B1002 0889 |         gap = _evaluate_gap_evidence(
B1003 0890 |             pair,
B1004 0891 |             gaps,
B1005 0892 |             method_override=gap_method,
B1006 0893 |             settings=settings,
B1007 0894 |             assessment=assessment,
B1008 0895 |         )
B1009 0896 |         mixing_result = _evaluate_mixing(
B1010 0897 |             mixing_rows,
B1011 0898 |             settings=settings,
B1012 0899 |             assessment=assessment,
B1013 0900 |         )
B1014 0901 |         phonon_result = _evaluate_phonon(
B1015 0902 |             phonon_rows,
B1016 0903 |             assessment=assessment,
B1017 0904 |         )
B1018 0905 |         phase_result = _evaluate_phase(
B1019 0906 |             phase_rows,
B1020 0907 |             settings=settings,
B1021 0908 |             assessment=assessment,
B1022 0909 |         )
B1023 0910 |         defect_result = _evaluate_defects(
B1024 0911 |             defects,
B1025 0912 |             pair_index=pair_index,
B1026 0913 |             structure_id=structure_id,
B1027 0914 |             require_defects=require_defects,
B1028 0915 |             assessment=assessment,
B1029 0916 |         )
B1030 0917 |
B1031 0918 |         model_values = sorted(assessment.model_sha256_values)
B1032 0919 |         if len(model_values) > 1:
B1033 0920 |             model_status = "incompatible"
B1034 0921 |             assessment.add_risk("mlp:model_sha256_mismatch", uncertain=True)
B1035 0922 |         elif len(model_values) == 1:
B1036 0923 |             model_status = "compatible"
B1037 0924 |         else:
B1038 0925 |             model_status = "missing_source"
B1039 0926 |
B1040 0927 |         if assessment.hard_negative:
B1041 0928 |             classification = "low-priority"
B1042 0929 |         elif assessment.uncertain:
B1043 0930 |             classification = "uncertain"
B1044 0931 |         else:
B1045 0932 |             classification = "promising"
B1046 0933 |
B1047 0934 |         defect_complete = defect_result["defect_status"] == "complete"
B1048 0935 |         evidence_level = _evidence_level(
B1049 0936 |             gap_complete=bool(gap["complete"]),
B1050 0937 |             mixing_complete=bool(mixing_result["complete"]),
```

src/sssreen/stability/recommendation.py

```

B1051 0938 |         phonon_complete=bool(phonon_result["complete"]),
B1052 0939 |         phase_complete=bool(phase_result["complete"]),
B1053 0940 |         defect_complete=defect_complete,
B1054 0941 |     )
B1055 0942 |     next_steps = _recommended_steps(
B1056 0943 |         classification, assessment, require_defects=require_defects
B1057 0944 |     )
B1058 0945 |     target_fraction = mixing_result["target_fraction_b"]
B1059 0946 |     actual_fraction = mixing_result["actual_fraction_b"]
B1060 0947 |     if len(phonon_rows) == 1:
B1061 0948 |         phonon_row = phonon_rows.iloc[0]
B1062 0949 |         if target_fraction is None:
B1063 0950 |             target_fraction = _finite_float(phonon_row.get("target_fraction_b"))
B1064 0951 |         if actual_fraction is None:
B1065 0952 |             actual_fraction = _finite_float(phonon_row.get("actual_fraction_b"))
B1066 0953 |
B1067 0954 |     row = {
B1068 0955 |         "schema_version": RECOMMENDATION_SCHEMA_VERSION,
B1069 0956 |         "recommendation_id": _recommendation_id(pair_index, structure_id, pair),
B1070 0957 |         "rank": None,
B1071 0958 |         "classification": classification,
B1072 0959 |         "evidence_level": evidence_level,
B1073 0960 |         "evidence_level_label": EVIDENCE_LEVEL_LABELS[evidence_level],
B1074 0961 |         "pair_index": pair_index,
B1075 0962 |         "source_group_index": pair.get("source_group_index"),
B1076 0963 |         "structure_id": structure_id,
B1077 0964 |         "target_fraction_b": target_fraction,
B1078 0965 |         "actual_fraction_b": actual_fraction,
B1079 0966 |         "fraction_basis": mixing_result["fraction_basis"],
B1080 0967 |         "mp_id_a": pair.get("mp_id_a"),
B1081 0968 |         "mp_id_b": pair.get("mp_id_b"),
B1082 0969 |         "comp_a": pair.get("comp_a"),
B1083 0970 |         "comp_b": pair.get("comp_b"),
B1084 0971 |         **{key: value for key, value in gap.items() if key != "complete"},
B1085 0972 |         **{
B1086 0973 |             key: value
B1087 0974 |             for key, value in mixing_result.items()
B1088 0975 |             if key
B1089 0976 |             not in {"complete", "target_fraction_b", "actual_fraction_b", "fraction_basis"}
B1090 0977 |         },
B1091 0978 |         **{key: value for key, value in phonon_result.items() if key != "complete"},
B1092 0979 |         **{key: value for key, value in phase_result.items() if key != "complete"},
B1093 0980 |         "defect_status": defect_result["defect_status"],
B1094 0981 |         "defect_signal": defect_result["defect_signal"],
B1095 0982 |         "defect_record_count": defect_result["record_count"],
B1096 0983 |         "model_sha256": model_values[0] if len(model_values) == 1 else None,
B1097 0984 |         "model_compatibility_status": model_status,
B1098 0985 |         "positive_evidence_count": len(assessment.positive),
B1099 0986 |         "risk_count": len(assessment.risks),
B1100 0987 |         "missing_data_count": len(assessment.missing),

```

src/sssreen/stability/recommendation.py

```

B1101 0988 |         "positive_evidence": _json_list(assessment.positive),
B1102 0989 |         "risks": _json_list(assessment.risks),
B1103 0990 |         "missing_data": _json_list(assessment.missing),
B1104 0991 |         "source_warnings": _json_list(assessment.source_warnings),
B1105 0992 |         "recommended_next_steps": _json_list(next_steps),
B1106 0993 |         "rationale": _rationale(classification, assessment),
B1107 0994 |     }
B1108 0995 |     rows.append(row)
B1109 0996 |
B1110 0997 |     rows.sort(
B1111 0998 |         key=lambda row: (
B1112 0999 |             CLASSIFICATION_ORDER[str(row["classification"])],
B1113 1000 |             -int(str(row["evidence_level"])[1:]),
B1114 1001 |             int(row["risk_count"]),
B1115 1002 |             int(row["missing_data_count"]),
B1116 1003 |             _sort_number(row.get("mixing_enthalpy_meV_per_atom")),
B1117 1004 |             _sort_number(row.get("energy_above_hull_eV_per_atom")),
B1118 1005 |             int(row["pair_index"]),
B1119 1006 |             str(row.get("structure_id") or ""),
B1120 1007 |         )
B1121 1008 |     )
B1122 1009 |     for rank, row in enumerate(rows, start=1):
B1123 1010 |         row["rank"] = rank
B1124 1011 |     frame = pd.DataFrame.from_records(rows, columns=OUTPUT_COLUMNS)
B1125 1012 |
B1126 1013 |     classification_counts = Counter(str(value) for value in frame["classification"])
B1127 1014 |     evidence_counts = Counter(str(value) for value in frame["evidence_level"])
B1128 1015 |     generated_at = datetime.now(UTC).isoformat()
B1129 1016 |     summary = {
B1130 1017 |         "schema_version": RECOMMENDATION_SCHEMA_VERSION,
B1131 1018 |         "generated_at": generated_at,
B1132 1019 |         "pairs_path": str(pairs_path),
B1133 1020 |         "gap_results_path": str(gaps_path),
B1134 1021 |         "mixing_enthalpy_path": str(mixing_path),
B1135 1022 |         "phonons_path": str(phonons_path),
B1136 1023 |         "phase_stability_path": str(phase_path),
B1137 1024 |         "defects_path": str(defects_file) if defects_file is not None else None,
B1138 1025 |         "output_path": str(output_path),
B1139 1026 |         "report_path": str(report_path),
B1140 1027 |         "input_pair_count": pair_count,
B1141 1028 |         "recommendation_count": len(frame),
B1142 1029 |         "classification_counts": dict(sorted(classification_counts.items())),
B1143 1030 |         "evidence_level_counts": dict(sorted(evidence_counts.items())),
B1144 1031 |         "require_defects": require_defects,
B1145 1032 |         "gap_method_override": gap_method,
B1146 1033 |         "settings": {
B1147 1034 |             "promising_max_mixing_enthalpy_meV_per_atom": settings.promising_max_mixing_enthalpy_mev_per_atom,
B1148 1035 |             "low_priority_mixing_enthalpy_meV_per_atom": settings.low_priority_mixing_enthalpy_mev_per_atom,
B1149 1036 |             "promising_max_hull_eV_per_atom": settings.promising_max_hull_ev_per_atom,
B1150 1037 |             "low_priority_hull_eV_per_atom": settings.low_priority_hull_ev_per_atom,

```

```
src/ssscreen/stability/recommendation.py -> src/ssscreen/stability/relax.py
```

```
B1151 1038 |         "gap_consistency_tolerance_eV": settings.gap_consistency_tolerance_ev,
B1152 1039 |     },
B1153 1040 |     "ranking_policy": (
B1154 1041 |         "classification, descending evidence level, risk count, missing-data count, "
B1155 1042 |         "mixing enthalpy, hull distance, pair index, structure ID; no opaque score"
B1156 1043 |     ),
B1157 1044 |     "scientific_scope": (
B1158 1045 |         "Recommendation tiers are decision support for further consistent DFT or experimental "
B1159 1046 |         "study. They do not prove finite-temperature stability, synthesizability, or device "
B1160 1047 |         "performance. Missing core evidence is uncertain, never an implicit pass."
B1161 1048 |     ),
B1162 1049 | )
B1163 1050 | _atomic_write_dataframe(frame, output_path)
B1164 1051 | _atomic_write_text(report_path, _render_markdown_report(frame, summary))
B1165 1052 | if summary_path is not None:
B1166 1053 |     _atomic_write_text(Path(summary_path), json.dumps(summary, indent=2, sort_keys=True) + "\n")
B1167 1054 | return frame, summary
B1168 0001 | """Manifest-driven, resumable MLP relaxation and result persistence."""
B1169 0002 |
B1170 0003 | from __future__ import annotations
B1171 0004 |
B1172 0005 | import hashlib
B1173 0006 | import json
B1174 0007 | import math
B1175 0008 | import os
B1176 0009 | import re
B1177 0010 | import tempfile
B1178 0011 | import time
B1179 0012 | from dataclasses import dataclass
B1180 0013 | from pathlib import Path
B1181 0014 | from typing import Any
B1182 0015 |
B1183 0016 | import numpy as np
B1184 0017 | from monty.serialization import dumpfn, loadfn
B1185 0018 | from pymatgen.core import Structure
B1186 0019 |
B1187 0020 | from ..config import MLPRelaxationSettings
B1188 0021 | from .mlp import RelaxationBackend, RelaxationOutcome
B1189 0022 |
B1190 0023 | RESULT_SCHEMA_VERSION = 1
B1191 0024 | EV_PER_ANGSTROM3_TO_GPA = 160.21766208
B1192 0025 |
B1193 0026 |
B1194 0027 | @dataclass(frozen=True)
B1195 0028 | class RelaxationTask:
B1196 0029 |     """One stable structure task derived from an SQS manifest."""
B1197 0030 |
B1198 0031 |     structure_id: str
B1199 0032 |     structure_role: str
B1200 0033 |     structure_path: Path
```

```
src/ssscreen/stability/relax.py
```

```
B1201 0034 |     metadata: dict[str, Any]

B1202 0035 |

B1203 0036 |

B1204 0037 | def _sha256_file(path: Path) -> str:

B1205 0038 |     digest = hashlib.sha256()

B1206 0039 |     with path.open("rb") as handle:

B1207 0040 |         for chunk in iter(lambda: handle.read(1024 * 1024), b""):

B1208 0041 |             digest.update(chunk)

B1209 0042 |     return digest.hexdigest()

B1210 0043 |

B1211 0044 |

B1212 0045 | def _safe_name(value: object) -> str:

B1213 0046 |     cleaned = re.sub(r"^[A-Za-z0-9_-]+", "-", str(value)).strip("-.")

B1214 0047 |     return cleaned or "structure"

B1215 0048 |

B1216 0049 |

B1217 0050 | def _atomic_write_text(path: Path, text: str) -> None:

B1218 0051 |     path.parent.mkdir(parents=True, exist_ok=True)

B1219 0052 |     descriptor, temporary = tempfile.mkstemp(prefix=f"{path.name}.", dir=path.parent)

B1220 0053 |     try:

B1221 0054 |         with os.fdopen(descriptor, "w", encoding="utf-8") as handle:

B1222 0055 |             handle.write(text)

B1223 0056 |             os.replace(temporary, path)

B1224 0057 |     except Exception:

B1225 0058 |         Path(temporary).unlink(missing_ok=True)

B1226 0059 |         raise

B1227 0060 |

B1228 0061 |

B1229 0062 | def _atomic_write_structure(structure: Structure, path: Path) -> None:

B1230 0063 |     path.parent.mkdir(parents=True, exist_ok=True)

B1231 0064 |     descriptor, temporary = tempfile.mkstemp(

B1232 0065 |         prefix=f"{path.stem}.", suffix=".json", dir=path.parent

B1233 0066 |     )

B1234 0067 |     os.close(descriptor)

B1235 0068 |     try:

B1236 0069 |         dumpfn(structure, temporary)

B1237 0070 |         os.replace(temporary, path)

B1238 0071 |     except Exception:

B1239 0072 |         Path(temporary).unlink(missing_ok=True)

B1240 0073 |         raise

B1241 0074 |

B1242 0075 |

B1243 0076 | def _read_manifest(path: Path) -> list[dict[str, Any]]:

B1244 0077 |     records: list[dict[str, Any]] = []

B1245 0078 |     with path.open(encoding="utf-8") as handle:

B1246 0079 |         for line_number, line in enumerate(handle, start=1):

B1247 0080 |             if not line.strip():

B1248 0081 |                 continue

B1249 0082 |             try:

B1250 0083 |                 record = json.loads(line)
```

src/sssreen/stability/relax.py

```
B1251 0084 |         except json.JSONDecodeError as exc:
B1252 0085 |             raise ValueError(f"invalid JSON on manifest line {line_number}") from exc
B1253 0086 |             if not isinstance(record, dict):
B1254 0087 |                 raise ValueError(f"manifest line {line_number} is not a JSON object")
B1255 0088 |             records.append(record)
B1256 0089 |     return records
B1257 0090 |
B1258 0091 |
B1259 0092 | def _resolve_input_path(raw_path: str | Path, manifest_path: Path) -> Path:
B1260 0093 |     path = Path(raw_path).expanduser()
B1261 0094 |     if path.is_absolute():
B1262 0095 |         return path
B1263 0096 |     relative_to_manifest = manifest_path.parent / path
B1264 0097 |     if relative_to_manifest.exists():
B1265 0098 |         return relative_to_manifest
B1266 0099 |     return path
B1267 0100 |
B1268 0101 |
B1269 0102 | def _metadata_from_record(record: dict[str, Any]) -> dict[str, Any]:
B1270 0103 |     keys = (
B1271 0104 |         "pair_index",
B1272 0105 |         "mp_id_a",
B1273 0106 |         "mp_id_b",
B1274 0107 |         "comp_a",
B1275 0108 |         "comp_b",
B1276 0109 |         "target_fraction_b",
B1277 0110 |         "supercell",
B1278 0111 |         "seed",
B1279 0112 |         "generator",
B1280 0113 |         "substitution",
B1281 0114 |         "replaced_sites",
B1282 0115 |         "available_substitution_sites",
B1283 0116 |         "actual_fraction_b",
B1284 0117 |         "mp_entry_id",
B1285 0118 |         "reduced_formula",
B1286 0119 |         "chemical_system",
B1287 0120 |         "parent_chemical_systems",
B1288 0121 |         "mp_energy_above_hull_eV_per_atom",
B1289 0122 |         "mp_thermo_type",
B1290 0123 |         "mp_database_version",
B1291 0124 |         "mp_retrieved_at_utc",
B1292 0125 |         "source_atom_count",
B1293 0126 |         "calculation_atom_count",
B1294 0127 |     )
B1295 0128 |     return {key: record[key] for key in keys if key in record}
B1296 0129 |
B1297 0130 |
B1298 0131 | def _verify_declared_hash(record: dict[str, Any], *, key: str, actual: str, label: str) -> None:
B1299 0132 |     declared = record.get(key)
B1300 0133 |     if declared is not None and declared != actual:
```

```
src/sssreen/stability/relax.py
```

```
B1301 0134 |     raise ValueError(
B1302 0135 |         f"{label} SHA-256 differs from manifest: declared={declared}, actual={actual}"
B1303 0136 |     )
B1304 0137 |
B1305 0138 |
B1306 0139 | def _structure_from_dataset(dataset: Any, material_id: str) -> Structure:
B1307 0140 |     if material_id not in dataset.index:
B1308 0141 |         raise ValueError(f"dataset is missing endpoint structure {material_id}")
B1309 0142 |     row = dataset.loc[material_id]
B1310 0143 |     for column in ("primitive_structure", "structure"):
B1311 0144 |         if column in row and isinstance(row[column], Structure):
B1312 0145 |             return row[column].copy()
B1313 0146 |     raise ValueError(f"dataset endpoint {material_id} has no pymatgen Structure")
B1314 0147 |
B1315 0148 |
B1316 0149 | def _endpoint_path(
B1317 0150 |     record: dict[str, Any],
B1318 0151 |     endpoint: str,
B1319 0152 |     *,
B1320 0153 |     manifest_path: Path,
B1321 0154 |     dataset: Any | None,
B1322 0155 |     output_dir: Path,
B1323 0156 | ) -> Path:
B1324 0157 |     path_key = f"endpoint_{endpoint}_structure_path"
B1325 0158 |     if path_key in record:
B1326 0159 |         return _resolve_input_path(record[path_key], manifest_path)
B1327 0160 |     if dataset is None:
B1328 0161 |         raise ValueError(
B1329 0162 |             f"manifest has no {path_key}; provide --dataset for an older Stage 6 manifest"
B1330 0163 |         )
B1331 0164 |     material_id = str(record[f"mp_id_{endpoint}"])
B1332 0165 |     path = output_dir / "inputs" / "endmembers" / f"{_safe_name(material_id)}.json"
B1333 0166 |     if not path.exists():
B1334 0167 |         _atomic_write_structure(_structure_from_dataset(dataset, material_id), path)
B1335 0168 |     return path
B1336 0169 |
B1337 0170 |
B1338 0171 | def build_relaxation_tasks(
B1339 0172 |     *,
B1340 0173 |     manifest_path: str | Path,
B1341 0174 |     output_dir: str | Path,
B1342 0175 |     include_endmembers: bool = False,
B1343 0176 |     dataset_path: str | Path | None = None,
B1344 0177 | ) -> list[RelaxationTask]:
B1345 0178 |     """Expand written SQS or competing-phase rows into de-duplicated tasks."""
B1346 0179 |     manifest_path = Path(manifest_path)
B1347 0180 |     output_dir = Path(output_dir)
B1348 0181 |     records = _read_manifest(manifest_path)
B1349 0182 |     dataset = None
B1350 0183 |     if include_endmembers and dataset_path is not None:
```

src/sssreen/stability/relax.py

```
B1351 0184 |     import pandas as pd
B1352 0185 |
B1353 0186 |     dataset = pd.read_pickle(dataset_path)
B1354 0187 |
B1355 0188 |     tasks: list[RelaxationTask] = []
B1356 0189 |     seen: set[str] = set()
B1357 0190 |     for record in records:
B1358 0191 |         if record.get("status") != "written":
B1359 0192 |             continue
B1360 0193 |         if record.get("structure_role") == "competing_phase":
B1361 0194 |             if "structure_path" not in record:
B1362 0195 |                 raise ValueError("written competing-phase manifest row has no structure_path")
B1363 0196 |             structure_path = _resolve_input_path(record["structure_path"], manifest_path)
B1364 0197 |             if not structure_path.is_file():
B1365 0198 |                 raise FileNotFoundError(
B1366 0199 |                     f"competing-phase input structure does not exist: {structure_path}"
B1367 0200 |                 )
B1368 0201 |             input_hash = _sha256_file(structure_path)
B1369 0202 |             _verify_declared_hash(
B1370 0203 |                 record,
B1371 0204 |                 key="structure_sha256",
B1372 0205 |                 actual=input_hash,
B1373 0206 |                 label="competing-phase input structure",
B1374 0207 |             )
B1375 0208 |             structure_id = record.get("structure_id")
B1376 0209 |             if not isinstance(structure_id, str) or not structure_id:
B1377 0210 |                 raise ValueError("written competing-phase row has no structure_id")
B1378 0211 |             if structure_id in seen:
B1379 0212 |                 continue
B1380 0213 |             tasks.append(
B1381 0214 |                 RelaxationTask(
B1382 0215 |                     structure_id=structure_id,
B1383 0216 |                     structure_role="competing_phase",
B1384 0217 |                     structure_path=structure_path,
B1385 0218 |                     metadata=_metadata_from_record(record),
B1386 0219 |                 )
B1387 0220 |             )
B1388 0221 |             seen.add(structure_id)
B1389 0222 |             continue
B1390 0223 |         if "structure_path" not in record:
B1391 0224 |             raise ValueError("written SQS manifest row has no structure_path")
B1392 0225 |         structure_path = _resolve_input_path(record["structure_path"], manifest_path)
B1393 0226 |         if not structure_path.is_file():
B1394 0227 |             raise FileNotFoundError(f"SQS input structure does not exist: {structure_path}")
B1395 0228 |         input_hash = _sha256_file(structure_path)
B1396 0229 |         _verify_declared_hash(
B1397 0230 |             record,
B1398 0231 |             key="structure_sha256",
B1399 0232 |             actual=input_hash,
B1400 0233 |             label="SQS input structure",
```

```
src/ssscreen/stability/relax.py
```

```
B1401 0234 |     )
B1402 0235 |     pair_index = int(record.get("pair_index", len(tasks)))
B1403 0236 |     fraction = float(record.get("target_fraction_b", 0.0))
B1404 0237 |     structure_id = f"sqs-pair-{pair_index:05d}-x-{fraction:.6f}-{input_hash[:12]}"
B1405 0238 |     if structure_id not in seen:
B1406 0239 |         tasks.append(
B1407 0240 |             RelaxationTask(
B1408 0241 |                 structure_id=structure_id,
B1409 0242 |                 structure_role="sqs",
B1410 0243 |                 structure_path=structure_path,
B1411 0244 |                 metadata=_metadata_from_record(record),
B1412 0245 |             )
B1413 0246 |         )
B1414 0247 |         seen.add(structure_id)
B1415 0248 |
B1416 0249 |     if not include_endmembers:
B1417 0250 |         continue
B1418 0251 |     for endpoint in ("a", "b"):
B1419 0252 |         endpoint_path = _endpoint_path(
B1420 0253 |             record,
B1421 0254 |             endpoint,
B1422 0255 |             manifest_path=manifest_path,
B1423 0256 |             dataset=dataset,
B1424 0257 |             output_dir=output_dir,
B1425 0258 |         )
B1426 0259 |         if not endpoint_path.is_file():
B1427 0260 |             raise FileNotFoundError(f"endpoint input structure does not exist: {endpoint_path}")
B1428 0261 |         endpoint_hash = _sha256_file(endpoint_path)
B1429 0262 |         _verify_declared_hash(
B1430 0263 |             record,
B1431 0264 |             key=f"endpoint_{endpoint}_structure_sha256",
B1432 0265 |             actual=endpoint_hash,
B1433 0266 |             label=f"endpoint {endpoint.upper()} input structure",
B1434 0267 |         )
B1435 0268 |         material_id = str(record[f"mp_id_{endpoint}"])
B1436 0269 |         endpoint_id = f"endmember-{_safe_name(material_id)}-{endpoint_hash[:12]}"
B1437 0270 |         if endpoint_id in seen:
B1438 0271 |             continue
B1439 0272 |         tasks.append(
B1440 0273 |             RelaxationTask(
B1441 0274 |                 structure_id=endpoint_id,
B1442 0275 |                 structure_role="endmember",
B1443 0276 |                 structure_path=endpoint_path,
B1444 0277 |                 metadata={
B1445 0278 |                     "material_id": material_id,
B1446 0279 |                     "composition": str(record[f"comp_{endpoint}"]),
B1447 0280 |                 },
B1448 0281 |             )
B1449 0282 |         )
B1450 0283 |         seen.add(endpoint_id)
```

```
src/sssreen/stability/relax.py
```

```
B1451 0284 |     return tasks
B1452 0285 |
B1453 0286 |
B1454 0287 | def _minimum_distance(structure: Structure) -> float | None:
B1455 0288 |     if len(structure) < 2:
B1456 0289 |         return None
B1457 0290 |     distances = np.asarray(structure.distance_matrix, dtype=float)
B1458 0291 |     distances[distances <= 1e-12] = np.inf
B1459 0292 |     minimum = float(np.min(distances))
B1460 0293 |     return minimum if math.isfinite(minimum) else None
B1461 0294 |
B1462 0295 |
B1463 0296 | def _validate_outcome(outcome: RelaxationOutcome, atom_count: int) -> tuple[np.ndarray, np.ndarray]:
B1464 0297 |     forces = np.asarray(outcome.forces_ev_per_angstrom, dtype=float)
B1465 0298 |     stress = np.asarray(outcome.stress_ev_per_angstrom3, dtype=float)
B1466 0299 |     if forces.shape != (atom_count, 3):
B1467 0300 |         raise ValueError(f"backend returned force shape {forces.shape}; expected {(atom_count, 3)}")
B1468 0301 |     if stress.shape != (3, 3):
B1469 0302 |         raise ValueError(f"backend returned stress shape {stress.shape}; expected (3, 3)")
B1470 0303 |     scalars = np.array(
B1471 0304 |         [outcome.initial_energy_total_ev, outcome.energy_total_ev, *forces.ravel(), *stress.ravel()]
B1472 0305 |     )
B1473 0306 |     if not np.all(np.isfinite(scalars)):
B1474 0307 |         raise ValueError("backend returned non-finite energy, force, or stress values")
B1475 0308 |     return forces, stress
B1476 0309 |
B1477 0310 |
B1478 0311 | def _quality_control(
B1479 0312 |     initial: Structure,
B1480 0313 |     final: Structure,
B1481 0314 |     *,
B1482 0315 |     settings: MLPRelaxationSettings,
B1483 0316 | ) -> dict[str, Any]:
B1484 0317 |     warnings: list[str] = []
B1485 0318 |     same_composition = initial.composition == final.composition and len(initial) == len(final)
B1486 0319 |     if not same_composition:
B1487 0320 |         warnings.append("atom count or composition changed during relaxation")
B1488 0321 |     volume_ratio = float(final.volume / initial.volume)
B1489 0322 |     if not settings.min_volume_ratio <= volume_ratio <= settings.max_volume_ratio:
B1490 0323 |         warnings.append(
B1491 0324 |             f"final/initial volume ratio {volume_ratio:.6g} is outside "
B1492 0325 |             f"[{settings.min_volume_ratio}, {settings.max_volume_ratio}]"
B1493 0326 |         )
B1494 0327 |     min_distance = _minimum_distance(final)
B1495 0328 |     if min_distance is not None and min_distance < settings.min_distance:
B1496 0329 |         warnings.append(
B1497 0330 |             f"minimum interatomic distance {min_distance:.6g} Angstrom is below "
B1498 0331 |             f"{settings.min_distance} Angstrom"
B1499 0332 |         )
B1500 0333 |     return {
```

```
src/ssscreen/stability/relax.py
```

```
B1501 0334 |     "passed": not warnings,
B1502 0335 |     "warnings": warnings,
B1503 0336 |     "initial_volume_angstrom3": float(initial.volume),
B1504 0337 |     "final_volume_angstrom3": float(final.volume),
B1505 0338 |     "volume_ratio": volume_ratio,
B1506 0339 |     "minimum_distance_angstrom": min_distance,
B1507 0340 | }
B1508 0341 |
B1509 0342 |
B1510 0343 | def _run_fingerprint(
B1511 0344 |     *, input_sha256: str, backend: dict[str, Any], settings: dict[str, Any]
B1512 0345 | ) -> str:
B1513 0346 |     payload = {
B1514 0347 |         "schema_version": RESULT_SCHEMA_VERSION,
B1515 0348 |         "input_sha256": input_sha256,
B1516 0349 |         "backend": backend,
B1517 0350 |         "settings": settings,
B1518 0351 |     }
B1519 0352 |     encoded = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
B1520 0353 |     return hashlib.sha256(encoded).hexdigest()
B1521 0354 |
B1522 0355 |
B1523 0356 | def _base_record(
B1524 0357 |     task: RelaxationTask,
B1525 0358 |     *,
B1526 0359 |     input_sha256: str,
B1527 0360 |     backend: dict[str, Any],
B1528 0361 |     settings: dict[str, Any],
B1529 0362 |     fingerprint: str,
B1530 0363 | ) -> dict[str, Any]:
B1531 0364 |     return {
B1532 0365 |         "schema_version": RESULT_SCHEMA_VERSION,
B1533 0366 |         "structure_id": task.structure_id,
B1534 0367 |         "structure_role": task.structure_role,
B1535 0368 |         **task.metadata,
B1536 0369 |         "input_structure": {
B1537 0370 |             "path": str(task.structure_path),
B1538 0371 |             "sha256": input_sha256,
B1539 0372 |         },
B1540 0373 |         "backend": backend,
B1541 0374 |         "settings": settings,
B1542 0375 |         "run_fingerprint": fingerprint,
B1543 0376 |     }
B1544 0377 |
B1545 0378 |
B1546 0379 | def _load_existing_record(path: Path) -> dict[str, Any] | None:
B1547 0380 |     if not path.exists():
B1548 0381 |         return None
B1549 0382 |     record = json.loads(path.read_text(encoding="utf-8"))
B1550 0383 |     if not isinstance(record, dict):
```

```
src/ssscreen/stability/relax.py
```

```
B1551 0384 |     raise ValueError(f"existing result is not a JSON object: {path}")
B1552 0385 |     return record
B1553 0386 |
B1554 0387 |
B1555 0388 | def relax_manifest(
B1556 0389 |     *,
B1557 0390 |     manifest_path: str | Path,
B1558 0391 |     output_dir: str | Path,
B1559 0392 |     results_path: str | Path,
B1560 0393 |     backend: RelaxationBackend,
B1561 0394 |     include_endmembers: bool = False,
B1562 0395 |     dataset_path: str | Path | None = None,
B1563 0396 |     fmax: float = MLPRelaxationSettings.force_tolerance,
B1564 0397 |     max_steps: int = MLPRelaxationSettings.max_steps,
B1565 0398 |     relax_cell: bool = True,
B1566 0399 |     overwrite: bool = False,
B1567 0400 |     retry_failed: bool = False,
B1568 0401 |     limit: int | None = None,
B1569 0402 |     qc_settings: MLPRelaxationSettings | None = None,
B1570 0403 | ) -> list[dict[str, Any]]:
B1571 0404 |     """Relax manifest structures and atomically persist complete per-task results."""
B1572 0405 |     if fmax <= 0:
B1573 0406 |         raise ValueError("fmax must be positive")
B1574 0407 |     if max_steps <= 0:
B1575 0408 |         raise ValueError("max_steps must be positive")
B1576 0409 |     if limit is not None and limit <= 0:
B1577 0410 |         raise ValueError("limit must be positive")
B1578 0411 |
B1579 0412 |     output_dir = Path(output_dir)
B1580 0413 |     results_path = Path(results_path)
B1581 0414 |     qc_settings = qc_settings or MLPRelaxationSettings(
B1582 0415 |         force_tolerance=fmax,
B1583 0416 |         max_steps=max_steps,
B1584 0417 |     )
B1585 0418 |     tasks = build_relaxation_tasks(
B1586 0419 |         manifest_path=manifest_path,
B1587 0420 |         output_dir=output_dir,
B1588 0421 |         include_endmembers=include_endmembers,
B1589 0422 |         dataset_path=dataset_path,
B1590 0423 |     )
B1591 0424 |     if limit is not None:
B1592 0425 |         tasks = tasks[:limit]
B1593 0426 |
B1594 0427 |     backend_metadata = backend.metadata
B1595 0428 |     run_settings = {
B1596 0429 |         "optimizer": "FIRE",
B1597 0430 |         "cell_mode": "full" if relax_cell else "positions",
B1598 0431 |         "fmax_ev_per_angstrom": float(fmax),
B1599 0432 |         "max_steps": int(max_steps),
B1600 0433 |         "quality_control": {
```

```
src/sssreen/stability/relax.py
```

```
B1601 0434 |         "min_volume_ratio": qc_settings.min_volume_ratio,
B1602 0435 |         "max_volume_ratio": qc_settings.max_volume_ratio,
B1603 0436 |         "min_distance_angstrom": qc_settings.min_distance,
B1604 0437 |     },
B1605 0438 | }
B1606 0439 | records: list[dict[str, Any]] = []
B1607 0440 | for task in tasks:
B1608 0441 |     input_sha256 = _sha256_file(task.structure_path)
B1609 0442 |     fingerprint = _run_fingerprint(
B1610 0443 |         input_sha256=input_sha256,
B1611 0444 |         backend=backend_metadata,
B1612 0445 |         settings=run_settings,
B1613 0446 |     )
B1614 0447 |     record_path = output_dir / "records" / f"{task.structure_id}.json"
B1615 0448 |     existing = _load_existing_record(record_path)
B1616 0449 |     if existing is not None and not overwrite:
B1617 0450 |         if existing.get("run_fingerprint") != fingerprint:
B1618 0451 |             raise ValueError(
B1619 0452 |                 f"existing result settings differ for {task.structure_id}; use --overwrite"
B1620 0453 |             )
B1621 0454 |         if existing.get("status") != "failed" or not retry_failed:
B1622 0455 |             records.append(existing)
B1623 0456 |             continue
B1624 0457 |
B1625 0458 |     started = time.perf_counter()
B1626 0459 |     base = _base_record(
B1627 0460 |         task,
B1628 0461 |         input_sha256=input_sha256,
B1629 0462 |         backend=backend_metadata,
B1630 0463 |         settings=run_settings,
B1631 0464 |         fingerprint=fingerprint,
B1632 0465 |     )
B1633 0466 |     try:
B1634 0467 |         initial = loadfn(task.structure_path)
B1635 0468 |         if not isinstance(initial, Structure):
B1636 0469 |             raise TypeError("input artifact is not a pymatgen Structure")
B1637 0470 |         outcome = backend.relax(
B1638 0471 |             initial.copy(),
B1639 0472 |             fmax=fmax,
B1640 0473 |             max_steps=max_steps,
B1641 0474 |             relax_cell=relax_cell,
B1642 0475 |         )
B1643 0476 |         forces, stress = _validate_outcome(outcome, len(initial))
B1644 0477 |         qc = _quality_control(initial, outcome.structure, settings=qc_settings)
B1645 0478 |         output_path = output_dir / "structures" / f"{task.structure_id}.json"
B1646 0479 |         _atomic_write_structure(outcome.structure, output_path)
B1647 0480 |         force_norms = np.linalg.norm(forces, axis=1)
B1648 0481 |         record = {
B1649 0482 |             **base,
B1650 0483 |             "output_structure": {
```

```
src/ssscreen/stability/relax.py -> src/ssscreen/stability/sqs.py
```

```
B1651 0484 |         "path": str(output_path),
B1652 0485 |         "sha256": _sha256_file(output_path),
B1653 0486 |     },
B1654 0487 |     "initial_energy_total_eV": float(outcome.initial_energy_total_ev),
B1655 0488 |     "energy_total_eV": float(outcome.energy_total_ev),
B1656 0489 |     "energy_per_atom_eV": float(outcome.energy_total_ev / len(initial)),
B1657 0490 |     "forces_eV_per_angstrom": forces.tolist(),
B1658 0491 |     "max_force_eV_per_angstrom": float(np.max(force_norms)),
B1659 0492 |     "rms_force_eV_per_angstrom": float(np.sqrt(np.mean(force_norms**2))),
B1660 0493 |     "stress_eV_per_angstrom3": stress.tolist(),
B1661 0494 |     "stress_GPa": (stress * EV_PER_ANGSTROM3_TO_GPA).tolist(),
B1662 0495 |     "stress_convention": "ASE Cartesian 3x3; tensile stress is positive",
B1663 0496 |     "steps": int(outcome.steps),
B1664 0497 |     "status": "success" if outcome.converged else "not_converged",
B1665 0498 |     "converged": bool(outcome.converged),
B1666 0499 |     "quality_control": qc,
B1667 0500 |     "usable_for_thermodynamics": bool(outcome.converged and qc["passed"]),
B1668 0501 |     "runtime_seconds": float(time.perf_counter() - started),
B1669 0502 |     "error": None,
B1670 0503 | }
B1671 0504 | except Exception as exc:
B1672 0505 |     record = {
B1673 0506 |         **base,
B1674 0507 |         "status": "failed",
B1675 0508 |         "converged": False,
B1676 0509 |         "usable_for_thermodynamics": False,
B1677 0510 |         "runtime_seconds": float(time.perf_counter() - started),
B1678 0511 |         "error": f"{type(exc).__name__}: {exc}"[:2000],
B1679 0512 |     }
B1680 0513 |     _atomic_write_text(record_path, json.dumps(record, indent=2, sort_keys=True) + "\n")
B1681 0514 |     records.append(record)
B1682 0515 |
B1683 0516 |     lines = "".join(json.dumps(record, sort_keys=True) + "\n" for record in records)
B1684 0517 |     _atomic_write_text(results_path, lines)
B1685 0518 |     return records
B1686 0001 | """Generate alloy/SQS input artifacts for preliminary stability screening."""
B1687 0002 |
B1688 0003 | from __future__ import annotations
B1689 0004 |
B1690 0005 | import hashlib
B1691 0006 | import json
B1692 0007 | import random
B1693 0008 | import re
B1694 0009 | from pathlib import Path
B1695 0010 |
B1696 0011 | import pandas as pd
B1697 0012 | from monty.serialization import dumpfn
B1698 0013 | from pymatgen.core import Composition, Structure
B1699 0014 |
B1700 0015 |
```

src/ssscreen/stability/sqs.py

```
B1701 0016 | def infer_single_site_substitution(comp_a: str, comp_b: str) -> tuple[str, str]:
B1702 0017 |     """Infer the one-element substitution needed to turn ``comp_a`` into ``comp_b``."""
B1703 0018 |     a = Composition(comp_a).fractional_composition
B1704 0019 |     b = Composition(comp_b).fractional_composition
B1705 0020 |     elements = sorted({str(el) for el in a.elements} | {str(el) for el in b.elements})
B1706 0021 |     removed: list[str] = []
B1707 0022 |     added: list[str] = []
B1708 0023 |     for element in elements:
B1709 0024 |         delta = b.get_atomic_fraction(element) - a.get_atomic_fraction(element)
B1710 0025 |         if delta < -1e-8:
B1711 0026 |             removed.append(element)
B1712 0027 |         elif delta > 1e-8:
B1713 0028 |             added.append(element)
B1714 0029 |     if len(removed) != 1 or len(added) != 1:
B1715 0030 |         raise ValueError(f"expected a single-site substitution, got {comp_a!r} -> {comp_b!r}")
B1716 0031 |     return removed[0], added[0]
B1717 0032 |
B1718 0033 |
B1719 0034 | def _load_pairs(path: str | Path) -> pd.DataFrame:
B1720 0035 |     pairs = pd.read_csv(path)
B1721 0036 |     unnamed = [col for col in pairs.columns if str(col).startswith("Unnamed:")]
B1722 0037 |     if unnamed:
B1723 0038 |         pairs = pairs.drop(columns=unnamed)
B1724 0039 |     required = {"comp_a", "comp_b", "mp_id_a", "mp_id_b"}
B1725 0040 |     missing = sorted(required - set(pairs.columns))
B1726 0041 |     if missing:
B1727 0042 |         raise ValueError(f"missing required pair columns: {' '.join(missing)}")
B1728 0043 |     return pairs
B1729 0044 |
B1730 0045 |
B1731 0046 | def _structure_for_row(row: pd.Series, structure_col: str) -> Structure:
B1732 0047 |     if "primitive_structure" in row and row["primitive_structure"] is not None:
B1733 0048 |         return row["primitive_structure"].copy()
B1734 0049 |     return row[structure_col].copy()
B1735 0050 |
B1736 0051 |
B1737 0052 | def _safe_filename(value: object) -> str:
B1738 0053 |     cleaned = re.sub(r"^[A-Za-z0-9_.-]+", "-", str(value)).strip("-.")
B1739 0054 |     return cleaned or "structure"
B1740 0055 |
B1741 0056 |
B1742 0057 | def _sha256_file(path: Path) -> str:
B1743 0058 |     digest = hashlib.sha256()
B1744 0059 |     with path.open("rb") as handle:
B1745 0060 |         for chunk in iter(lambda: handle.read(1024 * 1024), b""):
B1746 0061 |             digest.update(chunk)
B1747 0062 |     return digest.hexdigest()
B1748 0063 |
B1749 0064 |
B1750 0065 | def _write_endpoint_structure(structure: Structure, material_id: str, output_dir: Path) -> Path:
```

```
src/ssscren/stability/sqs.py
```

```
B1751 0066 | path = output_dir / "endmembers" / f"[_safe_filename(material_id)].json"
B1752 0067 | path.parent.mkdir(parents=True, exist_ok=True)
B1753 0068 | dumpfn(structure, str(path))
B1754 0069 | return path
B1755 0070 |
B1756 0071 |
B1757 0072 | def _replace_fraction(
B1758 0073 |     structure: Structure,
B1759 0074 |     *,
B1760 0075 |     from_element: str,
B1761 0076 |     to_element: str,
B1762 0077 |     target_fraction: float,
B1763 0078 |     seed: int,
B1764 0079 | ) -> tuple[Structure, int, int]:
B1765 0080 |     x_site_indices = [
B1766 0081 |         index
B1767 0082 |         for index, site in enumerate(structure)
B1768 0083 |         if getattr(site.specie, "symbol", str(site.specie)) == from_element
B1769 0084 |     ]
B1770 0085 |     if not x_site_indices:
B1771 0086 |         raise ValueError(f"prototype structure has no {from_element} sites")
B1772 0087 |     n_replace = round(target_fraction * len(x_site_indices))
B1773 0088 |     rng = random.Random(seed)
B1774 0089 |     selected = list(x_site_indices)
B1775 0090 |     rng.shuffle(selected)
B1776 0091 |     for index in selected[:n_replace]:
B1777 0092 |         structure.replace(index, to_element)
B1778 0093 |     return structure, n_replace, len(x_site_indices)
B1779 0094 |
B1780 0095 |
B1781 0096 | def _generate_icet_structure(
B1782 0097 |     structure: Structure,
B1783 0098 |     *,
B1784 0099 |     from_element: str,
B1785 0100 |     to_element: str,
B1786 0101 |     target_fraction: float,
B1787 0102 |     supercell: tuple[int, int, int],
B1788 0103 |     seed: int,
B1789 0104 |     cutoffs: list[float],
B1790 0105 |     sqs_steps: int | None,
B1791 0106 | ) -> tuple[Structure, int, int]:
B1792 0107 |     try:
B1793 0108 |         from icet import ClusterSpace
B1794 0109 |         from icet.tools.structure_generation import generate_sqs_from_supercells
B1795 0110 |         from pymatgen.io.ase import AseAtomsAdaptor
B1796 0111 |     except ImportError as exc:
B1797 0112 |         raise ImportError(
B1798 0113 |             "icet SQS generation requires the optional `icet` and `ase` dependencies"
B1799 0114 |         ) from exc
B1800 0115 |
```

```
src/ssscreen/stability/sqs.py
```

```
B1801 0116 | atoms = AseAtomsAdaptor.get_atoms(structure)
B1802 0117 | atoms.pbc = True
B1803 0118 | chemical_symbols = []
B1804 0119 | active_sites = 0
B1805 0120 | for site in structure:
B1806 0121 |     symbol = getattr(site.specie, "symbol", str(site.specie))
B1807 0122 |     if symbol == from_element:
B1808 0123 |         chemical_symbols.append([from_element, to_element])
B1809 0124 |         active_sites += 1
B1810 0125 |     else:
B1811 0126 |         chemical_symbols.append([symbol])
B1812 0127 | if active_sites == 0:
B1813 0128 |     raise ValueError(f"prototype structure has no {from_element} sites")
B1814 0129 |
B1815 0130 | cluster_space = ClusterSpace(
B1816 0131 |     atoms,
B1817 0132 |     cutoffs=cutoffs,
B1818 0133 |     chemical_symbols=chemical_symbols,
B1819 0134 | )
B1820 0135 | target_concentrations = {}
B1821 0136 | for sublattice in cluster_space.get_sublattices(atoms):
B1822 0137 |     if set(sublattice.chemical_symbols) == {from_element, to_element}:
B1823 0138 |         target_concentrations[sublattice.symbol] = {
B1824 0139 |             from_element: 1.0 - float(target_fraction),
B1825 0140 |             to_element: float(target_fraction),
B1826 0141 |         }
B1827 0142 | if not target_concentrations:
B1828 0143 |     raise ValueError(f"could not find active icet sublattice for {from_element}->{to_element}")
B1829 0144 |
B1830 0145 | supercell_atoms = atoms.repeat(supercell)
B1831 0146 | sqs_atoms = generate_sqs_from_supercells(
B1832 0147 |     cluster_space=cluster_space,
B1833 0148 |     supercells=[supercell_atoms],
B1834 0149 |     target_concentrations=target_concentrations,
B1835 0150 |     n_steps=sqs_steps,
B1836 0151 |     random_seed=seed,
B1837 0152 | )
B1838 0153 | generated = AseAtomsAdaptor.get_structure(sqs_atoms)
B1839 0154 | generated_to_sites = sum(
B1840 0155 |     1 for site in generated if getattr(site.specie, "symbol", str(site.specie)) == to_element
B1841 0156 | )
B1842 0157 | repeat_count = supercell[0] * supercell[1] * supercell[2]
B1843 0158 | baseline_to_sites = (
B1844 0159 |     sum(
B1845 0160 |         1
B1846 0161 |         for site in structure
B1847 0162 |         if getattr(site.specie, "symbol", str(site.specie)) == to_element
B1848 0163 |     )
B1849 0164 |     * repeat_count
B1850 0165 | )
```

```
src/ssscren/stability/sqs.py
```

```
B1851 0166 |     replaced = generated_to_sites - baseline_to_sites
B1852 0167 |     available = active_sites * repeat_count
B1853 0168 |     if replaced < 0 or replaced > available:
B1854 0169 |         raise ValueError(f"icet generated an invalid substitution count: {replaced}/{available}")
B1855 0170 |     return generated, replaced, available
B1856 0171 |
B1857 0172 |
B1858 0173 | def _record_base(
B1859 0174 |     pair_index: int,
B1860 0175 |     row: pd.Series,
B1861 0176 |     target_fraction: float,
B1862 0177 |     supercell: tuple[int, int, int],
B1863 0178 |     seed: int,
B1864 0179 |     generator: str,
B1865 0180 | ) -> dict:
B1866 0181 |     record = {
B1867 0182 |         "pair_index": int(pair_index),
B1868 0183 |         "mp_id_a": str(row["mp_id_a"]),
B1869 0184 |         "mp_id_b": str(row["mp_id_b"]),
B1870 0185 |         "comp_a": str(row["comp_a"]),
B1871 0186 |         "comp_b": str(row["comp_b"]),
B1872 0187 |         "target_fraction_b": float(target_fraction),
B1873 0188 |         "supercell": list(supercell),
B1874 0189 |         "seed": int(seed),
B1875 0190 |         "generator": generator,
B1876 0191 |     }
B1877 0192 |     for key in ("source_group_index", "gap_method", "gap_a", "gap_b"):
B1878 0193 |         if key in row and pd.notna(row[key]):
B1879 0194 |             value = row[key]
B1880 0195 |             record[key] = value.item() if hasattr(value, "item") else value
B1881 0196 |     return record
B1882 0197 |
B1883 0198 |
B1884 0199 | def _write_manifest(records: list[dict], path: str | Path) -> None:
B1885 0200 |     Path(path).parent.mkdir(parents=True, exist_ok=True)
B1886 0201 |     with open(path, "w") as handle:
B1887 0202 |         for record in records:
B1888 0203 |             handle.write(json.dumps(record, sort_keys=True) + "\n")
B1889 0204 |
B1890 0205 |
B1891 0206 | def _validate_target_fractions(target_fractions: tuple[float, ...] | list[float]) -> None:
B1892 0207 |     invalid = [fraction for fraction in target_fractions if fraction < 0.0 or fraction > 1.0]
B1893 0208 |     if invalid:
B1894 0209 |         raise ValueError("target fractions must be between 0 and 1")
B1895 0210 |
B1896 0211 |
B1897 0212 | def generate_sqs_inputs(
B1898 0213 |     *,
B1899 0214 |     pairs_path: str | Path,
B1900 0215 |     dataset_path: str | Path,
```

```
src/ssscren/stability/sqs.py
```

```
B1901 0216 | output_dir: str | Path,
B1902 0217 | manifest_path: str | Path,
B1903 0218 | target_fractions: tuple[float, ...] | list[float],
B1904 0219 | supercell: tuple[int, int, int] = (2, 2, 2),
B1905 0220 | seed: int = 0,
B1906 0221 | structure_col: str = "structure",
B1907 0222 | backend: str = "random",
B1908 0223 | cutoffs: tuple[float, ...] | list[float] = (4.0,),
B1909 0224 | sqs_steps: int | None = None,
B1910 0225 | ) -> list[dict]:
B1911 0226 | """Generate representative alloy structures and a JSONL manifest.
B1912 0227 |
B1913 0228 | This first Stage 6 generator is intentionally dependency-light. It creates
B1914 0229 | deterministic random-substitution structures and records the limitation in
B1915 0230 | the manifest; optimized SQS backends can consume the same artifact schema
B1916 0231 | later.
B1917 0232 | """
B1918 0233 | if backend not in {"random", "icet"}:
B1919 0234 |     raise ValueError(f"unsupported SQS backend: {backend}")
B1920 0235 | _validate_target_fractions(target_fractions)
B1921 0236 | generator = "icet-generate_sqs_from_supercells" if backend == "icet" else "random-substitution"
B1922 0237 | pairs = _load_pairs(pairs_path)
B1923 0238 | dataset = pd.read_pickle(dataset_path)
B1924 0239 | output_dir = Path(output_dir)
B1925 0240 | output_dir.mkdir(parents=True, exist_ok=True)
B1926 0241 | records: list[dict] = []
B1927 0242 |
B1928 0243 | for pair_index, row in pairs.iterrows():
B1929 0244 |     for fraction_index, target_fraction in enumerate(target_fractions):
B1930 0245 |         stable_seed = int(seed) + int(pair_index) * 1000003 + fraction_index
B1931 0246 |         record = _record_base(
B1932 0247 |             int(pair_index),
B1933 0248 |             row,
B1934 0249 |             float(target_fraction),
B1935 0250 |             supercell,
B1936 0251 |             stable_seed,
B1937 0252 |             generator,
B1938 0253 |         )
B1939 0254 |         try:
B1940 0255 |             from_element, to_element = infer_single_site_substitution(
B1941 0256 |                 str(row["comp_a"]), str(row["comp_b"])
B1942 0257 |             )
B1943 0258 |             record["substitution"] = {"from": from_element, "to": to_element}
B1944 0259 |             mp_id_a = str(row["mp_id_a"])
B1945 0260 |             mp_id_b = str(row["mp_id_b"])
B1946 0261 |             if mp_id_a not in dataset.index:
B1947 0262 |                 raise ValueError(f"dataset is missing endpoint structure {mp_id_a}")
B1948 0263 |             if mp_id_b not in dataset.index:
B1949 0264 |                 raise ValueError(f"dataset is missing endpoint structure {mp_id_b}")
B1950 0265 |             endpoint_a = _structure_for_row(dataset.loc[mp_id_a], structure_col)
```

src/ssscreen/stability/sqs.py

```
B1951 0266 |         endpoint_b = _structure_for_row(dataset.loc[mp_id_b], structure_col)
B1952 0267 |         endpoint_a_path = _write_endpoint_structure(endpoint_a, mp_id_a, output_dir)
B1953 0268 |         endpoint_b_path = _write_endpoint_structure(endpoint_b, mp_id_b, output_dir)
B1954 0269 |         record.update(
B1955 0270 |             {
B1956 0271 |                 "endpoint_a_structure_path": str(endpoint_a_path),
B1957 0272 |                 "endpoint_a_structure_sha256": _sha256_file(endpoint_a_path),
B1958 0273 |                 "endpoint_b_structure_path": str(endpoint_b_path),
B1959 0274 |                 "endpoint_b_structure_sha256": _sha256_file(endpoint_b_path),
B1960 0275 |             }
B1961 0276 |         )
B1962 0277 |         structure = endpoint_a.copy()
B1963 0278 |         if backend == "icet":
B1964 0279 |             structure, replaced, available = _generate_icet_structure(
B1965 0280 |                 structure,
B1966 0281 |                 from_element=from_element,
B1967 0282 |                 to_element=to_element,
B1968 0283 |                 target_fraction=float(target_fraction),
B1969 0284 |                 supercell=supercell,
B1970 0285 |                 seed=stable_seed,
B1971 0286 |                 cutoffs=list(cutoffs),
B1972 0287 |                 sqs_steps=sqs_steps,
B1973 0288 |             )
B1974 0289 |             limitation = (
B1975 0290 |                 "icet SQS generated by simulated annealing; quality metrics are "
B1976 0291 |                 "not yet exported"
B1977 0292 |             )
B1978 0293 |         else:
B1979 0294 |             structure.make_supercell(supercell)
B1980 0295 |             structure, replaced, available = _replace_fraction(
B1981 0296 |                 structure,
B1982 0297 |                 from_element=from_element,
B1983 0298 |                 to_element=to_element,
B1984 0299 |                 target_fraction=float(target_fraction),
B1985 0300 |                 seed=stable_seed,
B1986 0301 |             )
B1987 0302 |             limitation = "random substitution only; not an optimized SQS correlation match"
B1988 0303 |             filename = (
B1989 0304 |                 f"pair_{int(pair_index):05d}_"
B1990 0305 |                 f"x{float(target_fraction):.3f}_{mp_id_a}_{row['mp_id_b']}.json"
B1991 0306 |             )
B1992 0307 |             structure_path = output_dir / filename
B1993 0308 |             dumpfn(structure, str(structure_path))
B1994 0309 |             record.update(
B1995 0310 |                 {
B1996 0311 |                     "status": "written",
B1997 0312 |                     "structure_path": str(structure_path),
B1998 0313 |                     "structure_sha256": _sha256_file(structure_path),
B1999 0314 |                     "replaced_sites": int(replaced),
B2000 0315 |                     "available_substitution_sites": int(available),
```

```
src/ssscreen/stability/sqs.py -> src/ssscreen/stability/thermodynamics.py

B2001 0316 |         "actual_fraction_b": float(replaced / available),
B2002 0317 |         "limitation": limitation,
B2003 0318 |     }
B2004 0319 | )
B2005 0320 | except Exception as exc:
B2006 0321 |     record.update(
B2007 0322 |     {
B2008 0323 |         "status": "skipped",
B2009 0324 |         "structure_path": "",
B2010 0325 |         "limitation": str(exc),
B2011 0326 |     }
B2012 0327 | )
B2013 0328 |     records.append(record)
B2014 0329 |
B2015 0330 |     _write_manifest(records, manifest_path)
B2016 0331 |     return records
B2017 0001 | """Mixing-enthalpy estimates from compatible Stage 7 relaxation records."""
B2018 0002 |
B2019 0003 | from __future__ import annotations
B2020 0004 |
B2021 0005 | import hashlib
B2022 0006 | import json
B2023 0007 | import math
B2024 0008 | import os
B2025 0009 | import tempfile
B2026 0010 | from collections import Counter
B2027 0011 | from pathlib import Path
B2028 0012 | from typing import Any
B2029 0013 |
B2030 0014 | import pandas as pd
B2031 0015 |
B2032 0016 | MIXING_SCHEMA_VERSION = 1
B2033 0017 | ENERGY_BACKEND_FIELDS = ("name", "version", "model_sha256", "dtype")
B2034 0018 | ENERGY_SETTING_FIELDS = (
B2035 0019 |     "optimizer",
B2036 0020 |     "cell_mode",
B2037 0021 |     "fmax_ev_per_angstrom",
B2038 0022 |     "max_steps",
B2039 0023 |     "quality_control",
B2040 0024 | )
B2041 0025 | QUALITY_CONTROL_FIELDS = (
B2042 0026 |     "min_volume_ratio",
B2043 0027 |     "max_volume_ratio",
B2044 0028 |     "min_distance_angstrom",
B2045 0029 | )
B2046 0030 | REQUIRED_PAIR_COLUMNS = ("comp_a", "comp_b", "mp_id_a", "mp_id_b")
B2047 0031 | OUTPUT_COLUMNS = [
B2048 0032 |     "schema_version",
B2049 0033 |     "pair_index",
B2050 0034 |     "source_group_index",
```

src/ssscreen/stability/thermodynamics.py

```
B2051 0035 |     "mp_id_a",
B2052 0036 |     "mp_id_b",
B2053 0037 |     "comp_a",
B2054 0038 |     "comp_b",
B2055 0039 |     "target_fraction_b",
B2056 0040 |     "actual_fraction_b",
B2057 0041 |     "fraction_basis",
B2058 0042 |     "sqs_structure_id",
B2059 0043 |     "endmember_a_structure_id",
B2060 0044 |     "endmember_b_structure_id",
B2061 0045 |     "energy_sqs_per_atom_eV",
B2062 0046 |     "energy_endmember_a_per_atom_eV",
B2063 0047 |     "energy_endmember_b_per_atom_eV",
B2064 0048 |     "reference_energy_per_atom_eV",
B2065 0049 |     "mixing_enthalpy_eV_per_atom",
B2066 0050 |     "mixing_enthalpy_meV_per_atom",
B2067 0051 |     "mixing_signal",
B2068 0052 |     "backend_name",
B2069 0053 |     "backend_version",
B2070 0054 |     "model_name",
B2071 0055 |     "model_sha256",
B2072 0056 |     "dtype",
B2073 0057 |     "sqs_device",
B2074 0058 |     "endmember_a_device",
B2075 0059 |     "endmember_b_device",
B2076 0060 |     "energy_reference_sha256",
B2077 0061 |     "status",
B2078 0062 |     "usable_for_screening",
B2079 0063 |     "warnings",
B2080 0064 |     "error",
B2081 0065 | ]
B2082 0066 |
B2083 0067 |
B2084 0068 | def _atomic_write_text(path: Path, text: str) -> None:
B2085 0069 |     path.parent.mkdir(parents=True, exist_ok=True)
B2086 0070 |     descriptor, temporary = tempfile.mkstemp(prefix=f"{path.name}.", dir=path.parent)
B2087 0071 |     try:
B2088 0072 |         with os.fdopen(descriptor, "w", encoding="utf-8") as handle:
B2089 0073 |             handle.write(text)
B2090 0074 |             os.replace(temporary, path)
B2091 0075 |     except Exception:
B2092 0076 |         Path(temporary).unlink(missing_ok=True)
B2093 0077 |         raise
B2094 0078 |
B2095 0079 |
B2096 0080 | def _atomic_write_dataframe(frame: pd.DataFrame, path: Path) -> None:
B2097 0081 |     path.parent.mkdir(parents=True, exist_ok=True)
B2098 0082 |     descriptor, temporary = tempfile.mkstemp(
B2099 0083 |         prefix=f"{path.stem}.", suffix=path.suffix or ".csv", dir=path.parent
B2100 0084 |     )
```

```
src/sssreen/stability/thermodynamics.py
```

```
B2101 0085 | os.close(descriptor)
B2102 0086 | try:
B2103 0087 |     frame.to_csv(temporary, index=False)
B2104 0088 |     os.replace(temporary, path)
B2105 0089 | except Exception:
B2106 0090 |     Path(temporary).unlink(missing_ok=True)
B2107 0091 |     raise
B2108 0092 |
B2109 0093 |
B2110 0094 | def _load_pairs(path: Path) -> pd.DataFrame:
B2111 0095 |     pairs = pd.read_csv(path)
B2112 0096 |     unnamed = [column for column in pairs.columns if str(column).startswith("Unnamed:")]
B2113 0097 |     if unnamed:
B2114 0098 |         pairs = pairs.drop(columns=unnamed)
B2115 0099 |     missing = [column for column in REQUIRED_PAIR_COLUMNS if column not in pairs.columns]
B2116 0100 |     if missing:
B2117 0101 |         raise ValueError(f"missing required pair columns: {'', '.join(missing)}")
B2118 0102 |     return pairs.reset_index(drop=True)
B2119 0103 |
B2120 0104 |
B2121 0105 | def _read_jsonl(path: Path) -> list[dict[str, Any]]:
B2122 0106 |     records: list[dict[str, Any]] = []
B2123 0107 |     seen: set[str] = set()
B2124 0108 |     with path.open(encoding="utf-8") as handle:
B2125 0109 |         for line_number, line in enumerate(handle, start=1):
B2126 0110 |             if not line.strip():
B2127 0111 |                 continue
B2128 0112 |             try:
B2129 0113 |                 record = json.loads(line)
B2130 0114 |             except json.JSONDecodeError as exc:
B2131 0115 |                 raise ValueError(f"invalid JSON on relaxation-results line {line_number}") from exc
B2132 0116 |             if not isinstance(record, dict):
B2133 0117 |                 raise ValueError(f"relaxation-results line {line_number} is not a JSON object")
B2134 0118 |             structure_id = record.get("structure_id")
B2135 0119 |             if not isinstance(structure_id, str) or not structure_id:
B2136 0120 |                 raise ValueError(f"relaxation-results line {line_number} has no structure_id")
B2137 0121 |             if structure_id in seen:
B2138 0122 |                 raise ValueError(f"duplicate relaxation structure_id: {structure_id}")
B2139 0123 |             seen.add(structure_id)
B2140 0124 |             records.append(record)
B2141 0125 |     return records
B2142 0126 |
B2143 0127 |
B2144 0128 | def _energy_signature(record: dict[str, Any]) -> tuple[str | None, dict[str, Any] | None]:
B2145 0129 |     backend = record.get("backend")
B2146 0130 |     settings = record.get("settings")
B2147 0131 |     if not isinstance(backend, dict) or not isinstance(settings, dict):
B2148 0132 |         return None, None
B2149 0133 |     if any(backend.get(field) in (None, "") for field in ENERGY_BACKEND_FIELDS):
B2150 0134 |         return None, None
```

```
src/ssscreen/stability/thermodynamics.py
```

```
B2151 0135 |     if any(field not in settings for field in ENERGY_SETTING_FIELDS):
B2152 0136 |         return None, None
B2153 0137 |     quality_control = settings.get("quality_control")
B2154 0138 |     if not isinstance(quality_control, dict) or any(
B2155 0139 |         field not in quality_control for field in QUALITY_CONTROL_FIELDS
B2156 0140 |     ):
B2157 0141 |         return None, None
B2158 0142 |     payload = {
B2159 0143 |         "backend": {field: backend[field] for field in ENERGY_BACKEND_FIELDS},
B2160 0144 |         "settings": settings,
B2161 0145 |     }
B2162 0146 |     encoded = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
B2163 0147 |     return hashlib.sha256(encoded).hexdigest(), payload
B2164 0148 |
B2165 0149 |
B2166 0150 | def _finite_energy(record: dict[str, Any]) -> float | None:
B2167 0151 |     try:
B2168 0152 |         energy = float(record["energy_per_atom_eV"])
B2169 0153 |     except (KeyError, TypeError, ValueError):
B2170 0154 |         return None
B2171 0155 |     return energy if math.isfinite(energy) else None
B2172 0156 |
B2173 0157 |
B2174 0158 | def _is_usable(record: dict[str, Any]) -> bool:
B2175 0159 |     return (
B2176 0160 |         record.get("status") == "success"
B2177 0161 |         and record.get("usable_for_thermodynamics") is True
B2178 0162 |         and _finite_energy(record) is not None
B2179 0163 |     )
B2180 0164 |
B2181 0165 |
B2182 0166 | def _select_endmember(
B2183 0167 |     material_id: str,
B2184 0168 |     *,
B2185 0169 |     endmembers: dict[str, list[dict[str, Any]]],
B2186 0170 |     energy_signature: str,
B2187 0171 | ) -> tuple[dict[str, Any] | None, str | None, str | None]:
B2188 0172 |     candidates = endmembers.get(material_id, [])
B2189 0173 |     if not candidates:
B2190 0174 |         return None, "missing_endmember", f"no relaxed endmember record for {material_id}"
B2191 0175 |     usable = [record for record in candidates if _is_usable(record)]
B2192 0176 |     if not usable:
B2193 0177 |         return None, "unusable_endmember", f"no usable endmember energy for {material_id}"
B2194 0178 |     compatible = [record for record in usable if _energy_signature(record)[0] == energy_signature]
B2195 0179 |     if not compatible:
B2196 0180 |         return (
B2197 0181 |             None,
B2198 0182 |             "incompatible_provenance",
B2199 0183 |             f"endmember {material_id} was not relaxed with the SQS energy reference",
B2200 0184 |         )
```

```
src/ssscreen/stability/thermodynamics.py
```

```
B2201 0185 |     if len(compatible) > 1:
B2202 0186 |         return (
B2203 0187 |             None,
B2204 0188 |             "ambiguous_endmember",
B2205 0189 |             f"multiple compatible endmember records exist for {material_id}",
B2206 0190 |         )
B2207 0191 |     return compatible[0], None, None
B2208 0192 |
B2209 0193 |
B2210 0194 | def _fraction_b(record: dict[str, Any]) -> tuple[float | None, str | None, list[str], str | None]:
B2211 0195 |     warnings: list[str] = []
B2212 0196 |     target_raw = record.get("target_fraction_b")
B2213 0197 |     try:
B2214 0198 |         target = float(target_raw)
B2215 0199 |     except (TypeError, ValueError):
B2216 0200 |         return None, None, warnings, "SQS record has no valid target_fraction_b"
B2217 0201 |
B2218 0202 |     basis = "actual"
B2219 0203 |     if record.get("actual_fraction_b") is not None:
B2220 0204 |         try:
B2221 0205 |             fraction = float(record["actual_fraction_b"])
B2222 0206 |         except (TypeError, ValueError):
B2223 0207 |             return None, None, warnings, "SQS record has an invalid actual_fraction_b"
B2224 0208 |     elif (
B2225 0209 |         record.get("replaced_sites") is not None
B2226 0210 |         and record.get("available_substitution_sites") is not None
B2227 0211 |     ):
B2228 0212 |         try:
B2229 0213 |             available = int(record["available_substitution_sites"])
B2230 0214 |             replaced = int(record["replaced_sites"])
B2231 0215 |         except (TypeError, ValueError):
B2232 0216 |             return None, None, warnings, "SQS record has invalid substitution-site counts"
B2233 0217 |         if available <= 0:
B2234 0218 |             return None, None, warnings, "available_substitution_sites must be positive"
B2235 0219 |         fraction = replaced / available
B2236 0220 |         basis = "site_counts"
B2237 0221 |     else:
B2238 0222 |         fraction = target
B2239 0223 |         basis = "target_fallback"
B2240 0224 |         warnings.append("actual fraction unavailable; target_fraction_b was used")
B2241 0225 |
B2242 0226 |     if not math.isfinite(target) or not math.isfinite(fraction):
B2243 0227 |         return None, None, warnings, "SQS fraction is not finite"
B2244 0228 |     if not 0.0 <= target <= 1.0 or not 0.0 <= fraction <= 1.0:
B2245 0229 |         return None, None, warnings, "SQS fractions must be between 0 and 1"
B2246 0230 |     if not math.isclose(target, fraction, rel_tol=0.0, abs_tol=1e-12):
B2247 0231 |         warnings.append(
B2248 0232 |             f"actual_fraction_b={fraction:.12g} differs from target_fraction_b={target:.12g}"
B2249 0233 |         )
B2250 0234 |     return fraction, basis, warnings, None
```

```
src/ssscreen/stability/thermodynamics.py
```

```
B2251 0235 |
B2252 0236 |
B2253 0237 | def _base_row(sqs: dict[str, Any], pair_index: int | None) -> dict[str, Any]:
B2254 0238 |     row = {column: None for column in OUTPUT_COLUMNS}
B2255 0239 |     row.update(
B2256 0240 |         {
B2257 0241 |             "schema_version": MIXING_SCHEMA_VERSION,
B2258 0242 |             "pair_index": pair_index,
B2259 0243 |             "source_group_index": sqs.get("source_group_index"),
B2260 0244 |             "mp_id_a": sqs.get("mp_id_a"),
B2261 0245 |             "mp_id_b": sqs.get("mp_id_b"),
B2262 0246 |             "comp_a": sqs.get("comp_a"),
B2263 0247 |             "comp_b": sqs.get("comp_b"),
B2264 0248 |             "target_fraction_b": sqs.get("target_fraction_b"),
B2265 0249 |             "sqs_structure_id": sqs.get("structure_id"),
B2266 0250 |             "status": "invalid_sqs_metadata",
B2267 0251 |             "usable_for_screening": False,
B2268 0252 |             "warnings": "",
B2269 0253 |         }
B2270 0254 |     )
B2271 0255 |     backend = sqs.get("backend")
B2272 0256 |     if isinstance(backend, dict):
B2273 0257 |         row.update(
B2274 0258 |             {
B2275 0259 |                 "backend_name": backend.get("name"),
B2276 0260 |                 "backend_version": backend.get("version"),
B2277 0261 |                 "model_name": backend.get("model_name"),
B2278 0262 |                 "model_sha256": backend.get("model_sha256"),
B2279 0263 |                 "dtype": backend.get("dtype"),
B2280 0264 |                 "sqs_device": backend.get("device"),
B2281 0265 |             }
B2282 0266 |         )
B2283 0267 |     return row
B2284 0268 |
B2285 0269 |
B2286 0270 | def _failed_row(
B2287 0271 |     row: dict[str, Any], status: str, error: str, warnings: list[str]
B2288 0272 | ) -> dict[str, Any]:
B2289 0273 |     row.update(
B2290 0274 |         {
B2291 0275 |             "status": status,
B2292 0276 |             "usable_for_screening": False,
B2293 0277 |             "warnings": "; ".join(warnings),
B2294 0278 |             "error": error,
B2295 0279 |         }
B2296 0280 |     )
B2297 0281 |     return row
B2298 0282 |
B2299 0283 |
B2300 0284 | def calculate_mixing_enthalpies(
```

src/sssreen/stability/thermodynamics.py

```

B2301 0285 | *,
B2302 0286 | pairs_path: str | Path,
B2303 0287 | relax_results_path: str | Path,
B2304 0288 | output_path: str | Path,
B2305 0289 | summary_path: str | Path,
B2306 0290 | ) -> tuple[pd.DataFrame, dict[str, Any]]:
B2307 0291 | """Calculate per-SQS mixing enthalpies using compatible relaxed endmembers.
B2308 0292 |
B2309 0293 | The per-atom definition is ``E_sqs(x) - [(1-x) E_a + x E_b]``. Records are
B2310 0294 | never silently discarded: unusable or incompatible inputs produce rows with
B2311 0295 | an explicit non-success status and no mixing enthalpy.
B2312 0296 | """
B2313 0297 | pairs_path = Path(pairs_path)
B2314 0298 | relax_results_path = Path(relax_results_path)
B2315 0299 | output_path = Path(output_path)
B2316 0300 | summary_path = Path(summary_path)
B2317 0301 | pairs = _load_pairs(pairs_path)
B2318 0302 | relaxation_records = _read_jsonl(relax_results_path)
B2319 0303 |
B2320 0304 | endmembers: dict[str, list[dict[str, Any]]] = {}
B2321 0305 | sqs_records: list[dict[str, Any]] = []
B2322 0306 | for record in relaxation_records:
B2323 0307 |     role = record.get("structure_role")
B2324 0308 |     if role == "endmember":
B2325 0309 |         material_id = record.get("material_id")
B2326 0310 |         if isinstance(material_id, str) and material_id:
B2327 0311 |             endmembers.setdefault(material_id, []).append(record)
B2328 0312 |     elif role == "sqs":
B2329 0313 |         sqs_records.append(record)
B2330 0314 |
B2331 0315 | rows: list[dict[str, Any]] = []
B2332 0316 | for sqs in sqs_records:
B2333 0317 |     try:
B2334 0318 |         pair_index = int(sqs["pair_index"])
B2335 0319 |     except (KeyError, TypeError, ValueError):
B2336 0320 |         rows.append(
B2337 0321 |             _failed_row(
B2338 0322 |                 _base_row(sqs, None),
B2339 0323 |                 "invalid_sqs_metadata",
B2340 0324 |                 "SQS record has no valid pair_index",
B2341 0325 |                 [],
B2342 0326 |             )
B2343 0327 |         )
B2344 0328 |         continue
B2345 0329 |
B2346 0330 | row = _base_row(sqs, pair_index)
B2347 0331 | if pair_index < 0 or pair_index >= len(pairs):
B2348 0332 |     rows.append(
B2349 0333 |         _failed_row(row, "pair_not_found", f"pair_index {pair_index} is out of range", [])
B2350 0334 |     )

```

```
src/sssreen/stability/thermodynamics.py
```

```
B2351 0335 |         continue
B2352 0336 |
B2353 0337 |         pair = pairs.iloc[pair_index]
B2354 0338 |         row.update(
B2355 0339 |             {
B2356 0340 |                 "source_group_index": pair.get("source_group_index", row["source_group_index"]),
B2357 0341 |                 "mp_id_a": str(pair["mp_id_a"]),
B2358 0342 |                 "mp_id_b": str(pair["mp_id_b"]),
B2359 0343 |                 "comp_a": str(pair["comp_a"]),
B2360 0344 |                 "comp_b": str(pair["comp_b"]),
B2361 0345 |             }
B2362 0346 |         )
B2363 0347 |         for endpoint in ("a", "b"):
B2364 0348 |             sqs_id = sqs.get(f"mp_id_{endpoint}")
B2365 0349 |             if sqs_id is None or str(sqs_id) != row[f"mp_id_{endpoint}"]:
B2366 0350 |                 rows.append(
B2367 0351 |                     _failed_row(
B2368 0352 |                         row,
B2369 0353 |                         "pair_metadata_mismatch",
B2370 0354 |                         f"SQS mp_id_{endpoint} does not match pair row {pair_index}",
B2371 0355 |                         [],
B2372 0356 |                     )
B2373 0357 |                 )
B2374 0358 |             break
B2375 0359 |         else:
B2376 0360 |             if not _is_usable(sqs):
B2377 0361 |                 rows.append(
B2378 0362 |                     _failed_row(
B2379 0363 |                         row,
B2380 0364 |                         "unusable_sqs",
B2381 0365 |                         "SQS relaxation is not successful and usable for thermodynamics",
B2382 0366 |                         [],
B2383 0367 |                     )
B2384 0368 |                 )
B2385 0369 |             continue
B2386 0370 |
B2387 0371 |             signature, _ = _energy_signature(sqs)
B2388 0372 |             if signature is None:
B2389 0373 |                 rows.append(
B2390 0374 |                     _failed_row(
B2391 0375 |                         row,
B2392 0376 |                         "invalid_provenance",
B2393 0377 |                         "SQS record lacks a complete backend/settings energy reference",
B2394 0378 |                         [],
B2395 0379 |                     )
B2396 0380 |                 )
B2397 0381 |             continue
B2398 0382 |
B2399 0383 |             endpoint_a, status, error = _select_endmember(
B2400 0384 |                 row["mp_id_a"], endmembers=endmembers, energy_signature=signature
```

src/ssscreen/stability/thermodynamics.py

```
B2401 0385 |         )
B2402 0386 |         if endpoint_a is None:
B2403 0387 |             rows.append(_failed_row(row, status or "missing_endmember", error or "", []))
B2404 0388 |             continue
B2405 0389 |         endpoint_b, status, error = _select_endmember(
B2406 0390 |             row["mp_id_b"], endmembers=endmembers, energy_signature=signature
B2407 0391 |         )
B2408 0392 |         if endpoint_b is None:
B2409 0393 |             rows.append(_failed_row(row, status or "missing_endmember", error or "", []))
B2410 0394 |             continue
B2411 0395 |
B2412 0396 |         fraction, basis, warnings, fraction_error = _fraction_b(sqs)
B2413 0397 |         if fraction is None:
B2414 0398 |             rows.append(
B2415 0399 |                 _failed_row(
B2416 0400 |                     row,
B2417 0401 |                     "invalid_fraction",
B2418 0402 |                     fraction_error or "invalid SQS fraction",
B2419 0403 |                     warnings,
B2420 0404 |                 )
B2421 0405 |             )
B2422 0406 |             continue
B2423 0407 |
B2424 0408 |         energy_sqs = _finite_energy(sqs)
B2425 0409 |         energy_a = _finite_energy(endpoint_a)
B2426 0410 |         energy_b = _finite_energy(endpoint_b)
B2427 0411 |         if energy_sqs is None or energy_a is None or energy_b is None:
B2428 0412 |             rows.append(
B2429 0413 |                 _failed_row(
B2430 0414 |                     row,
B2431 0415 |                     "invalid_energy",
B2432 0416 |                     "one or more per-atom energies are missing or non-finite",
B2433 0417 |                     warnings,
B2434 0418 |                 )
B2435 0419 |             )
B2436 0420 |             continue
B2437 0421 |
B2438 0422 |         reference_energy = (1.0 - fraction) * energy_a + fraction * energy_b
B2439 0423 |         mixing_enthalpy = energy_sqs - reference_energy
B2440 0424 |         if mixing_enthalpy < 0.0:
B2441 0425 |             signal = "exothermic"
B2442 0426 |         elif mixing_enthalpy > 0.0:
B2443 0427 |             signal = "endothermic"
B2444 0428 |         else:
B2445 0429 |             signal = "neutral"
B2446 0430 |         endpoint_a_backend = endpoint_a["backend"]
B2447 0431 |         endpoint_b_backend = endpoint_b["backend"]
B2448 0432 |         row.update(
B2449 0433 |             {
B2450 0434 |                 "actual_fraction_b": fraction,
```

```
src/sssreen/stability/thermodynamics.py
```

```
B2451 0435 |         "fraction_basis": basis,
B2452 0436 |         "endmember_a_structure_id": endpoint_a["structure_id"],
B2453 0437 |         "endmember_b_structure_id": endpoint_b["structure_id"],
B2454 0438 |         "energy_sqs_per_atom_eV": energy_sqs,
B2455 0439 |         "energy_endmember_a_per_atom_eV": energy_a,
B2456 0440 |         "energy_endmember_b_per_atom_eV": energy_b,
B2457 0441 |         "reference_energy_per_atom_eV": reference_energy,
B2458 0442 |         "mixing_enthalpy_eV_per_atom": mixing_enthalpy,
B2459 0443 |         "mixing_enthalpy_meV_per_atom": mixing_enthalpy * 1000.0,
B2460 0444 |         "mixing_signal": signal,
B2461 0445 |         "endmember_a_device": endpoint_a_backend.get("device"),
B2462 0446 |         "endmember_b_device": endpoint_b_backend.get("device"),
B2463 0447 |         "energy_reference_sha256": signature,
B2464 0448 |         "status": "success",
B2465 0449 |         "usable_for_screening": True,
B2466 0450 |         "warnings": "; ".join(warnings),
B2467 0451 |         "error": None,
B2468 0452 |     }
B2469 0453 | )
B2470 0454 | rows.append(row)
B2471 0455 |
B2472 0456 | frame = pd.DataFrame.from_records(rows, columns=OUTPUT_COLUMNS)
B2473 0457 | status_counts = Counter(str(status) for status in frame["status"])
B2474 0458 | successful_pair_indices = sorted(
B2475 0459 |     {int(value) for value in frame.loc[frame["status"] == "success", "pair_index"]}
B2476 0460 | )
B2477 0461 | summary = {
B2478 0462 |     "schema_version": MIXING_SCHEMA_VERSION,
B2479 0463 |     "definition": "E_sqs(x) - [(1-x) E_a + x E_b] using per-atom energies",
B2480 0464 |     "pairs_path": str(pairs_path),
B2481 0465 |     "relax_results_path": str(relax_results_path),
B2482 0466 |     "output_path": str(output_path),
B2483 0467 |     "input_pair_count": int(len(pairs)),
B2484 0468 |     "relaxation_record_count": int(len(relaxation_records)),
B2485 0469 |     "sqs_record_count": int(len(sqs_records)),
B2486 0470 |     "endmember_material_count": int(len(endmembers)),
B2487 0471 |     "status_counts": dict(sorted(status_counts.items())),
B2488 0472 |     "success_count": int(status_counts.get("success", 0)),
B2489 0473 |     "failed_count": int(len(frame) - status_counts.get("success", 0)),
B2490 0474 |     "successful_pair_indices": successful_pair_indices,
B2491 0475 |     "missing_pair_indices": sorted(set(range(len(pairs))) - set(successful_pair_indices)),
B2492 0476 |     "target_fraction_fallback_count": int((frame["fraction_basis"] == "target_fallback").sum()),
B2493 0477 |     "provenance_policy": (
B2494 0478 |         "backend name/version, model SHA-256, dtype, and complete relaxation settings "
B2495 0479 |         "must match; compute device may differ"
B2496 0480 |     ),
B2497 0481 | }
B2498 0482 | _atomic_write_dataframe(frame, output_path)
B2499 0483 | _atomic_write_text(summary_path, json.dumps(summary, indent=2, sort_keys=True) + "\n")
B2500 0484 | return frame, summary
```